PAR
Selection of Exams (with Solutions)

Eduard Ayguadé, José R. Herrero,
Daniel Jiménez-González and Gladys Utrera

Departament d'Arquitectura de Computadors
Universitat Politècnica de Catalunya, UPC, BarcelonaTech

Course 2025-26 (Fall semester)

Contents

Part I

In-term Exams

PAR – In-Term Exam – Course 2022/23-Q1

November 3^{rd}, 2022

Problem 1 (3.0 points) Given the following code:

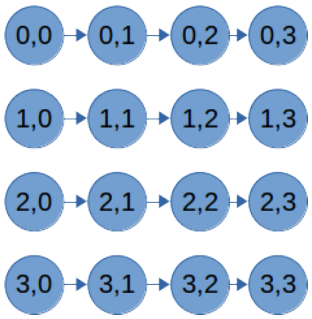
```
#define N  256
#define BS  64
int m[N][N];

for (int ii=0; ii<N/BS; ii++) {
   for (int jj=0; jj<N/BS; jj++) {
      tareador_start_task("task");

      // In general, a task processes a block of BSxBS elements
      // However, if the task is labeled with (ii,jj=0),
      //          this task processes BSx(BS-1) elements
      int i_start = ii*BS; int i_end = i_start+BS;
      int j_start = jj*BS; int j_end = j_start+BS;
      for (int i=i_start; i<i_end; i++)
        for (int j=max(j_start,1); j<j_end; j++)
          m[i][j] = compute (m[i][j], m[i][j-1]);  // tc t.u. (time units)

      tareador_end_task("task");
} }
```

Assume the innermost loop body takes t_c t.u., all variables but matrix m are in registers, function `compute` only reads the values received as arguments and does not modify other positions in the memory, BS perfectly divides N, being BS and N defined in the code above. **We ask you:**

1. (1.0 points) Draw the task dependence graph (TDG), indicating the cost of each task as a function of BS and t_c. Label each task with the values of ii and jj.

   Solution:

   Tasks (ii,jj) when jj!=0 have a cost of $BS \times BS \times t_c$. Tasks (ii,jj) when jj=0 have a cost $BS \times (BS-1) \times t_c$

2. (1.0 points) Compute T_1, T_∞, P_{min} as a function of BS, N and t_c.

   Solution:

   T_1 can be computed as:

   $T_1 = N \times (N-1) \times t_c$;

   T_∞ is the execution time of any of the rows of tasks of the TDG:

   $T_\infty = (BS-1) \times (BS) \times t_c + (\frac{N}{BS} - 1) \times (BS^2) \times t_c$;

This T_∞ is obtained when one processor is assigned to the computation of a row of tasks in the TDG. Therefore:

$P_{min} = \frac{N}{BS}$;

3. (1.0 points) Assuming the assignment of tasks to 4 processors of the table below, calculate T_4 and speedup S_4.

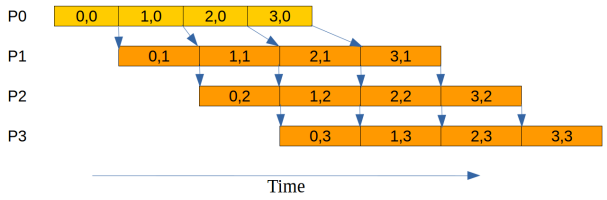
Processor	Tasks
P_0	$(0,0),(1,0),(2,0),(3,0)$
P_1	$(0,1),(1,1),(2,1),(3,1)$
P_2	$(0,2),(1,2),(2,2),(3,2)$
P_3	$(0,3),(1,3),(2,3),(3,3)$

Solution:

$S_4 = \frac{T_1}{T_4}$

$T_1 = N \times (N-1) \times t_c = 65280 \times t_c$ t.u.

Figure below shows the execution timeline considering the assignment of tasks to 4 processors above and their dependences. Each processor executes $\frac{N}{BS}$ tasks. Let's identify those that originate the critical path in the execution timeline. Processor 0 executes tasks with cost $BS \times (BS-1) \times t_c$. First task of Processor 0 contributes to T_4: $((BS-1) \times BS \times t_c)$. Then, first tasks of processors 1 an 2 also contribute to T_4: $((\frac{N}{BS} - 2) \times BS^2 \times t_c)$. Finally, all tasks executed in processor 3 contribute to T_4: $(\frac{N}{BS} \times BS^2 \times t_c)$.



$T_4 = ((BS-1) \times BS + (\frac{N}{BS} - 2) \times BS^2 + \frac{N}{BS} \times BS^2) \times t_c = 28608 \times t_c$ t.u.

$S_4 = \frac{T_1}{T_4} = 2.28\times$

Problem 2 (2.0 points) Given the same code and mapping of tasks to 4 processors as in the previous exercise, assume

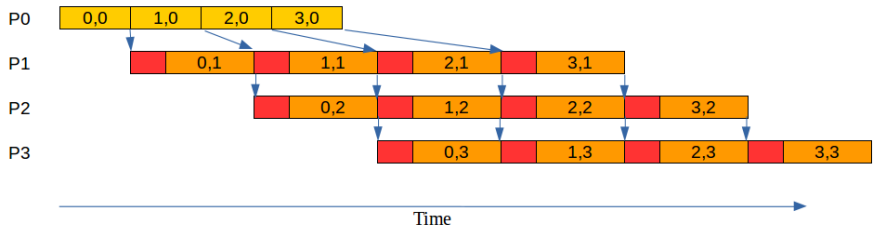
- A distributed-memory architecture with $P = 4$ processors;
- Matrix m is initially distributed by columns (BS consecutive columns per processor).
- Data sharing model with $t_{comm} = t_s + W \times t_w$, being W the number of elements to transfer, and t_s and t_w the start-up time and transfer time of one element, respectively;
- The execution time for a single iteration of the innermost loop body takes t_c t.u.

We ask you: Draw the execution timeline of the execution of tasks and write the expression that determines the execution time T_P, clearly indicating the contribution of the computation time $T_{P\,comp}$ and data sharing overhead $T_{P\,mov}$, as a function of N, BS, P, t_c, t_s and t_w.

Solution:

Figure below shows the execution timeline considering the assignment of tasks to 4 processors above, their dependences and the remote memory accesses. Processors 1, 2 and 3 have to wait for completness of tasks $ii, jj-1$ whose results are saved in the remote memory of the previous processor. As we are now considering data sharing overheads, processors 1, 2 and 3 have to perform a remote memory access of the BS elements of the left boundary before each task is executed. Therefore, the cost of each communication is $t_s + BS \times t_w$. Tasks in processor 0 perform local memory access with no data sharing overhead.

Each processor executes $\frac{N}{BS}$ tasks.



In particular, the contribution of computation and data sharing overheads is the following:

- Processor 0 executes tasks with cost $BS \times (BS-1) \times t_c$. First task of Processor 0 contributes to T_4: $(BS-1) \times BS \times t_c$.
- First tasks of processors 1 an 2 also contribute to T_4: $(\frac{N}{BS} - 2) \times ((t_s + BS \times t_w) + (BS^2 \times t_c))$.
- Finally, all tasks executed in processor 3 contribute to T_4: $(\frac{N}{BS}) \times ((t_s + BS \times t_w) + (BS^2 \times t_c))$.

$T_4 = T_{comp} + T_{comm}$, being:

$T_{comp} = ((BS-1) \times BS + (\frac{N}{BS} - 2) \times BS^2 + \frac{N}{BS} \times BS^2) \times t_c$

$T_{comm} = (\frac{N}{BS} - 2) \times (t_s + BS \times t_w) + \frac{N}{BS} \times (t_s + BS \times t_w) = (2 \times \frac{N}{BS} - 2) \times (t_s + BS \times t_w)$

Problem 3 (5.0 points) Consider a multiprocessor system with a hybrid NUMA/UMA architecture which is composed by 3 identical NUMAnodes. Each NUMAnode has 20 Gbytes of main memory and 2 processors with its own private cache of 8 Mbytes. The memory cache lines are 32 bytes wide, and data coherence is guaranteed using *Write-Invalidate MSI protocol* within each NUMAnode and using a *Write-Invalidate MSU Directory-based* cache coherency protocol among NUMAnodes.

We ask you to answer the following questions:

1. (1.0 points) Compute the total number of bits that are necessary **in each cache memory** to maintain the coherence, indicating the function of those bits.

   Solution:

   We need 2 state bits (MSI) for each line of cache memory. Each cache memory has $(8 \times 2^{20}) \div 32$ lines, that is 2^{18} lines; therefore the number of bits per cache is $2^{18} \times 2 = 2^{19}$ bits.

2. (1.0 points) Compute the total number of bits that are necessary **in each NUMAnode's directory** to maintain the coherence, indicating the function of those bits.

   Solution:

   We need 2 state bits (MSU) and 3 presence bits for each line of main memory. For the overall 20 GB, this is $(20 \times 2^{30}) \div 32$ lines, that is 20×2^{25} lines; therefore the number of bits in the directory is $20 \times 2^{25} \times (2 + 3) = 100 \times 2^{25}$ bits.

Given the following OpenMP code excerpt which is executed on processor 0 from the previous described multiprocessor system:

```
#define N (6*1024)
#define NUM_THREADS 6
int v[N], count[NUM_THREADS];
...
for (int k = 0; k < NUM_THREADS; k++)
   count[k]=0;

/*** POINT A ***/

#pragma omp parallel num_threads (NUM_THREADS)
{
   int id=omp_get_thread_num();
   int num_iter = N/NUM_THREADS;

   for (int k = id*num_iter; k < id*num_iter+num_iter; k++) {
       int value = v[k];
       if (is_prime(value)) /* returns true if "value" is a prime number */
           count[id]++;
   }
}
```

and assuming that: 1) the initial memory address of vector `count` is aligned to the start of a memory/cache line; 2) the size of an `int` data type is 4 bytes; and 3) processors 0 and 1 belong to NUMAnode0, processors 2 and 3 belong to NUMAnode1 and processors 4 and 5 belong to NUMAnode2; and 4) the Operating System applies the "first touch" policy for data allocation in memory. **We ask you to:**

3. (1.25 points) Complete the table in the provided answer sheet with the required information: affected cache line (numbered from the first position where vector `count` is allocated), cache line states (`I/S/M`) in processors 0 to 5, directory entry state (`U/S/M`) and presence bits (0/1, where the lowest ordered bit, the rightmost one, corresponds to NUMAnode0), to keep cache coherence, **when the execution of the previous code reaches POINT A**.

   Solution: The cache line size is 32 bytes, and each element of the vector occupies 4 bytes, so the entire vector `count` fits in a unique memory line.

	Affected line	Home NUMAnode	Cache line state						Directory entry	
			0	1	2	3	4	5	State	Presence bits
count[0]	0	0	M	-	-	-	-	-	M	001
count[1]	0	0	M	-	-	-	-	-	M	001
count[2]	0	0	M	-	-	-	-	-	M	001
count[3]	0	0	M	-	-	-	-	-	M	001
count[4]	0	0	M	-	-	-	-	-	M	001
count[5]	0	0	M	-	-	-	-	-	M	001

4. (1.25 points) Complete the table in the provided answer sheet with the required information: affected cache line, access in cache (hit/miss), CPU command for processor k ($PrRd_k/PrWr_k$), Bus transaction(s) from Snoopy in processor k ($BusRd_k$ / $BusRdX_k$ / $BusUpgr_k$ / $Flush_k$ / $Nothing$), cache line states in processors 0 to 5, directory entry state and presence bits, to keep cache coherence, **AFTER the execution of each** memory access in the table. Assume initially memory and cache states from previous question.

   Solution:

Memory access	Affected line	Hit/Miss	CPU command	Bus transaction(s)	Cache line state						Directory entry	
					0	1	2	3	4	5	State	Presence bits
Processor 1 reads count[1]	0	Miss	PrRd1	BusRd1 / Flush0	S	S	-	-	-	-	S	001
Processor 0 reads count[0]	0	Hit	PrRd0	Nothing	S	S	-	-	-	-	S	001
Processor 1 writes count[1]	0	Hit	PrWr1	BusUpgr1	I	M	-	-	-	-	M	001
Processor 2 reads count[2]	0	Miss	PrRd2	BusRd2 / Flush1	I	S	S	-	-	-	S	011
Processor 0 writes count[0]	0	Miss	PrWr0	BusRdX0	M	I	I	-	-	-	M	001

   Notice that, as the entirely vector `count` fits in a cache line, access to any element of the vector provokes access to the same cache line.

5. (0.5 points) Have you observed any potential efficiency problem in the previous code? Justify briefly your answer.

   Solution:

   False sharing: Vector `count` lies on a single memory line, then all the threads are eventually accessing for writing the same cache line during the concurrent execution of the program, but to different memory addresses, resulting in unnecessary coherence traffic, even between nodes.

   Possible memory bottleneck in the access to memory of processor 0: Vector count is entirely allocated in that memory, and all the threads have to access to it during execution.

Student name: .....

Answer for question 3.3

	Affected line	Home NUMAnode	Cache line state						Directory entry	
			0	1	2	3	4	5	State	Presence bits
count[0]										
count[1]										
count[2]										
count[3]										
count[4]										
count[5]										

Answer for question 3.4

Memory access	Affected line	Hit/Miss	CPU command	Bus transaction(s)	Cache line state						Directory entry	
					0	1	2	3	4	5	State	Presence bits
Processor 1 reads count[1]												
Processor 0 reads count[0]												
Processor 1 writes count[1]												
Processor 2 reads count[2]												
Processor 0 writes count[0]												

PAR – In-Term Exam – Course 2022/23-Q2

April 26^{th}, 2023

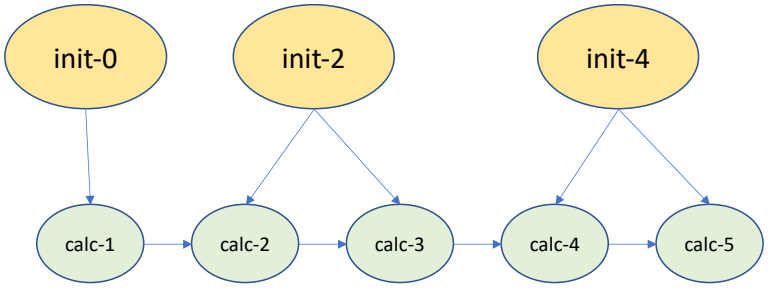
Problem 1 (5.0 points) Given the following code instrumented with *Tareador*:

```
#define N 6  // always a multiple of 2
int m[N][N];
...
// initialization
for (int i=0; i<N-1; i+=2) {   // loop step is 2
   sprintf(label, "init-%d", i);
   tareador_start_task(label);
   for (int k=0; k<N; k++) {
      m[i][k]   = foo(m[i][k], i);
      m[i+1][k] = foo(m[i][k], i);
   }
   tareador_end_task(label);
}

// calculation
for (int i=1; i<N; i++) {  // loop step is 1
   sprintf(label, "calc-%d", i);
   tareador_start_task(label);
   for (int k=0; k<N; k++)
      m[i][k] = goo(m[i-1][k], m[i][k]);
   tareador_end_task(label);
}
```

Assume that the execution time for each `init-i` and `calc-i` task is 20 and 4 time units, respectively, functions `foo()` and `goo()` do not perform any memory access and that the execution cost in time for the rest of the code is negligible. **We ask you to**:

1. (1.0 points) Draw the *Task Dependence Graph* (*TDG*) based on the above *Tareador* task definitions and for $N = 6$. Include for each task its name and its cost in time units. Notice that the outer loop in the initialization phase has a step value of 2.

   Solution:



2. (1.0 points) Compute the values for T_1, T_∞, the amount of *Parallelism* and P_{min}. Draw the temporal diagram for the execution of the *TDG* on P_{min} processors.

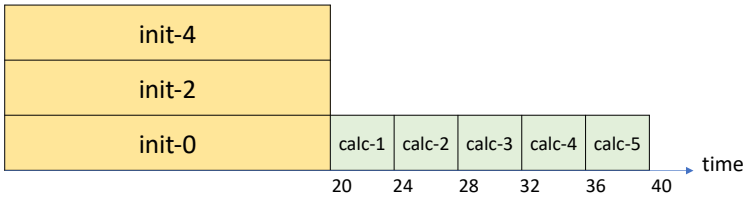
   Solution:

$T_1 = 20 \times 3 + 4 \times 5 = 80$ time units.

$T_\infty = 20 + 4 \times 5 = 40$ time units , determined by the critical path: *init-0, calc-1, calc-2, calc-3, calc-4, calc-5.*

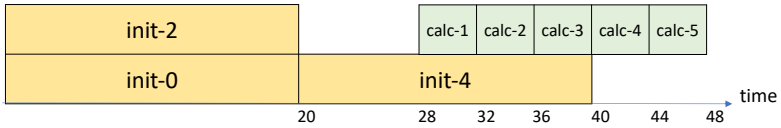
$Par = 80/40 = 2$

$P_{min} = 3$ as shown in the following temporal diagram.



3. (1.0 points) Indicate the best assignment for the tasks in the previous *TDG* to 2 processors that minimizes the computation time. Draw the temporal diagram showing the execution of the *TDG* with the proposed mapping. Compute the values for T_2 and S_2.

   Solution:



   $T_2 = 2 \times 20 + 2 \times 4 = 48$

   $S_2 = T_1/T_2 = 80/48 = 1.67$

4. (2.0 points) Consider a distributed memory architecture with P processors and N any value multiple of 2. Let's assume that matrix m is stored in the memory of processor 0 and that the assignment of task to the P processors is as follows:

   - *init-i* tasks are assigned to `P` different processors (i.e. the number of processors `P` is equal to the number of tasks *init-i*).
   - All *calc-i* tasks are assigned to processor 0

We ask you to: Write the expression that determines the execution time, T_p as a function of N (NOT as a function of P, since P is directly related with the value of N), clearly identifying the contribution of the computation time and the data sharing overheads, assuming the data sharing model explained in class in which the overhead to perform a remote memory access is $t_s + t_w \times m$, being t_s the start-up time, t_w the time to transfer one element and m the number of elements to be transferred; at a given time, a processor can only perform one remote access to another processor and serve one remote access from another processor.

Solution:

Let's write the expression for T_p considering on one hand the contribution from the computation part as T_{comp} and on the other the contribution from data sharing overhead as T_{ov}.

The critical path in the given task allocation to processors is given by the sequence: *init-0, calc-1, calc-2, calc-3, calc-4, calc-5.*

Consequently $T_{comp} = 20 + 4 \times (N - 1)$

The data sharing overhead impacts on T_p : 1) before the execution of all the *init-i* tasks (except for *init-0*), as matrix m is stored in the memory of processor 0; and after that 2) before the execution of all the *calc-i* tasks (except for *calc-0* which use the result data from init-0 on processor 0).

So we obtain from 1) $T_{ov1} = (N/2 - 1) \times (t_s + N \times t_w)$ We only need to access even rows. The odd rows will be generated in the initialization.

and from 2) $T_{ov2} = (N/2 - 1) \times (t_s + 2N \times t_w)$ We need updated even and odd rows.

Finally the expression for T_p is:

$T_p = 20 + 4 \times (N - 1) + (N/2 - 1) \times (t_s + N \times t_w) + (N/2 - 1) \times (t_s + 2N \times t_w)$

Problem 2 (5.0 points) Consider a multiprocessor system with a hybrid NUMA/UMA architecture composed of 2 identical NUMAnodes. Each NUMAnode has 16 Gbytes of main memory and 4 processors, each processor with its own private cache of 16 Mbytes. Memory cache lines are 16 bytes wide and data coherence is guaranteed using a *Write-Invalidate MSI protocol* within each NUMAnode and using a *Write-Invalidate MSU Directory-based* cache coherency protocol between NUMAnodes. Processors 0 to 3 belong to *NUMAnode0* and processors 4 to 7 to *NUMAnode1*. **We ask you to** answer the following two questions:

1. (1.0 point) Compute the total number of bits that are necessary **in each cache memory** to maintain the coherence, indicating the function of those bits.

   Solution:

   We need 2 state bits (MSI) for each line of cache memory. Each cache memory has $(16 \times 2^{20}) \div 16$ lines, that is 2^{20} lines; therefore the number of bits per cache is $2^{20} \times 2 = 2^{21}$ bits.

2. (1.0 point) Compute the total number of bits that are necessary **in each NUMAnode's directory** to maintain the coherence, indicating the function of those bits.

   Solution:

   We need 2 state bits (MSU) and 2 presence bits for each line of main memory. For the overall 16 GB, this is $(16 \times 2^{30}) \div 16$ lines, that is 2^{30} lines; therefore the number of bits in the directory is $2^{30} \times (2 + 2) = 2^{32}$ bits.

Given the following OpenMP code to be executed on the multiprocessor system described above:

```
#define NUM_THREADS 8
#define M 1
int hist[NUM_THREADS][M];
#pragma omp parallel num_threads(NUM_THREADS)
{
  int id=omp_get_thread_num();
  hist[id][0]=foo();
}
```

and assuming that: 1) no accesses to `hist` are performed before the execution of the parallel region; 2) the initial memory address of `hist` is aligned to the beginning of a memory/cache line and other variables are stored in registers; 3) the size of an `int` data type is 4 bytes; and 4) the Operating System applies the *"First touch"* policy for data allocation in memory. **We ask you to answer the following question:**

3. (0.25 points) How many directory entries will be required to store the coherence information of `hist` and which NUMAnodes will hold them after the execution of the previous parallel region?

   Solution:

   `hist` size (in bytes) is: $NUM\_THREADS * M * 4 = 8 * 1 * 4 = 32$ bytes. Therefore, we require two memory/cache lines of 16 bytes to fit the full `hist` variable; and two directory entries are required to store the coherence information. The NUMAnodes that will hold can be figured out looking at the program. Thread i runs in processor i. So, threads 0 to 3 are running in NUMAnode 0 and any touch to first memory line of `hist` variable (positions `hist[0][0]` to `hist[3][0]`) will make this line to be hold in NUMAnode 0. Threads 4 to 7 are runing in NUMAnode 1 and will make second line of `hist` variable to be hold in NUMAnode 1.

Considering the following execution order for the multiple instances of `hist[id][0]=foo()`: *id=0,2,4,6,1,3,5,7* and the fact that function `foo()` does not perform any memory access to any memory line, **we ask you to:**

4. (1.5 points) Complete the table in the provided answer sheet with the required information in its columns: affected $line_m$, hit or miss in cache, CPU command by processor k ($PrRd_k/PrWr_k$), bus transaction(s) by the Snoopy in processor k ($BusRd_k$ / $BusRdX_k$ / $BusUpgr_k$ / $Flush_k$ / $Nothing$), new line state ($I/S/M$) for the copies of $line_m$ in the affected processors, and directory entry information for the affected NUMAnode: number of node, state ($U/S/M$) and presence bits (0/1, where the lowest ordered bit, the rightmost one, corresponds to NUMAnode0).

After the execution of the previous parallel region processor 0 executes the following sequential code:

```
int sum = 0; // sum is stored in a register
for (int i=0; i<8; i++) // i is stored in a register
  sum += hist[i][0];
```

5. **We ask you to answer the following questions:**
   - (0.25 points) During the execution of the sequential code by processor 0, do you expect any coherence protocol transaction/s between the two NUMANodes?

     Solution:

     Yes. There should be a $RdReq_{0to1}$ and $DReply_{1to0}$ to obtain the second memory line of `hist`, hold in NUMAnode 1.
   - (0.25 points) Once the sequential code has been executed, which will be the home NUMAnode for the memory lines containing `hist` accessed by processor 0?

     Solution:

     Once a memory line is hold in a NUMAnode, its NUMAnode home will remain the same for the full execution. Therefore, no changes happen due to processor 0 accesses, and first and second line of `hist` remain in NUMAnode 0 and 1, respectively.
   - (0.25 points) Which will be the value of the state and presence bits in the directory for those memory lines?

     Solution:

     For the first line of `hist`, the coherence information in the directory entry is the following: state: shared and presence bits: 01, meaning that the memory line is shared with one or more clean copies in NUMAnode 0.

     For the second line of `hist`, the coherence information in the directory entry is the following: state: shared and presence bits: 11, meaning that the memory line is shared with one or more clean copies in NUMAnodes 0 and 1.

And finally,

6. (0.5 points) Do you observe any potential efficiency problem in the parallel region related to the cache coherence protocol? Briefly justify your answer and indicate a modification of the code that avoids this problem.

   Solution: There is false sharing accessing to `hist` variable. Threads are updating consecutive positions of a memory line. The modification should make each position of `hist[i][0]` to be in different memory lines. That can be done by making M constant to be 4. M equal 4 makes that each row of `hist` requires 16 bytes, which is the size of a full memory line, and then, accesses to `hist[i][0]` do not share memory lines.

Student name: .....

Answer for question 2.4

Time Unit		Affected $line_m$	hit or miss	Processor command	Bus transaction/s	Processor : Cache line state	Directory entry		
							NUMAnode #	State	Presence bits
1	hist[0][0]=foo()								
2	hist[2][0]=foo()								
3	hist[4][0]=foo()								
4	hist[6][0]=foo()								
5	hist[1][0]=foo()								
6	hist[3][0]=foo()								
7	hist[5][0]=foo()								
8	hist[7][0]=foo()								

Student name: .....

Answer for question 2.4

Time Unit		Affected $line_m$	hit or miss	Processor command	Bus transaction/s	Processor : Cache line state	Directory entry		
							NUMAnode #	State	Presence bits
1	hist[0][0]=foo()	0	miss	$PrWr_0$	$BusRdX_0$	0:M	0	M	01
2	hist[2][0]=foo()	0	miss	$PrWr_2$	$BusRdX_2$ $Flush_0$	2:M 0:I	0	M	01
3	hist[4][0]=foo()	1	miss	$PrWr_4$	$BusRdX_4$	4:M	1	M	10
4	hist[6][0]=foo()	1	miss	$PrWr_6$	$BusRdX_6$ $Flush_4$	6:M 4:I	1	M	10
5	hist[1][0]=foo()	0	miss	$PrWr_1$	$BusRdX_1$ $Flush_2$	1:M 2:I	0	M	01
6	hist[3][0]=foo()	0	miss	$PrWr_3$	$BusRdX_3$ $Flush_1$	3:M 1:I	0	M	01
7	hist[5][0]=foo()	1	miss	$PrWr_5$	$BusRdX_5$ $Flush_6$	5:M 6:I	1	M	10
8	hist[7][0]=foo()	1	miss	$PrWr_7$	$BusRdX_7$ $Flush_5$	7:M 5:I	1	M	10

PAR – In-Term Exam – Course 2023/24-Q1

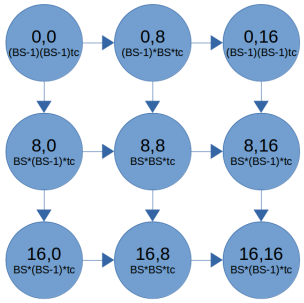
November 2^{nd}, 2023

Problem 1 (5.0 points) Given the following code:

```
#define BS 8
#define N  24
double A[N][N];
int ii, jj, i, j;
...

for (ii=0; ii<N; ii+=BS)
  for (jj=0; jj<N; jj+=BS) {
     tareador_start_task("Comp_ii_jj");
     for (i=max(1,ii); i<min(ii+BS,N); i++)
       for (j=max(1,jj); j<min(jj+BS,N-1); j++)
          A[i][j] = A[i][j-1] + A[i-1][j] + A[i][j+1]; // cost of this line: tc t.u.
     tareador_end_task("Comp_ii_jj");
  }
...
```

1. (1 point) Draw the Task Dependence Graph (TDG) based on the above Tareador task definitions and for `BS=8` and `N=24`. Each task should be clearly labeled with the values of ii, jj and its cost in time units.

   Solution:



2. (2.0 points) Compute the values for T_1, T_∞ and P_{min}. Draw the temporal diagram for the execution of the TDG in the previous question on P_{min} processors. As indicated, consider the cost of the innermost loop body to be t_c time units.

   Solution:

   $T_1 = (N-1)(N-2) \times t_c$

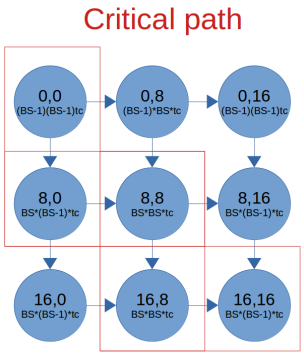
   $T_\infty = ((BS-1)^2 + (BS)(BS-1) + BS^2 + BS^2 + (BS)(BS-1)) \times t_c$

   $P_{min} = 3$

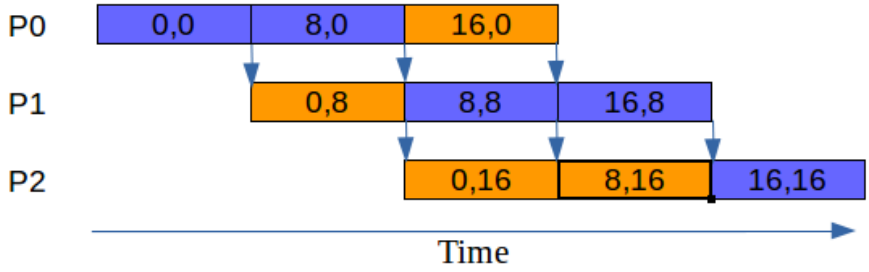
   Considering the particular values of `N` and `BS` provided in the statement of the exercise this translates into:

   $T_1 = 506 \times t_c$

   $T_\infty = 289 \times t_c$



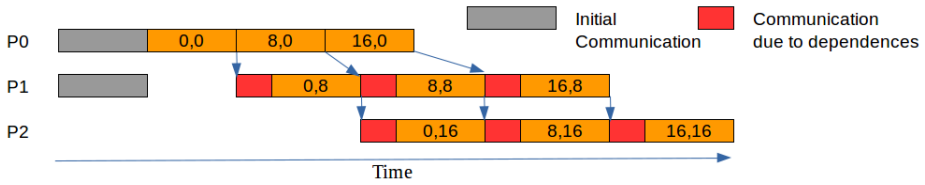
We consider $(BS-1) \approx BS$ in order to simplify the time diagram:



3. (2.0 points) Assume the same task definition, and consider a distributed memory architecture with P processors, $BS = N/P$, and N a very large value multiple of P (you can assume $BS-1$ is approximately the same as BS to simplify the model). Let's assume that matrix A is initially distributed by columns (N/P consecutive columns per processor) and tasks are scheduled so that a task is executed in the processor that stores the data the task has to update.

   We ask you to:

   (a) (1.0 points) Draw the time diagram for the execution of the tasks in P processors clearly identifying the computation and the data sharing time. Note: you can assume $P = 3$ for this question a).



   Initial communication corresponds to the access of the processor p to the 1st column, of N elements, of the processor $p+1$ due to access $A[i][j+1]$ (last processor has no initial communication). And for communications due to dependencies ($A[i][j-1]$), each processor p has to read a chunk of BS elements of the last column calculated by processor $p-1$ before starting each of its tasks (except for the first processor).

   (b) (1.0 points) Write the expression that determines the execution time, T_p as a function of N and P clearly identifying the contribution of the computation time and the data sharing overheads, assuming the data sharing model explained in class in which the overhead to perform a remote memory access is $t_s + t_w \times m$, being t_s the start-up time, t_w the time to transfer one element and m the number of elements to be transferred; at a given time, a processor can only perform one remote access to another processor and serve one remote access from another processor.

   Solution: Note: $(BS-1) \rightarrow BS$, and $BS = N/P$.

$$T_P = T_{comp} + T_{comm}$$
$$T_{task} = BS \times \frac{N}{P} \times t_c = \frac{N}{P} \times \frac{N}{P} \times t_c$$
$$T_{comp} = T_{task} \times (\frac{N}{BS} + P - 1) = T_{task} \times (P + P - 1)$$
$$T_{comm} = t_s + N \times t_w + (P + P - 2)(t_s + BS \times t_w)$$

Problem 2 (5.0 points) Given the following sequential recursive algorithm:

```
#define N 1024
#define NSTATES 128
#define MINROWS 2

int histogram[NSTATES];

int base_processing (int data[N][N], int start, int nrows) {
     int outofrange=0;

     for (int i=start; i<start+nrows; i++) {
       for (int k=0; k<N; k++) {
          int value = compute (data[i][k]); /* perform computation on the parameter */
          if (value >= NSTATES)
               outofrange++;
          else if (value >= 0)
               histogram[value]++;
       }
     }
     return outofrange;
}

int rec_processing (int data[N][N], int start, int nrows) {
    int res1, res2=0;

    if (nrows < MINROWS)
            res1 = base_processing (data, start, nrows);
    else {
            res1 = rec_processing (data, start, nrows/2);
            res2 = rec_processing (data, start+nrows/2, nrows-nrows/2);
    }
    return res1 + res2;
}

int main() {
  int data[N][N];
  ...
  int res = rec_processing(data, 0, N);
  ...
}
```

We ask you to answer the following **independent** questions:

1. (2.5 points) Write an OpenMP parallel version of the `base_processing` function, following an *Iterative* Task Decomposition, making use of the OpenMP explicit tasks. Your implementation should minimize synchronization overheads and take into account **the potential imbalance** generated by the `compute` function.

Some considerations about the all the proposed solutions:

- All of them have `parallel` and `single`.
- All of them create explicit tasks.
- All of them avoid creating tasks with only one iteration of k (too much task creation overhead).
- All of them avoid creating tasks doing full loop k (too much imbalance due to compute).
- All of them avoid waiting for all tasks at each iteration of i (we are looking for parallelism). I.e. nogroup should be used in the taskloop, but them reduction is not allowed.
- All of them perform a reduction of `outofrange` variable (avoid data race condition and reduce data synchronization).
- All of them perform an of `atomic` to update histogram (reduction could be possible also but it may be costy in memory).

Possible Solution 1: Using explicit tasks with hand-made reduction

```
#define GRANULARITY 4

int base_processing (int data[N][N], int start, int nrows) {
    int outofrange=0;
    #pragma omp parallel
    #pragma omp single
    {
      for (int i=start; i<start+nrows; i++) {
        for (int kk=0; kk<N; kk+=GRANULARITY) {
          #pragma omp task firstprivate(kk)
          {
           int tmp_outofrange = 0;
           for (int k=kk; k<min(kk+GRANULARITY,N); k++) {
            int value = compute (data[i][k]); /* perform computation on the parameter */
            if (value >= NSTATES)
                tmp_outofrange++;
            else if (value >= 0)
                #pragma omp atomic
                histogram[value]++;
           }
           #pragma omp atomic
           outofrange+=tmp_outofrange;
          }
        }
    }
   }
   return outofrange;
}
```

Some considerations about the implementation proposal:

- We do not create one task per each k-loop iteration to avoid too much task creation overhead. Instead, we have done strip-mining of the k-loop to create one task per each `GRANULARITY` number of iterations of k loop. This number of iterations has been set to 4.
- There is a possible data race condition updating variable `outofrange`. We do a hand-made reduction of variable `outofrange`: `tmp_outofrange` local variable is used to avoid data race conditions and, after the `GRANULARITY` iterations, we update global variable `outofrange`. This way only one atomic per task is paid (`GRANULARITY` times better than one atomic per k iteration).

- There is a possible data race condition updating variable `histogram[value]`. We use `atomic` to update it. The atomic protection on the histogram update may generate synchronization overhead depending on how often each position is updated. A workaround to reduce this synchronization would be to have local histograms for each task and combine it later, outside the loop. This could be much costly depending on the size of the histogram.

Possible Solution 2: Using explicit tasks with taskgroup and task_reduction

This solution is equivalent to previous one. In this solution `task_reduction` and `in_reduction` clauses are used instead of a hand-made reduction.

```
#define GRANULARITY 4

int base_processing (int data[N][N], int start, int nrows) {
   int outofrange=0;
   #pragma omp parallel
   #pragma omp single
   {
     #pragma omp taskgroup task_reduction(+:outofrange)
     {
      for (int i=start; i<start+nrows; i++) {
        for (int kk=0; kk<N; kk+=GRANULARITY) {
          #pragma omp task firstprivate(kk) in_reduction(+:outofrange)
          {
           for (int k=kk; k<min((kk+GRANULARITY),N); k++) {
            int value = compute (data[i][k]); /* perform computation on the parameter */
            if (value >= NSTATES)
                outofrange++;
            else if (value >= 0)
                #pragma omp atomic
                histogram[value]++;
           }
          }
        }
      }
     }
   }
   return outofrange;
}
```

Some considerations about the implementation proposal:

- `taskgroup` is not done at the kk-loop level because we do not want to wait for the sibling tasks at the end of each i iteration.

Possible Solution 3: Using taskloop in_reduction nogroup + taskgroup task_reduction
This solution includes a in_reduction, and it is equivalent to Solution 2.

```
#define GRANULARITY 4

int base_processing (int data[N][N], int start, int nrows)
{
    int outofrange=0;
#pragma omp parallel
#pragma omp single
{
  #pragma omp taskgroup task_reduction(+:outofrange)
  {
    for (int i=start; i<start+nrows; i++) {
          #pragma omp taskloop grainsize(GRANULARITY) firstprivate(i)\
                               in_reduction(+:outofrange) nogroup
          {
          for (int k=0; k<N; k++) {
            int value = compute (data[i][k]); /* perform computation on the parameter */
            if (value >= NSTATES)
                outofrange++;
            else if (value >= 0)
               #pragma omp atomic
                histogram[value]++;
          }
         }
    }
   }
}
    return outofrange;
}
```

Some considerations about the implementation proposal:

- `taskloop nogroup` is used to avoid waiting for all tasks at each i iteration.
- `taskloop in_reduction` is used instead of `reduction` due to the `nogroup` clause. This clause doesn't allow to use reduction alone. Instead, `in_reduction` makes each created task with `taskloop` contribute to the `task_reduction` of the `taskgroup`.

Alternative Non-expected Solution: Using taskloop with collapse - not seen at class

This solution includes a reduction, and it is equivalent to Solutions 2 and 3.

```
#define GRANULARITY 4
int base_processing (int data[N][N], int start, int nrows) {
    int outofrange=0;
#pragma omp parallel
#pragma omp single
#pragma omp taskloop collapse(2) reduction(+:outofrange) grainsize(GRANULARITY)
    for (int i=start; i<start+nrows; i++) {
        for (int k=0; k<N; k++) {
            int value = compute (data[i][k]); /* perform computation on the parameter */
            if (value >= NSTATES)
                outofrange++;
            else if (value >= 0)
                #pragma omp atomic
                histogram[value]++;
        }
    }
    return outofrange;
}
```

Some considerations about the implementation proposal:

- `taskloop collapse(2)` creates tasks of `GRANULARITY` iterations of the collapsed `i + k` loops. `collapse(2)` joins 2 loops (i and k) in only one loop with $nrows \times N$ iterations.

2. (2.5 points) Write an OpenMP parallel version of the sequential recursive program, following a *Recursive* Task Decomposition using the *Tree* strategy. The implementation should take into account the overhead due to task creation, by limiting their creation once a certain level in the recursive tree is reached.

Solution:

```
#define MAXDEPTH X  // Any value, not specified in the exercise

int base_processing (int data[N][N], int start, int nrows)
{
     int outofrange=0;

     for (int i=start; i<start+nrows; i++) {
        for (int k=0; k<N; k++) {
          int value = data[i][k];
          if (value >= NSTATES)
               outofrange++;
          else if (value >= 0)
               #pragma omp atomic
               histogram[value]++;
       }
     }
     return outofrange;
}

int rec_processing (int data[N][N], int start, int nrows, int depth)
{
    int res1, res2=0;

    if (nrows < MINROWS)
            res1 = base_processing (data, start, nrows);
    else {
         if (!omp_in_final()) {
            #pragma omp task final (depth >= MAXDEPTH) shared (res1)
            res1 = rec_processing (data, start, nrows/2, depth+1);
            #pragma omp task final (depth >= MAXDEPTH) shared (res2)
            res2 = rec_processing (data, start+nrows/2, nrows-nrows/2, depth+1);
            #pragma omp taskwait
         }
         else {
            res1 = rec_processing (data, start, nrows/2, depth);
            res2 = rec_processing (data, start+nrows/2, nrows-nrows/2, depth);
         }
    }
    return res1 + res2;
}

int main()
{
  int data[N][N];
  int outofrange;
  ...
  #pragma omp parallel
  #pragma omp single
  outofrange = rec_processing(data, 0, N, 0);
  ...
}
```

A recursive task descomposition with *Tree* strategy was applied to the code. In addition, to control

the task creation overhead, a cutoff is added, which allows the creation of tasks until the recursion level equal to MAXDEPTH is reached.

PAR – In-Term Exam – Course 2023/24-Q2

April 4^{th}, 2024

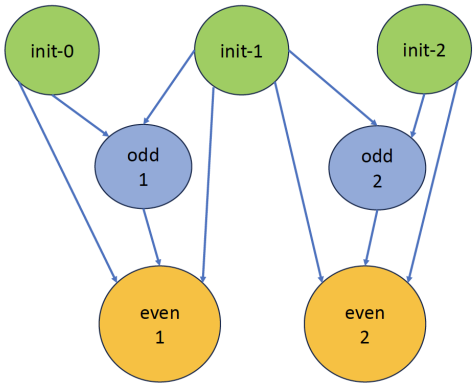
Problem 1 (4.0 points) Given the following code:

```
#define N  ... // value determined at each section
float A[N][N], B[N][N];
...
// initialization
 for (int i=0; i<N; i++) {
   tareador_start_task("init");
   for (k=0; k<N; k++) {
        A[i][k] = init(); // 10 time units
   }
   tareador_end_task("init");
}
// calculation
 for (int i=1; i<N; i++) {
   tareador_start_task("odd"); // Process only odd columns
   for (k=1; k<N; k+=2) {
        B[i][k] = A[i-1][k] + A[i][k] + foo(); // 20 time units
   }
   tareador_end_task("odd");
   tareador_start_task("even"); // Process only even columns
   for (k=2; k<N; k+=2) {
        B[i][k] = A[i-1][k] + A[i][k] + B[i][k-1] + goo(); // 40 time units
   }
   tareador_end_task("even");
 }
```

Assuming that functions `foo`, `goo` and `init` do not modify any element from matrix A and matrix B, **we ask you to:**

1. (1.0 points) Draw the Task Dependence Graph (TDG) based on the Tareador task definitions in the instrumented code above. Each task should be clearly labeled with the value of i and its cost in time units. Assume that $N = 3$.

   Solution:

   Where the cost for each `init` task is 30 time units, for each `odd` task is 20 time units and for each `even` task is 40 time units.

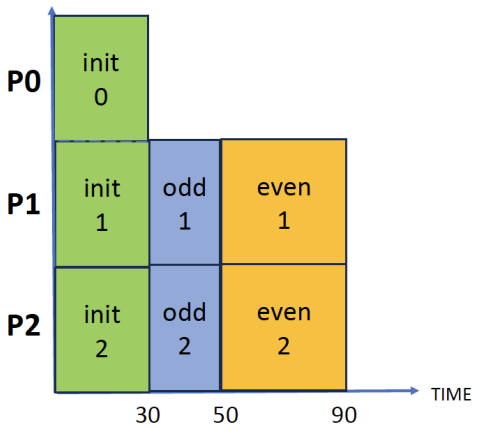
2. (1.0 points) Compute the values for T_1, T_∞ and P_{min}. Draw the temporal diagram for the execution of the TDG if executed using P_{min} processors.

Solution:

$T_1 = 30 \times 3 + (20 + 40) \times 2 = 210$ time units

$T_\infty = 30 + 20 + 40 = 90$ time units

$P_{min} = 3$

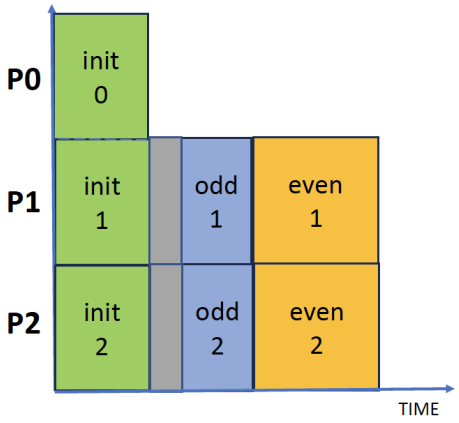


3. (2.0 points) Assume the same task definition, and consider a distributed memory architecture with P processors. Let's assume that matrix A and matrix B are distributed by rows along the P processors (row `i` on processor `i`). Tasks are scheduled on the processor where is allocated the data they have to update. This means that a task that works on row `i` executes on processor `i`.

   We ask you to:

   (a) (1.0 points) Draw the time diagram for the execution of the tasks in P processors clearly identifying the computation and the data sharing time. Note: to answer this question you can assume $N = 3$.

   Solution:



   The grey bars correspond to the data sharing time.

   (b) (1.0 points) Write the expression that determines the execution time T_p as a function of N and P. Identify clearly the contribution of the computation time and the data sharing overheads, assuming the data sharing model explained in class in which the overhead to perform a remote memory access is $t_s + t_w \times m$, being t_s the start-up time, t_w the time to transfer one element and m the number of elements to be transferred. Remember that at a given time, a processor can only perform one remote access to another processor and serve one remote access from another processor.

Solution:

$T_P = T_{comp} + T_{comm}$

It is necessary to distinguish if N is an odd or an even value:

i. If N is an odd value, the expression for T_{comp} is:
$T_{comp} = 10 \times N + (20 \times (N-1)/2 + 40 \times (N-1)/2)$

ii. If N is an even value, the expression for T_{comp} is:
$T_{comp} = 10 \times N + (20 \times N/2 + 40 \times (N-2)/2)$

$T_{comm} = ts + (N-1) \times t_w$

Every task `odd-i` and `even-i` needs 2 rows from matrix `A`. One of the rows is available in the local processor and the other has to be transferred from processor `i-1`, which is the one that counts for the data sharing overhead.

Problem 2 (3.0 points) Consider the following sequential code:

```
object_type *mat[256][256];
int histo[5] = {0, 0, 0, 0, 0};
int object_val(object_type *p); // return value of object *p in the range 0..4

int main() {
     for (int i=0 ; i<256; i++)
        for (int j=0; j<256; j++)
            histo[object_val(mat[i][j])]++;
}
```

that computes the histogram of the values in the range [0..4] related to the objects pointed by a square matrix of 256x256 elements.

We ask you to: Write an iterative task decomposition strategy using OpenMP that efficiently exploits the parallellism reducing task creation and synchronization overheads. You can propose a solution based on implicit or explicit tasks.

Solution with implicit tasks:

```
int main() {
   #pragma omp parallel
   {
    int BS = 256/omp_get_num_threads(); // assume nt divides 256
    int id = omp_get_thread_num();
    int histo_l[5] = {0,0,0,0,0};
    for (int i=id*BS ; i<(id+1)*BS; i++)
       for (int j=0; j<256; j++)
           histo_l[object_val(mat[i][j])]++;

    for (int i=0; i<5; i++)
       #pragma omp atomic
       histo[i] += histo_l[i];
   }
}
```

Solution with explicit tasks:

```
int main() {
   #pragma omp parallel
   #pragma omp single
   {
    int BS = 256/omp_get_num_threads();
    for (int ii=0; ii<256; ii+=BS)
    {
     #pragma omp task
     {
       int histo_l[5] = {0,0,0,0,0};
       for (int i=ii ; i<ii+BS; i++)
         for (int j=0; j<256; j++)
             histo_l[object_val(mat[i][j])]++;

       for (int i=0; i<5; i++)
          #pragma omp atomic
```

```
            histo[i] += histo_l[i];
      }

     }

    }
 }
```

Solution with explicit tasks - taskloop:

```
int main() {
   #pragma omp parallel
   #pragma omp single
   {
    #pragma omp taskloop num_tasks(omp_get_num_threads())
    for (int i=0 ; i<256; i++)
      for (int j=0; j<256; j++)
      {
          int tmp=object_val(mat[i][j]));
          #pragma omp atomic
          histo[tmp]++;
      }
   }
}
```

Problem 3 (3.0 points) Given the following sequential code:

```
unsigned int fibbonacci(unsigned int n) {
    unsigned int fib;
    if(n == 0)       fib = 0;
    else if(n == 1)  fib = 1;
    else {
       fib = fibbonacci(n-1);
       fib += fibbonacci(n-2);
    }
    return fib;
}
int main() {
   unsigned int fibnumber;
   ...
   fibnumber=fibbonacci(N)
   printf("Fibbonacci(%d)\n",fibnumber);
   ...
}
```

We ask you to:

1. (1.5 points) Create a parallel version in OpenMP using a recursive task decomposition for the `fibbonacci` function. Select the most appropriate strategy (*tree* or *leaf*) that will maximize the processor utilisation assuming a system with a high number of processors and without considering task creation and synchronization overheads.
2. (1.0 points) Modify the previous code to implement a task generation control mechanism based on the depth level. Use `MAX_DEPTH` as the maximum depth level to decide if tasks must be created or not.

3. (0.5 points) Assume your first parallel version. Which type of synchronization is required to guarantee that your tasks have finished before the `printf` statement in the main program if you can not assume any implicit or explicit thread barrier between the fibbonacci and `printf` calls?.

Solution a+b+c:

The correct parallel strategy is the tree recursive task decomposition.

The code already modified with the cutoff mechanism follows:

```
unsigned int fibbonacci(unsigned int n, int depth) {
    unsigned int fib1, fib2, fib;
    if(n == 0){
       fib = 0;
    } else if(n == 1) {
       fib = 1;
    } else {
      if (!omp_in_final())
      {
       #pragma omp task shared(fib1) final(depth>=MAX_DEPTH)
       fib1 = fibbonacci(n-1, depth+1);
       #pragma omp task shared(fib2) final(depth>=MAX_DEPTH)
       fib2 = fibbonacci(n-2, depth+1);
       #pragma omp taskwait
      }
      else
      {
       fib1 = fibbonacci(n-1, depth+1);
       fib2 = fibbonacci(n-2, depth+1);
      }
      fib = fib1+fib2;
    }
    return fib;
}

int main() {
   unsigned int fibnumber;
   ...

   #pragma omp parallel
   #pragma omp single
   {
      fibnumber=fibbonacci(N,0)
      // 3) nothing is needed because
      //    each parallel level has a taskwait synch
      printf("Fibbonacci(%d)\n",fibnumber);
   }
   ..
}
```

We do not need any taskwait neither taskgroup since each parallel level already waits for its tasks.

PAR – In-Term Exam – Course 2024/25-Q1

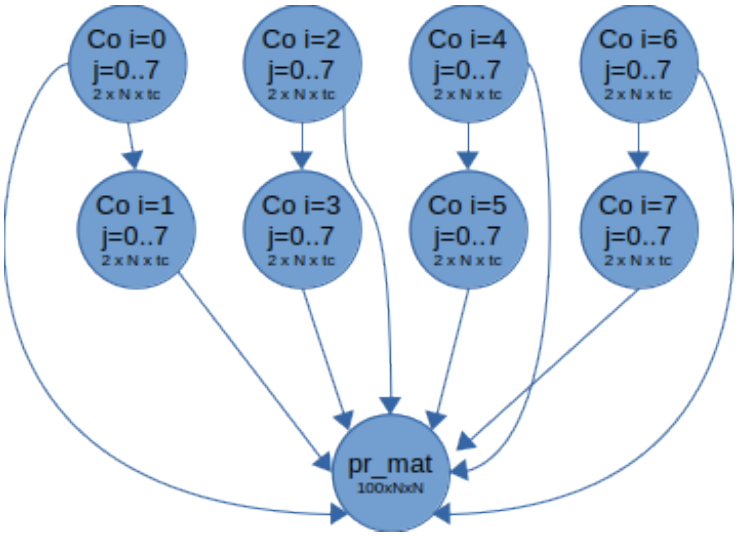
November 6^{th}, 2024

Problem 1 (4.0 points) Given the following code:

```
int mat[N][N];
void main() {
   for (int i=0; i<N; i++) {
     tareador_start_task("compute");
     for (int j=0; j<N; j++) {
        if (i%2 == 0)                          // cost of this line: tc t.u.
          mat[i][j] = mat[i][j] + mat[i+1][j]; // cost of this line: tc t.u.
        else
          mat[i][j] = mat[i][j] - mat[i-1][j]; // cost of this line: tc t.u.
     }
     tareador_end_task("compute");
   }
   tareador_start_task("print_matrix");
   print_matrix(mat);                          // cost of this function: 100*N*N t.u.
   tareador_end_task("print_matrix");
}
```

1. (1 point) Draw the Task Dependence Graph (TDG) based on the above Tareador task definitions and for `N=8`. Each task should be clearly labeled with the values of ranges of i, j, if needed, and its cost in time units (look at comments of the code to figure out the task cost lines).

   Solution:



   There are N `compute` tasks, one for each iteration of the outer loop `i`. The tasks computing even iterations have no Read After Write (RAW) dependencies. However, tasks computing odd iterations depend on the previous iteration. Each of these N `compute` tasks executes the whole set of iterations of the inner loop `j`, i.e. it performs the inner loop body N times. The associated cost of each task is $2 \times N \times t_c$ time units (which results in $16 \times t_c$ t.u. for $N = 8$) regardless of computing an even or odd iteration.

   A final task `print_matrix` prints the whole matrix. It depends on all the previous tasks since each row is previously updated by a different `compute` task. Its associated cost is $100 \times N^2$ time units.

2. (2.0 points) Assume `N=8` and the task costs of previous exercise. Compute the values for T_1, T_∞ and P_{min} and draw the temporal diagram for the execution of the TDG in the previous question on P_{min} processors.

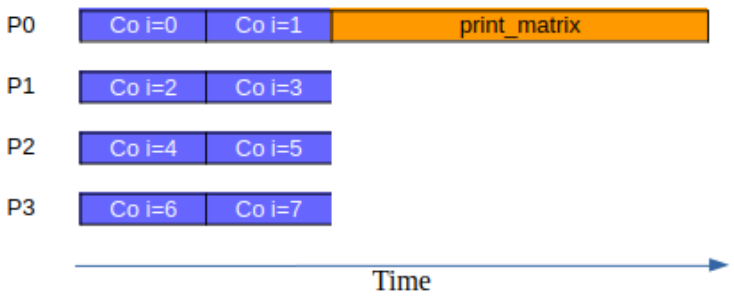
   Solution: Assuming `N=8`,

   $T_1 = N \times 2 \times N \times t_c + 100 \times N^2 = 128 \times t_c + 6400$

   $T_\infty = 2 \times 2 \times N \times t_c + 100 \times N^2 = \ \ 32 \times t_c + 6400$

   $P_{min} = 4$

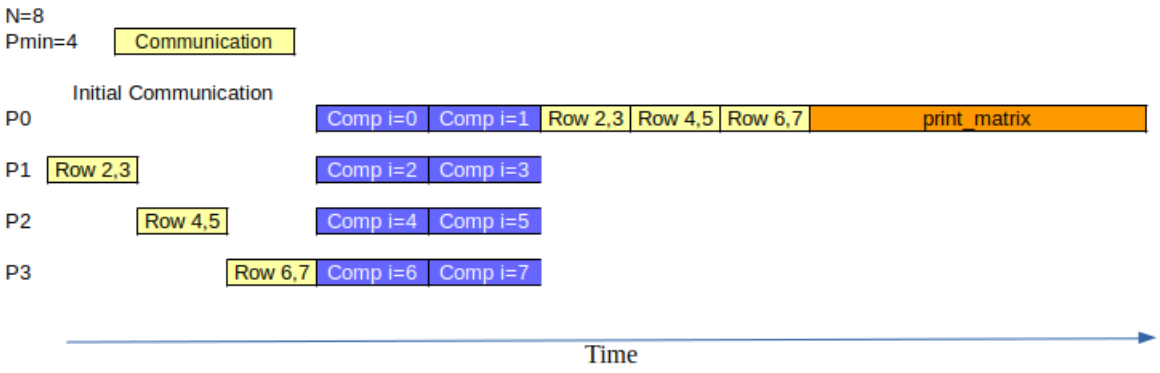
   Temporal diagram for the execution of the TDG in the previous question on $P_{min} = 4$ processors.



3. (1.0 points) Assume the same task definition, `N` a very large value multiple of 2, and consider a distributed memory architecture with $P = P_{min}$ processors. Let's assume that matrix `mat` is initially stored in the memory of processor 0 and tasks are scheduled to P processors in a similar way as you indicated in previous exercise, trying to minimize the data sharing overheads. Write the expression, function of N and $P = P_{min}$, that determines the execution time, T_p, clearly identifying the contribution of the computation time and the data sharing overheads, assuming the data sharing model explained in class in which the overhead to perform a remote memory access is $t_s + t_w \times m$, being t_s the start-up time, t_w the time to transfer one element and m the number of elements to be transferred; at a given time, a processor can only perform one remote access to another processor and serve one remote access from another processor.

   Solution:

   Next, we show a temporal diagram for the execution of the tasks in the TDG shown in the 1^{st} question on $P_{min} = 4$ processors, considering the data sharing overheads. This is an example of the execution of the application for $N = 8$, for which $P_{min} = 4$. Note that processors start their computation once the initial communication is done, as explained in class, to symplify the data sharing model.



   In general, $P_{min} = \frac{N}{2}$, where each processor executes the set of two `compute` tasks which compute consecutive even-odd iterations. And one of them executes the final task `print_matrix`.

$T_{Computations} = 2 \times 2 \times N \times t_c + 100 \times N^2$

Prior to performing any computation, $P_1, P_2 \ldots P_{min} - 1$ need to receive 2 consecutive rows from P_0. Consequently, since P_0 can only serve one remote access at a time:

$T_{Initial\_Data\_Sharing} = (P_{min} - 1) \times (t_s + 2N \times t_w)$

Regardless of the processor in which task `print_matrix` is executed, it'll need to receive 2 consecutive rows from each of the other $P_{min} - 1$ processors. Thus,

$T_{Final\_Data\_Sharing} = (P_{min} - 1) \times (t_s + 2N \times t_w)$

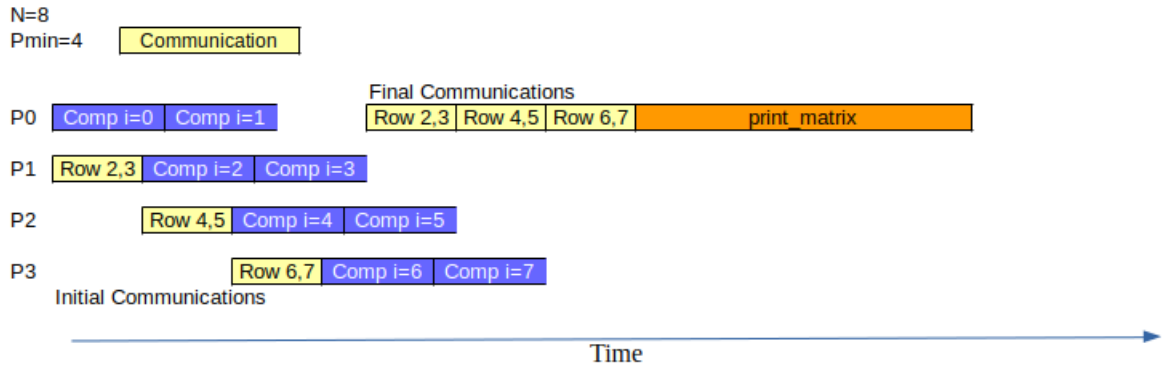
Therefore,

$$\begin{aligned} T_{P_{min}} &= T_{Initial\_Data\_Sharing} + T_{Computations} + T_{Final\_Data\_Sharing} \\ &= (P_{min} - 1) \times (t_s + 2N \times t_w) + 4 \times N \times t_c + 100 \times N^2 + (P_{min} - 1) \times (t_s + 2N \times t_w) \end{aligned}$$

$$\begin{aligned} &= 4 \times N \times t_c + 100 \times N^2 + 2 \times (P_{min} - 1) \times (t_s + 2N \times t_w) \\ &= 4 \times N \times t_c + 100 \times N^2 + 2 \times (\tfrac{N}{2} - 1) \times (t_s + 2N \times t_w) \\ &= 4 \times N \times t_c + 100 \times N^2 + (N - 2) \times (t_s + 2N \times t_w) \end{aligned}$$

Alternative optimized solution:

If we depart from the simplified model used in class and overlap communications with computations, then for $N = 8$:



And, in general:

$$\begin{aligned} T_{P_{min}} &= T_{Computations} + T_{Data\_Sharing} \\ &= 4 \times N \times t_c + 100 \times N^2 + (P_{min} - 1) \times (t_s + 2N \times t_w) + 1 \times (t_s + 2N \times t_w) \\ &= 4 \times N \times t_c + 100 \times N^2 + \tfrac{N}{2} \times (t_s + 2N \times t_w) \end{aligned}$$

Problem 2 (3.0 points) Consider the following sequential code:

```
int mat[N][N];

void main() {
    for (int i=0 ;i<N; i++)
      for (int j=0; j<N; j++)
        if (i%2 == 0)
             mat[i][j] = mat[i][j] + mat[i+1][j];
        else
             mat[i][j] = mat[i][j] - mat[i-1][j];
}
```

Assume N is a very large constant multiple of 2. Write an scalable OpenMP parallel program, being P the number of available processors, with the following conditions: 1) You are ONLY allowed to use explicit tasks, but NOT allowed to use taskloop; 2) The granularity of the task corresponds to the process of a chunk of BS consecutive rows of the matrix; 3) The synchronization needed to preserve the same result of the sequential execution, has to be minimized and 4) the overhead due to task creation and execution management has to be minimized.

Solution:

Analyzing the true dependencies in the sequential program we see that each odd row only depends on the previous (even) row. In this way, in order to minimize the synchronization we should guarantee that the chunk of rows assigned to a task should be an even number. In order to minimize the overhead of task management we could create a unique task for processing the maximum number of consecutive rows depending on N and P.

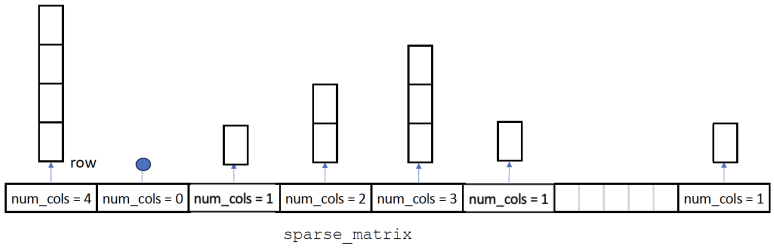
A possible solution:

```
int mat[N][N];

void main() {

 #pragma omp parallel num_threads(P)
 #pragma omp single
 {
    int BS = N/P;
    if (BS%2) BS++;
    for (int ii=0; ii<N; ii+=BS)
    #pragma omp task
    for (int i=ii; i<min(N,ii+BS); i++)
        for (int j=0; j<N; j++)
            if (i%2 == 0)
                mat[i][j] = mat[i][j] + mat[i+1][j];
            else
                mat[i][j] = mat[i][j] - mat[i-1][j];
 } }
```

We also could analyze the execution of each pair of even and odd rows and conclude that they could be processed jointly in the same iteration leading to the following code:

```
#pragma omp parallel num_threads(P)
#pragma omp single
{
    int BS = N/P/2;
    for (int ii=0; ii<N; ii+=BS*2)
    #pragma omp task
    for (i=ii; i<min(N,ii+BS*2); i+=2)
        for (j=0; j<N; j++) {
            int tmp = mat[i][j];
            mat[i][j] = mat[i][j] + mat[i+1][j];
            mat[i+1][j] = - tmp;
        }
```

Problem 3 (3.0 points) Given the following representation of a sparse matrix:



and the following data structures that model the sparse matrix with a sequential recursive algorithm to process it:

```
#define N ... // a large value and represents the number of rows
#define MIN_ROWS 2
#define MIN_NELEMS N
typedef {
    float value;
    int col;
} tRow;
typedef struct {
    int num_cols; // number of columns in the row
    tRow *row;
} tEntry;

// Perform calculation on first n rows of the sparse matrix.
float process (tEntry *sm, int n);

// Calculate pivot index (row index) such that the sum of num_cols (to process)
// up to row index (not included) is **similar** to the sum of num_cols (to process)
// at row index and after. This pivot index is returned in pointer int *index
// This total sum of num_cols before pivot index is also returned in *total_left
void partition (tEntry *sm, int n, int *index, int *total_left);

// Recursive calculation on the first n rows of sparse matrix sm
float rec_process (tEntry *sm, int n, int total_elems) {
    float result;
    if (n <= MIN_ROWS || total_elems <= MIN_NELEMS)
        result = process (sm, n);
    else {
        int index, total_left;
        partition (sm, n, &index, &total_left);
        result =  rec_process (sm, index, total_left);
        result += rec_process (&sm[index], n-index, total_elems-total_left);
    }
    return result;
}
int main() {
    tEntry sparse_matrix[N];
    int total_elems;
    ...
    float result = rec_process (sparse_matrix, N, total_elems);
    ...
}
```

We ask you to: Write a parallel OpenMP parallel version using a recursive task decomposition. Select the most appropriate strategy (tree or leaf) that will maximize the processor utilisation assuming a system with a high number of processors. The implementation should take into account the overhead due to task creation, limiting their creation and ensuring tasks are well-balanced.

Solution:

```
#define CUTOFF X // a value greater or equal than 2N
// Recursive calculation on the first n rows of sparse matrix sm
float rec_process (tEntry *sm, int n, int total_elems) {
    float result1, result2 = 0.0;
    if (n <= MIN_ROWS || total_elems <= MIN_NELEMS) {
        result1 = process (sm, n);
    }
    else {
        int index, total_left;
        partition (sm, n, &index, &total_left);
        if (!omp_in_final()) {
           #pragma omp task shared(result1) final (total_left < CUTOFF)
            result1 =  rec_process (sm, index, total_left);
           #pragma omp task shared(result2) final (total_elems-total_left < CUTOFF)
            result2 += rec_process (&sm[index], n-index, total_elems-total_left);
           #pragma omp taskwait
        }
        else {
            result1 =  rec_process (sm, index, total_left);
            result2 += rec_process (&sm[index], n-index, total_elems-total_left);
        }
    }
    return result1 + result2;
}
int main() {
    tEntry sparse_matrix[N];
    int total_elems;
    ...
    #pragma omp parallel
    #pragma omp single
    float result = rec_process (sparse_matrix, N, total_elems);
    ...
}
```

Notice that one task can achieve the cutoff condition, while the other one not, as the conditions are evaluated using different variables. This is in fact an advantage of marking a task as final individually, instead of performing a global condition.

PAR – In-Term Exam – Course 2024/25-Q1

April 3^{rd}, 2025
Grade Publication April 24^{th}, 2025 - Exam Review 25^{th} - 13:30h–14:30h - C6E101

Problem 1 (4.0 points) We have a code which reverses the contents of a vector:

```
#define min(a, b) ( (a < b) ? a : b )

#define   N    16    /* Problem size */
#define  BS     2    /* Block size for computation (reverse) tasks */
#define BS2  BS*2    /* Block size for initialization  tasks */

int main (int argc, char *argv[])
{
  int tmp, vect[N];

  tareador_ON ();

  for (int ii = 0; ii < N; ii += BS2)     /* Initialize vector */
    {
      tareador_start_task ("init");
      for (int i = ii; i < min(ii+BS2, N); i++)
        vect[i] = i;                      /* 10 t.u. per inner loop iteration */
      tareador_end_task ("init");
    }

  for (int ii = 0; ii < N/2; ii += BS)    /* Reverse vector contents */
    {
      tareador_start_task ("reverse");
      for (int i = ii; i < min(ii+BS, N/2); i++)
        {                                 /* 40 t.u. per inner loop iteration */
          tmp = vect[i];  vect[i] = vect[N - 1 - i];  vect[N - 1 - i] = tmp;
        }
      tareador_end_task ("reverse");
    }

  tareador_start_task ("print");
  for (int i = 0; i < N; i++)
    printf ("%d ", vect[i]);              /* 10 t.u. per inner loop iteration */
  tareador_end_task ("print");
  printf ("\n\n ");

  tareador_OFF ();
}
```

The code has three phases: 1) vector initialization; 2) vector reversal; 3) print resulting vector. Let's assume that each iteration of the inner loops of each of the phases has the cost specified as a comment in the code above (10, 40 and 10 time units respectively); and the rest of the code has negligible associated costs. Note that the loop in the reversal phase swaps elements between the left and right part of the vector and only has $N/2$ iterations.

1. (1 point) Considering the original code without instrumentation with `Tareador`, give an upper bound on the achievable speedup if we parallelize the intialization and vector reversal phases while keeping the final print phase sequential.

   Solution: Amdahl's Law is useful for obtaining an upper bound on the achievable speedup of a parallel program. The theoretical maximum speedup is limited by the serial portion of the workload.

The execution time of the parts that we want to parallelize, T_{par}, i.e. the initialization and reversal phases, will add up to: $T_{par} = 10 \times N$ t.u. $+ 40 \times N/2$ t.u. $= 10 \times N$ t.u. $+ 20 \times N$ t.u. $= 30 \times N$ t.u.

The final part, which prints the vector, will remain sequential. Its execution time, T_{seq}, will be: $T_{seq} = 10 \times N$ t.u.,

The total sequential execution time: $T_1 = T_{par} + T_{seq} = 30 \times N$ t.u. $+ 10 \times N$ t.u. $= 40 \times N$ t.u.

Therefore, the *parallel fraction* is: $\varphi = T_{par}/T_1 = \frac{30 \times Nt.u.}{40 \times Nt.u.} = \frac{3}{4} = 0.75$

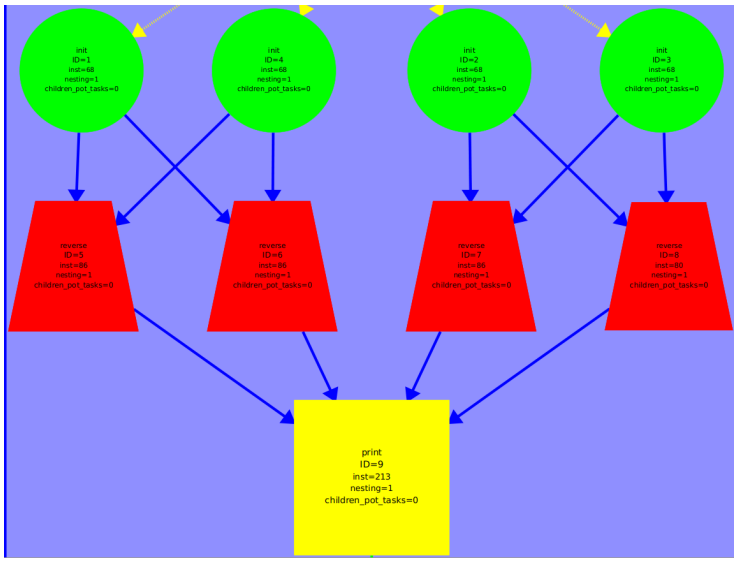
When $P \to \infty$ the expression of the speedup becomes:

$S_{p \to \infty} = \frac{1}{(1-\varphi)} = \frac{1}{(1-0.75)} = 4$

Consequently, the upper bound on the speedup is 4.

2. (1 point) In view of the instrumentation with `Tareador` shown above, draw a *Task Dependence Graph (TDG)* for the values N, BS and BS2 defined within the code. Each task must be clearly labeled with the label provided to `Tareador` in the instrumentation, plus an identifier representing the order in which each task is created, as automatically done by `Tareador`. the first task being created should have identifier 1. You do not need to include the initial and final tasks representing `Tareador`.

   Solution:



   The tasks which manipulate the first and last quarter of the vector are shown on the left side of the TDG. The ones manipulating the 2nd and 3rd quarter are shown on the right part.

3. (1 point) Give T_1, T_∞, *Parallelism*, P_{min} for the values of N and BS defined in the code above.

   Solution:

   $T_1 = (10 \times N + 40 \times N/2 + 10 \times N)$ t.u. $= 40 \times N$ t.u. $= 40 \times 16$ t.u $= 640$ t.u.

   The *critical path* consists of one initialization task, which performs $BS \times 2$ iterations at cost 10 t.u. each; one reverse task, which performs BS iterations at cost 40 t.u. each; and the final print task:

   $T_\infty = (10 \times (BS \times 2) + 40 \times BS + 10 \times N)$ t.u. $= (60 \times BS + 10 \times N)$ t.u.

   Thus,

   $T_\infty = (60 \times 2 + 10 \times 16)$ t.u. $= 280$ t.u.

   For the values of N and BS defined in the code above:

   $Parallelism = \frac{T_1}{T_\infty} = \frac{640}{280} = 2.28$

All paths in the TDG have the same associated cost. Therefore, all have to be executed simultaneously to avoid any delay in executing the critical path(s). Both the initialization and reversal phases create $\frac{N}{BS \times 2}$ tasks which is therefore the value of P_{min}. For the values of N and BS above:

$P_{min} = \frac{N}{BS \times 2} = 4$

4. (1 point) Independently of the results of the previous question, let's now assume that the initialization and reversal phases are perfectly parallelizable and we have a parallel system with a number of processors P with which we can achieve maximum parallelism for all the parallel phases. However, we wonder which would be the contribution of *data sharing*, $T_{Data\_Sharing}$, to the total parallel execution time if our parallel system was a *distributed memory system*. For this, let's assume the data sharing model explained in class in which the overhead to perform a remote memory access is $t_s + t_w \times m$, being t_s the start-up time, t_w the time to transfer one element and m the number of elements to be transferred; at a given time, a processor can only perform one remote access to another processor and serve one remote access from another processor.

   Consider that we have already distributed the vector initially by assigning N/P consecutive vector elements to each of the P processors, and P, BS and $2 \times BS$ perfectly divide N. In view of the TDG drawn before and considering that in each parallel phase we achieve perfect parallelism, we ask you, which would be the upper bound on the contribution of data sharing $T_{Data\_Sharing}$ to the total parallel execution time? Expres it as a function of t_s, t_w, P, N and/or BS.

   Solution: $T_{Data\_Sharing} \leq 2 \times (t_s + BS \times t_w) + (P-1) \times (t_s + 2 \times BS \times t_w)$

   The initialization tasks are the first ones which write the vector in memory. Each of these tasks initializes $2 \times BS$ consecutive vector elements. Data is usually allocated in the memory of the processor from which data is written for the 1st time. Since the number of processors P allows maximum parallelism, each of the initialization tasks will be executed on a different processor. We will need $P = \frac{N}{BS \times 2}$ processors. After their execution, each of these P processors will have $2 \times BS$ consecutive vector elements in its local memory.

   Each reversal task swaps elements from a set of BS consecutive elements in the 1^{st} part of the vector and BS consecutive elements in the 2^{nd} part of the vector.

   Since the initilialization is perfectly parallel, elements of both parts of the vector are necessarily initialized in different tasks, and this means different processors. Thus, at most, a reversal task can find BS elements in the memory of the processor where it is executed. The best case would correspond to an assignment of tasks where all tasks already find BS elements in the local memory, having to access the other BS elements on the other half of the vector from some remote memory. In this case, a single remote access would be needed. Such access would have a cost of $t_s + BS \times t_w$ time units. Having all tasks assigned to different processors with perfect parallelism and needing to retrieve data from different processors would imply that all such accesses could happen simultaneously in time. However, we are asked for the upper bound: the worst case would correspond to executing a reversal task in a processor that does not have in its memory any of the two parts of the vector which are needed, each of size BS. Thus, each processor may need to perform 2 remote accesses, each of size BS. And in our model, those 2 remote accesses must be performed sequentially. However, all remote accesses among different processors could be performed in parallel. Consequently, the contribution to data sharing time would be $2 \times (t_s + BS \times t_w)$ time units.

   The final print task needs all the vector. Let's assume that this task is executed in one of the P processors. Then, $2 \times BS$ elements will already be present in local memory. However, the rest need to be retrieved from the remote memory of the other $P-1$ processors, each of them providing $2 \times BS$ elements.

   Therefore, the upper bound on the contribution of data sharing to the parallel execution time is:

   $T_{Data\_Sharing} \leq 2 \times (t_s + BS \times t_w) + (P-1) \times (t_s + 2 \times BS \times t_w)$

Problem 2 (3.0 points) Given the following C code:

```
#define N 1024
#define T 1000
double A[N], B[N];
double norm=0.0;
double do_work (double a, double b);  // 10 time units
double calc (double a, double b);     // 5 time units
...
for (int t = 0; t < T; t++) {
    for (int i = 1; i < N-1; i++) {
        if (t % 2 == 0) {
            A[i] = do_work (B[i-1], B[i+1]);
        } else {
            B[i] = do_work (A[i-1], A[i+1]);
        }
        norm += calc (A[i],B[i]);
    }
}
```

Assume that: a) function `do_work` executes in 10 time units, function `calc` executes in 5 time units and that the cost of the rest of the code is negligible; b) no element of vectors `A` and `B` is modified within those functions. Let's define the *granularity* of a task as its cost in *time units*.

We ask you to: Without modifying the sequential base code, write **TWO** *OpenMP* parallel versions of the code, following an *Iterative Task Decomposition* and making use of the *OpenMP* explicit tasks, such that:

1. In order to control task creation overhead, the generated tasks should have granularity of 120 time units. The proposed solution should also minimize synchronization overheads.

   Solution:

   ```
   #define N 1024
   #define T 1000
   double A[N], B[N];
   double norm=0.0;
   double do_work (double a, double b);  // 10 time units
   double calc (double a, double b);     // 5 time units
   ...
   #pragma omp parallel
   #pragma omp single
    for (int t = 0; t < T; t++) {
       #pragma omp taskloop reduction(+:norm) grainsize(8)
        for (int i = 1; i < N-1; i++) {
            if (t % 2 == 0) {
                A[i] = do_work (B[i-1], B[i+1]);
            } else {
                B[i] = do_work (A[i-1], A[i+1]);
            }
            norm += calc (A[i],B[i]);
        }
    }
   ```

   Using the *taskloop* clause to define the explicit tasks, allow us to define the granularity (*grainsize*), a reduction to prevent the data race of the norm variable and an implicit barrier at each `t-loop` iteration as there are dependencies between them, and all these without increasing the complexity of the code.

   Potentially *grainsize* should work as described. However, depending on the implementation the *num_tasks* clause would be a more secure way to define the same: *num_tasks* $((N-1)/8)$

The cost of each inner iteration (`i-loop`) is calculated as the sum up of the costs of `do_work` and `calc` functions, which results in 15 time units.

Then: $120/15 = 8$

Consequently to have tasks of granularity 120 time units, each generated task must execute a bunch of 8 iterations.

The data race on the norm variable is protected with the *reduction* clause. An *atomic* clause would be an option, but it would introduce more synchronization overhead than the *reduction* clause.

An implementation using *task* it is also valid, but it would require to implement manually the previously commented restrictions, as shown in the following code:

```
#define N 1024
#define T 1000
double A[N], B[N];
double norm=0.0;
double do_work (double a, double b);  // 10 time units
double calc (double a, double b);     // 5 time units
...
#define MIN(a,b) (((a)<(b))?(a):(b))
int CHUNK = 8;
#pragma omp parallel
#pragma omp single
 for (int t = 0; t < T; t++) {
   #pragma omp taskgroup task_reduction(+:norm)
   {
    for (int ii = 0; ii < N-1; ii += CHUNK) {
      #pragma omp task in_reduction (+:norm)
       for (int i = ii; i <  MIN(ii+CHUNK, N-1); i++) {
          if (t % 2 == 0) {
              A[i] = do_work (B[i-1], B[i+1]);
          } else {
              B[i] = do_work (A[i-1], A[i+1]);
          }
          norm += calc (A[i],B[i]);
       }
    }
   }
 }
```

2. There is no control of task creation overhead and the proposed solution should: exploit the maximum parallelism and minimize synchronization overheads. As an orientation, the task granularity should be no more than 10 time units.

   Solution:

```
#define N 1024
#define T 1000
double A[N], B[N];
double norm=0.0;
double do_work (double a, double b);  // 10 time units
double calc (double a, double b);     // 5 time units
...
#pragma omp parallel
#pragma omp single
#pragma omp taskgroup task_reduction(+:norm)
 for (int t = 0; t < T; t++) {
     for (int i = 1; i < N-1; i++) {
         if (t % 2 == 0) {
```

```
                #pragma omp task depend (out: A[i]) depend (in:B[i-1],B[i+1])
                 A[i] = do_work (B[i-1], B[i+1]);
            } else {
                #pragma omp task depend (out: B[i]) depend (in:A[i-1],A[i+1])
                 B[i] = do_work (A[i-1], A[i+1]);
            }
            #pragma omp task depend (in: A[i], B[i]) in_reduction(+:norm)
             norm += calc (A[i], B[i]);
        }
    }
```

In order to generate tasks with cost of 10 time units or less, we need to define a task for each function `do_work` and `calc`. In addition to this we must preserve the order of execution among tasks. To that aim, we define dependencies to explicitly define the order between the individual tasks, and taking as a reference the input parameters of the functions. An *taskwait* clause would enforce a task ordering between `do_work` and `calc`, but serializing the i-loop so no parallelism is achieved. Finally, to avoid data race on the norm variable, a reduction clause is manage at the *taskgroup* level.

Problem 3 (3.0 points) Given the following incomplete parallel OpenMP code:

```
#define MIN_SIZE 1024
#define N (1 << 22)

int swap_pairs(char* vector, int n) {
    char tmp;
    int i, n4, count=0;
    if (n<MIN_SIZE) {
       for (i=0; i<n-1;i+=2) {
         if (vector[i]==vector[i+1]) count++;
         else { tmp = vector[i]; vector[i]=vector[i+1]; vector[i+1]=tmp; }
       }
    }
    else {
        n4 = n/4;
        count = swap_pairs(vector, n4);
        count += swap_pairs(vector+n4, n4);
        count += swap_pairs(vector+2*n4, n4);
        count += swap_pairs(vector+3*n4, n4);
    }
    return count;
}

void main()
{
   char *vector;
   vector = (char *)malloc(N*sizeof(char));
   int count_equal_pairs;
   ...
   #pragma omp parallel
   {
      ...

      count_equal_pairs = swap_pairs(vector,N);

      printf("Swap pairs done. Number of equal pairs: %d\n",count_equal_pairs);

      ...
   }

}
```

We ask you to complete the parallel OpenMP code of the main program and use a recursive task decomposition strategy for function `swap_pairs`. Select the most appropriate strategy (tree or leaf) that will maximize the processor utilisation assuming a system with a high number of processors. The implementation should take into account the overhead due to synchronizations and task creation, limiting their creation.

Solution

```
#define MIN_SIZE 1024
#define N (1 << 22)
#define CUTOFF 5


int swap_pairs(char* vector, int n, int depth) {
    char tmp;
    int i, n4, count=0, count1, count2, count3, count4;
    if (n<MIN_SIZE) {
```

```
        for (i=0; i<n-1;i+=2) {
          if (vector[i]==vector[i+1]) count++;
          else { tmp = vector[i]; vector[i]=vector[i+1]; vector[i+1]=tmp; }
        }
     }
     else {
         n4 = n/4;
         if (!omp_in_final())
         {
           #pragma omp task shared(count1) final(depth>=CUTOFF)
           count1 = swap_pairs(vector, n4, depth+1);
           #pragma omp task shared(count2) final(depth>=CUTOFF)
           count2 = swap_pairs(vector+n4, n4, depth+1);
           #pragma omp task shared(count3) final(depth>=CUTOFF)
           count3 = swap_pairs(vector+2*n4, n4, depth+1);
           #pragma omp task shared(count4) final(depth>=CUTOFF)
           count4 = swap_pairs(vector+3*n4, n4, depth+1);
           #pragma omp taskwait
         }
         else
         {
           count1 = swap_pairs(vector, n4, depth);
           count2 = swap_pairs(vector+n4, n4, depth);
           count3 = swap_pairs(vector+2*n4, n4, depth);
           count4 = swap_pairs(vector+3*n4, n4, depth);
         }
         count = count1+count2+count3+count4;
     }
     return count;
}

void main()
{
   char *vector;
   vector = (char *)malloc(N*sizeof(char));
   int count_equal_pairs;
   ...
   #pragma omp parallel
   #pragma omp single
   {
   ...
   count_equal_pairs = swap_pairs(vector,N,0);
   // We do not need any sychronization because each level of the parallel
   // recursion has its taskwait
   printf("Swap pairs done. Number of equal pairs: %d\n",count_equal_pairs);
   ...

   }
}
```

Part II

Final Exams

PAR – Final Exam – Course 2021/22-Q1

January 17^{th}, 2022

Problem 1 (2.5 points)

The following code computes matrix `u[N][N]` by blocks of $BS \times N$ elements, with `N` very large:

```
double u[N][N];

// Compute elements in a block of BS x N elements
void compute_row_block(int ii) {
    for (int i=max(1, ii); i<min(ii+BS, N-1); i++)
        for (int j=1; j<N-1; j++) {
            double tmp = u[i+1][j] + u[i-1][j] + u[i][j+1] + u[i][j-1] - 4*u[i][j];
            u[i][j] = tmp/4;
        }
}

void main() {
    int NB = 2*P;          // Total number of row blocks
    int BS = N/NB;         // Number of rows per block

    // EVEN loop: traversing all EVEN blocks
    for (int ii=0; ii<NB; ii+=2)
        compute_row_block(ii*BS);

    // ODD loop: traversing all ODD blocks
    for (int ii=1; ii<NB; ii+=2)
        compute_row_block(ii*BS);
}
```

In this code the computation is divided in two parts: the so called *EVEN* loop computing half of the blocks first (blocks 0, 2, ...), and the so called *ODD* loop computing the other half of the blocks later (blocks 1, 3, ...). Before answering the first question below, think about the parallel execution opportunities in this code when defining each iteration of the *EVEN* and *ODD* loops as a task. Could all the tasks in the *EVEN* loop be executed in parallel? Could all the tasks in the *ODD* loop be executed in parallel? And could the execution of tasks in the *ODD* loop be performed in parallel with the execution of tasks in the *EVEN* loop?. Having all that in mind, **we first ask you to:**

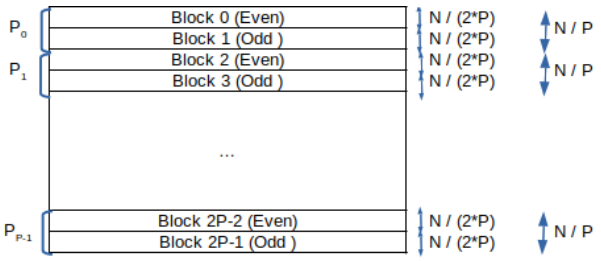
1. Write the expression for the contribution to the total parallel time T_P coming from the parallel computation time T_P^{comp} on an ideal machine with P processors, assuming that: 1) $NB = 2 * P$ perfectly divides the problem size `N`; 2) the execution time for a single iteration of the innermost loop body in routine `compute_row_block` takes t_c time units; and 3) no parallelisation overheads should be considered at this point.

   Solution:

   Since N is very large, we can consider $N - 2 \approx N$. Then,

   $T_P^{comp} = 2 \times BS \times N \times t_c = N^2/P \times t_c$

   Explanation:



All the tasks in the *EVEN* loop can be executed in parallel. All the tasks in the ODD loop can also be executed in parallel. However, tasks in the *ODD* loop can only be executed once the tasks in the EVEN loop computing the surrounding blocks have been completed. Each of the two loops iterate P times. Therefore, with P processors each processor will execute only 1 iteration of each loop. Since N is large, we can consider $N - 2 \approx N$. Thus, each processor computes N/P rows of size N, for a total of N^2/P executions of the innermost loop of routine `compute_row_block`. Note that half of them correspond to its *EVEN* block and the other half to its *ODD* block.

Next we consider that the ideal machine has the memory distributed among the P processors, In that machine, matrix `u` is divided row-wise in $NB = 2 * P$ blocks, where each block contains $BS = N/(2 * P)$ consecutive rows. Pairs of consecutive blocks are assigned to the same processor, so for example blocks 0 and 1 are owned by processor 0, blocks 2 and 3 are owned by processor 1; and so on and so forth. Each processor is in charge of computing all the rows in the blocks that are assigned to it, and therefore it will have to execute one iteration of the *EVEN* loop and one iteration of the *ODD* loop. Before answering the second question below, and having in mind the assignment of blocks and iterations, think about the need for processors to perform any remote access before starting the execution of tasks in the *EVEN* loop, or before starting the execution of tasks in the *ODD* loop; and, if affirmative, how many elements need to be transferred in each case and which processors to do them. Having all that in mind, next **we ask you to:**

2. Write the expression for the contribution to T_P coming from the data sharing overheads $T_P^{data\_sharing}$, if any, as part of the overall $T_P = T_P^{comp} + T_P^{data\_sharing}$. For that you should consider the data sharing model explained in class. Accesses to local memory are performed with zero overhead; accesses to remote memory take $t_{comm} = t_s + m \times t_w$, being t_s and t_w the start–up time for the remote access and transfer time of one element, respectively. At a given moment, a processor can only perform one remote memory access to another processor, and can only serve a remote memory access from another.

   Solution:

   $T_P^{data\_sharing} = 2 \times (t_s + N \times t_w)$

   Explanation:

   In general, before an EVEN block can be computed a processor needs to perform a remote access to the N elements in the last row of the block above, which is owned by the previous processor (except for processor 0). No remote access is required to access the first row of the next block, because it is an ODD block owned by the same processor. All the processors can perform such remote accesses in parallel.

   An ODD block can only be computed once its surrounding EVEN blocks are computed. Before an ODD block can be computed a processor needs to perform a remote access to the N elements in the first row of the block below, which is owned by the next processor. No remote access is required to access the last row of the previous block, because it is an EVEN block owned by the same processor. All the processors can perform such remote accesses in parallel.

Problem 2 (2.5 points)

Given the following code fragment:

```
#define CACHE_LINE_SIZE  128
#define DBSize           (32*1024*1024)
#define NTHREADS         3

int count[NTHREADS];
int DB[DBSize];
int key;

// Initialization loop
for (int i = 0; i < NTHREADS; i++)
    count[i] = 0;
```

```
#pragma omp parallel num_threads(NTHREADS)
{
    int my_id = omp_get_thread_num();
    for (int i = my_id; i < DBSize; i += NTHREADS) {
        // read access to DB[i]
        if (DB[i]==key)
            // read access to count[my_id] followed by a write access
            count[my_id] = count[my_id] + 1;
    }
}
```

Assume a shared–memory UMA parallel system with 3 processors, each one with its own (private) cache memory. Data coherence in the system is maintained using Write Invalidate `MSI` protocol, with a Snoopy attached to each cache memory. Also assume 1) empty caches at the beginning of the program; 2) `count[0]` and `DB[0]` are aligned to the beginning of different memory lines; 3) the rest of variables are stored in registers; and 4) the size for a variable of type `int` is 4 bytes and cache line size is 128 bytes. **We ask you:**

1. Assume thread 0, running on processor 0, starts executing the sequential part of the program until the parallel region starts (i.e. it executes the initialization loop). How many cache lines are used to store the full `count` vector after the execution of the initialization loop? What is the cache coherence state for each cache line storing elements of `count` and in which private cache it is located?

   Solution:

   We only require one cache line to store the three elements of the vector `count`. The coherence state of the cache line will be M (Modified) and the memory cache that will store the vector is MC_0.

2. Assume the cache states in previous question (after the execution of the initialization loop) and that each thread i runs on processor P_i of the UMA system within the parallel region. Complete the table in the provided answer sheet which represents the first sequence of actions (from top to bottom of the table) executed by the three threads in the parallel region for the first iterations of the loop. State MC_i in each row has to be filled with the state of the cache line after the memory access in the action. For the rest of columns, you should specify the processor event ($PrRd_i$, $PrWr_i$), if there is cache hit or miss, the bus command ($BusRd_i$, $BusRdX_i$, $BusUpgr_i$), and the flush transaction ($Flush_i$), where i is the number of the processor where it is generated. Note: Observe that if you want to leave a cell empty, you can write "-".

   Solution:

Parallel Region Execution							
Action	CPU event	Miss/Hit	Bus command	Flush?	State MC_0	State MC_1	State MC_2
P_0 read DB[0]	PrRd_0	miss	BusRd_0	-	S	-	-
P_0 read count[0]	PrRd_0	hit	-	-	M	-	-
P_1 read DB[1]	PrRd_1	miss	BusRd_1	-	S	S	-
P_1 read count[1]	PrRd_1	miss	BusRd_1	Flush_0	M->S	S	-
P_0 write count[0]	PrWr_0	hit	BusUpgr_0	-	S->M	S->I	-
P_1 write count[1]	PrWr_1	miss	BusRdX_1	Flush_0	M->I	I->M	-
P_2 read DB[2]	PrRd_2	miss	BusRd_2	-	S	S	S
P_2 read count[2]	PrRd_2	miss	BusRd_2	Flush_1	I	M->S	S
P_2 write count[2]	PrWr_2	hit	BusUpgr_2	-	I	S->I	S->M
P_0 read DB[3]	PrRd_0	hit	-	-	S	S	S
P_1 read DB[4]	PrRd_1	hit	-	-	S	S	S

3. Assuming the amount of data we are accessing in the program, a system with a main memory of 32 GBytes and 32 Kbytes of private cache memory per processor, what is the total number of bits that an UMA system would use to keep the cache coherence per processor and the overall system? Would this number of bits change if we change the protocol from `MSI` to `MESI`?

   Solution:

   Per processor/cache:
   $32kbytes \times \frac{1line}{128bytes} \times \frac{2bits}{1line} \rightarrow \frac{2^{15} \times 2}{2^7} bits \rightarrow 2^9 bits \rightarrow 512 bits$

   Overall:
   3×512 bits devoted to coherence

   The number of bits would not change because we can still use 2 bits to represent 4 states (M, E, S, I) as when we have 3 states (M, S, I).

Problem 3 (2.5 points)

Given the following sequential program that computes the number of times the value stored in variable `element` appears in a vector of lists `data`:

```
#define NUM_ELEMS 10000
typedef struct list {
    int elem;
    struct list * next;
} list; // the basic component of a list

list * data[NUM_ELEMS]; // vector of lists, each list with varying number of elements
int element, count = 0; // value to search within data and number of times it appears

// function that returns the number of times element appears in theList
int list_search(list * theList, int element);

void main() {
    ...
    for (int entry = 0; entry < NUM_ELEMS; entry++) {
         int tmp = list_search(data[entry], element);
         count += tmp;
    }
    ...
}
```

Each of the lists may have a different number of elements, so when parallelising the program one should take care of load balancing. A parallel version for the sequential program above is also available, in which the original `for` loop has been substituted by a parallel region that assigns individual iterations to explicit tasks in such a way that tasks are generated under certain circumstances. One new shared variable `active_tasks` and two functions to operate it have also been added:

```
...
int active_tasks = 0;
// Functions to atomically add or subtract 1 to memory location pointed by address.
// The operation is saturated to the max or min value (i.e. the result can not be
// greater than max and smaller than min, respectively). They return value in memory
// location before operation
int atomic_inc(int *address, int max);
int atomic_dec(int *address, int min);

void main() {
    #pragma omp parallel
    #pragma omp single
    {
```

```
        int workers = omp_get_num_threads() - 1; // one thread focuses on
                                                 // task creation, the rest execute tasks
        for (int entry = 0; entry < NUM_ELEMS; entry++) {
            while (atomic_inc(&active_tasks, workers) == workers);
            #pragma omp task depend(inout: count)
                {
                int tmp = list_search(data[entry], element);
                count += tmp;
                atomic_dec(&active_tasks, 0);
                }
        }
    }
}
```

After compiling the parallel program and executing it with P processors (with $P > 1$) we detect that the program **is not achieving any speed–up**, although it produces a correct result.

1. Assuming the functionality for functions `atomic_inc` and `atomic_dec` explained in the code itself, what are these two functions used for in the program? Which is the number of implicit and explicit tasks that are generated during the execution of the program?

   Solution:

   The program generates P implicit tasks, one for each thread in the `parallel` construct. Only one of them (the one entering in `single`) is in charge of generating the explicit tasks: one explicit task is generated for each iteration of the for loop, so in total `NUM_ELEMS` explicit tasks. Functions `int atomic_inc(int *address, int max)` and `int atomic_dec(int *address, int min)` are used to control the number of tasks generated (kind of cut-off mechanism), in such a way that no more than $P - 1$ explicit tasks are simultaneously pending to execute or executing.

2. In the parallel version provided above, how many of these explicit tasks can be simultaneously executing? Rewrite the program above making the minimum appropriate changes in order to increase this number and, as a consequence, achieve a much better speed–up. Make sure the program generates the correct result. After these minimal changes, which can be the maximum number of explicit tasks that could be simultaneously executing?

   Solution:

   The `depend(inout:count)` clause used in `task` forces explicit tasks to execute sequentially, in the order they are created. This is not the most appropriate way to guarantee the race condition in this program when updating variable `count`; instead the use of `atomic` would enable the parallelism in the execution of multiple invocations to `list_search` while guaranteeing the correct update of variable `count`. With this change, a maximum of $P - 1$ tasks could be executing simultaneously.

   ```
   #pragma omp task // shared(count) and firstprivate(entry) by default
       {
       int tmp = list_search(data[entry], element);
       #pragma omp atomic
       count += tmp;
       atomic_dec(&active_tasks, 0);
       }
   ```

   Alternatively, a solution based on task reductions would also be valid:

   ```
   #pragma omp taskgroup task_reduction(+: count)
   for (int entry = 0; entry < NUM_ELEMS; entry++) {
       while (atomic_inc(&active_tasks, workers) == workers);
   ```

```
            #pragma omp task in_reduction(+: count)
                {
                int tmp = list_search(data[entry], element);
                count += tmp;
                atomic_dec(&active_tasks, 0);
                }
        }
```

3. Do an implementation for function `atomic_inc` making use of load–linked (`ll`) and store–conditional (`sc`) operations, defined as follows:

```
int ll(int *address); // returns the value stored in address
int sc(int *address, int value); // stores value in address if atomicity with ll
                                 // has been accomplished, returning true (1);
                                 // retuns false (0) otherwise
```

Solution:

A possible solution could be:

```
int atomic_inc(int *address, int max) {
    int tmp = ll(address);
    while ((tmp < max) && !sc(address, tmp+1)) tmp = ll(address);
    return(tmp);
}
```

In this solution, if the value read from memory `address` is equal to `max`, then the while loop finishes, returning the value just read. If the value is smaller than `max`, then a `sc` of the incremented value on the same memory `address` is attempted; if it succeeds the while loop is finished, again returnin the original value read from memory. If `sc` fails, then the new value from memory `address` is read again.

An equivalent code written in a different (more explicit) way would be:

```
int atomic_inc(int *address, int max) {
    int retsc = 0;
    do {
        int tmp = ll(address);
        if (tmp == max) return(tmp);
        retsc = sc(address, tmp+1);
    } while (retsc == 0);
    return(tmp);
}
```

Another version that always writes to memory `address` would be:

```
int atomic_inc(int *address, int max) {
    int tmp, newvalue;
    do {
        tmp = newvalue = ll(address);
        if (tmp < max) newvalue++;
    } while (!sc(address, newvalue));
    return(tmp);
}
```

Observe that in this case if the value in memory `address` is already `max`, the same value `max` is unnecessarily written again to memory `address`.

4. Finally we found a recursive version alternative to the original sequential code:

```
...
void rec_list_search(list ** data, int size, int element) {
    int tmp = list_search(data[0], element);
    count += tmp;
    if (size > 1)
        rec_list_search(data+1, size-1, element);
}

void main() {
    rec_list_search(&data[0], NUM_ELEMS, element);
}
```

Write a parallel version for it that implements a *recursive* **leaf** *task decomposition*. This new version should not implement any cut-off mechanism to restrict the number of tasks that are generated.

Solution:

```
void rec_list_search(list ** data, int size, int element) {
    #pragma omp task shared(count)
    {
        int tmp = list_search(data[0], element);
        #pragma omp atomic
        count += tmp;
    }
    if (size > 1)
        rec_list_search(data+1, size-1, element);
}

void main() {
    ...
    #pragma omp parallel
    #pragma omp single
    rec_list_search(&data[0], NUM_ELEMS, element);
    ...
}
```

Problem 4 (2.5 points)

Given the following code fragment computing matrix `m[N][N]`, with `N` much larger than the number of processors P to be used in the parallel execution, and with `N` not necessarily a multiple of P:

```
telem m[N][N];

for (int i=1; i<N-1; i++)
   for (int j=0; j<N; j++)
       m[i][j] = compute (m[i][j], m[i-1][j], m[i+1][j]);
```

We ask you to:

1. Decide the most appropriate *geometric data decomposition strategy* for matrix `m` and write a parallel version of the code above using OpenMP that corresponds to it. Your solution should a) minimize the synchronization overhead among implicit tasks and b) guarantee that the load unbalance is limited to `N` elements (i.e. the number of elements in a row or column of the matrix).

   Solution:

   There are dependencies between iterations of the `for-i` loop: a RAW dependency given by $m[i][j]$ to $m[i-1][j]$ and a WAR dependency given by $m[i][j]$ to $m[i][j+1]$ for i in $[2..N-2]$. There are

no dependencies between iterations of the `for-j` loop, so we can fully parallelize it, with no need for synchronization.

Consequently, it's advisable to choose a *Geometric Block Data decomposition* by columns. We must take care of adjusting the block size (number of columns of N elements each) so that there is at most 1 column of difference in size between the different blocks. In order to apply the owner compute rule, the distribution of the matrix `m` is by columns with the resulting block size. A *Geometric Cyclic Data decomposition* by columns would also be correct, as the amount of work would be balanced automatically, without any extra calculation.

```
telem m[N][N];
...
#pragma omp parallel num_threads(P)
{
  int myid = omp_get_thread_num();
  int BS = N / P;
  int start = myid * BS;
  int end = start + BS;
  int mod = N % P;
  if (mod > 0) {
     if (myid < mod)
        start += myid;
        end = start + BS + 1;
     }
     else {
         start += mod;
         end += start + BS;
     }
  }
  for (int i=1; i<N-1; i++) {
    for (int j=start; j<end; j++) {
        m[i][j] = compute (m[i][j], m[i-1][j], m[i+1][j]);
    }
  }
}
```

2. Now consider that the program is going to be executed on a parallel machine in which memory lines are 128 bytes long. The allocation in memory for matrix `m` is aligned to the start of a memory line and `sizeof(telem)` is 8 bytes (`N` is not necessarily multiple of `sizeof(telem)`). Decide the most appropriate *geometric data decomposition strategy* in this case and re-write the previous OpenMP parallel code and, if necessary, the definition of matrix `m`. Your solution should a) maximize parallelism among implicit tasks; and b) maximize data locality and reduce coherence traffic.

   Solution:

   In order to preserve data locality we must take care of the memory line size when accessing elements of the matrix. In this sense, given that `sizeof(telem) = 8 bytes`, a memory line of 128 bytes can hold up to 16 elements. For this reason we will consider only block sizes with values multiple of 16. Consequently we apply a *Geometric Block-cyclic data decomposition* by columns, with block size = 16. Given that N » P we can assume a load unbalance of one block (N x 16 elements). In addition, in case N is not multiple of `sizeof(telem)`, we must add padding at the end of the row to avoid generating false sharing with the beginning of the next row. In order to apply the owner compute rule, the distribution of the matrix `m` is by columns with the resulting block size.

```
#define MEMORYLINESIZE 128
#define BS MEMORYLINESIZE/sizeof(telem)
#define X (N%BS==0? 0: (BS - (N % BS)))
```

```
telem m[N][N+X];

int BS = MEMORYLINESIZE / sizeof(telem);
...
#pragma omp parallel num_threads(P)
{
  int myid = omp_get_thread_num();
  int start = myid * BS;
  int end = N;

  for (int i=1; i<N-1; i++) {
    for (int jj=start; jj<end; jj+=BS*P)
        for (int j=jj; j<j+BS; j++)
            m[i][j] = compute (m[i][j], m[i-1][j], m[i+1][j]);
}
```

Student name: .....

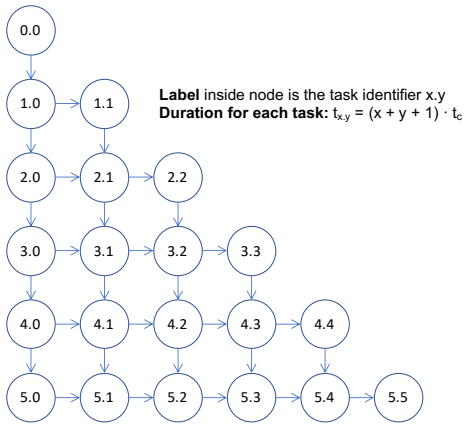
Answer sheet for **Question 2.2**.

Parallel Region Execution							
Action	CPU event	Cache Miss/Hit	Bus command	Flush?	State MC_0	State MC_1	State MC_2
P_0 reads DB[0]							
P_0 reads count[0]							
P_1 reads DB[1]							
P_1 reads count[1]							
P_0 writes count[0]							
P_1 writes count[1]							
P_2 reads DB[2]							
P_2 reads count[2]							
P_2 writes count[2]							
P_0 reads DB[3]							
P_1 reads DB[4]							

PAR – Final Exam – Course 2021/22-Q2

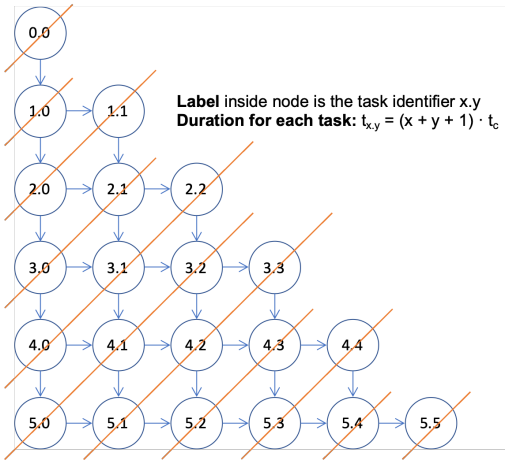
June 20^{th}, 2022

Problem 1 (1.5 points) Given the following Task Dependence Graph associated to the task decomposition applied to a certain program:

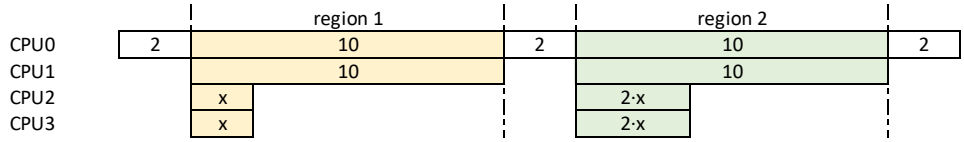


The duration, in time units, of each `compute_task` depends on the value of the row (`x`) and column (`y`) where it is placed (label `x.y`): $t_{x.y} = (x + y + 1) \times t_c$. **We ask you** to compute the values for T_1, T_∞, $Parallelism$ and P_{min}.

Solution: $T_1 = 126 \times t_c$ (sum of the cost of all nodes in the TDG above), $T_\infty = 66 \times t_c$ (defined by the critical path in the TDG, including all nodes in the column on the left and all nodes in the row at the bottom of the TDG above) and $Parallelism = T_1 \div T_\infty = 126 \div 66 = 1.\widehat{90}$. The minimum number of processors to achieve it is $P_{min} = 3$, since we need enough simultaneous processors to execute the widest anti–diagonal in the wavefront execution of the TDG, as shown in the picture below.



Problem 2 (1.5 points) The following diagram plots the timeline for the execution of a parallel application, with two parallel regions, on 4 processors (time evolves from left to right):



Each box represents a burst executed on a processor and the number inside its execution time. There are 3 sequential regions with constant execution time of 2 time units. The execution time is unknown for two of the bursts in $region1$ and in $region2$, both affected by the value x (the timeline shows the case for which

$(2 \times x) < 10$).

- Obtain all the possible values for value x that would lead to an speed-up $S_4 = 2.5$ when the application is executed on 4 processors. Observe that for values $0 < x \leq 5$, $5 < x \leq 10$ and $x > 10$ the two parallel regions have a different contribution in the timeline.

  Solution: In all cases, $T_1 = 2 + (10 + 10 + x + x) + 2 + (10 + 10 + 2 \times x + 2 \times x) + 2 = 46 + 6 \times x$.

  - For x smaller than 5, the execution time for both parallel regions is 10 units, so $T_4 = 2 + 10 + 2 + 10 + 2 = 26$. Therefore from $S_4 = T_1/T_4 = 2.5$ we get $x = 19/6 = 3.16$.
  - For x larger than 5 but smaller than 10, the execution time for the first parallel region is still 10 but for the second one is $2 \times x$. Therefore $T_4 = 16 + 2 \times x$. From $S_4 = T_1/T_4 = 2.5$ it is not possible to get a positive value for x.
  - Finally, for x larger than 10, both parallel regions are dominated by the value of x, being the execution time $T_4 = 6 + 3 \times x$. Again from $S_4 = T_1/T_4 = 2.5$ we obtain $x = 31/1.5 = 20.66$.

- For the values of x in the previous question, obtain the value for $S_{p \to \infty}$, assuming that parallel regions ideally scale with the number of processors.

  Solution: For the two cases above, $T_{p \to \infty} = 6$. Therefore:

  - For $x = 19/6 = 3.16$ we get $S_{p \to \infty} = 65/6 = 10.83$.
  - Similarly, for $x = 31/1.5 = 20.66$ we get $S_{p \to \infty} = 170/6 = 28.33$.

Problem 3 (3.0 points) Given the following data structures that models a graph in which each node can have up to 4 neighbour nodes. Vector `g` holds the information for `N` nodes; for each node the `telem` struct stores the weight of the node `w`, an integer to identify each of the 4 possible neighbours (`north`, `east`, `west` and `south`) and a field that is used to traverse the graph (`visited`) .

```
#define N ... /* large value */

typedef struct {
        int w;
        int north, east, west, south; // -1 value to indicate no neighbour
        char visited;
} telem;
telem g[N];
```

The following code processes all the nodes that are reachable from a given node (0 in the invocation from main):

```
int compute (int label, int weight); // heavy computation, does not access the graph

int traverse_reachable (int label) {
        int ret=0, ret1, ret2, ret3, ret4;
        if (label >= 0) {
            if (!g[label].visited) {
                g[label].visited=1;
                ret1 = traverse_reachable (g[label].north);
                ret2 = traverse_reachable (g[label].east);
                ret3 = traverse_reachable (g[label].west);
                ret4 = traverse_reachable (g[label].south);
                ret = compute(label, g[label].w) + ret1 + ret2 + ret3 + ret4;
            }
        }
        return ret;
}

int main() {
```

```
    ...
    int ret = traverse_reachable (0);
    ...
}
```

We ask you to write an *OpenMP* parallel version of the previous code using a *recursive tree task decomposition* strategy. The implementation should maximize parallelism and minimize the possible synchronization overheads. The implementation must also include a task generation control mechanism based on the recursion level. Use *MAX_ DEPTH* as the value for the maximum recursion level for which tasks must be generated.

Solution:

```
typedef struct {
        int w;
        int north, east, west, south;
        char visited;
        omp_lock_t lock;    // new field to protect the access to the node
} telem;
telem g[N];

int traverse_reachable (int label, int d) { // new argument d to control recursion level
        int ret=0, ret1, ret2, ret3, ret4, tmp;

        if (label >= 0) {
             if (!g[label].visited) {
                omp_set_lock (&g[label].lock);
                if (!g[label].visited) {
                    g[label].visited=1;
                    omp_unset_lock (&g[label].lock);

                    if (!omp_in_final()) {
                        #pragma omp task shared(ret1) final (d>= MAX_DEPTH)
                        ret1 = traverse_reachable (g[label].north, d+1);
                        #pragma omp task shared(ret2) final (d>= MAX_DEPTH)
                        ret2 = traverse_reachable (g[label].east, d+1);
                        #pragma omp task shared(ret3) final (d>= MAX_DEPTH)
                        ret3 = traverse_reachable (g[label].west, d+1);
                        #pragma omp task shared(ret4) final (d>= MAX_DEPTH)
                        ret4 = traverse_reachable (g[label].south, d+1);
                        tmp = compute(label, g[label].w);
                        #pragma omp taskwait     // compute does not need to wait for the graph t
                    } else {
                        ret1 = traverse_reachable (g[label].north, d+1);
                        ret2 = traverse_reachable (g[label].east, d+1);
                        ret3 = traverse_reachable (g[label].west, d+1);
                        ret4 = traverse_reachable (g[label].south, d+1);
                        int tmp = compute(label, g[label].w);
                    }
                    ret = tmp + ret1 + ret2 + ret3 + ret4;
                } else {
                    omp_unset_lock (&g[label].lock);
                }
            }
        }
        return ret;
}

int main() {
    ...
    #pragma omp parallel
```

```
    #pragma omp single
    int ret = traverse_reachable (0, 0);
    ...
}
```

Problem 4 (4.0 points) Given the following sequential code:

```
void saxpy(int n, float a, float * x, float * y) {
  for (int i = 0; i < n; ++i)
      y[i] = a*x[i] + y[i];
}
```

Assume parameters `x` and `y` point to two completely disjoint vectors, first element of each vector aligned to a memory/cache line boundary. Cache line size is 64 bytes and sizeof(float) is 4 bytes. **We ask you to:**

1. Write a first OpenMP parallel version for the previous sequential function that obeys to an *input block geometric data decomposition* strategy, so that each thread accesses to a single block of consecutive elements. The *block geometric decomposition* should minimise the possible load unbalance that occurs when the size of the vectors `n` is not a multiple value of the number of threads. Assume that `n` is large compared to the number of threads.

   Solution:

   ```
   void saxpy(int n, float a, float * x, float * y) {
     #pragma omp parallel
     {
        int nt;
        int my_id;
        int red;
        int i_start, i_end, n_elems;
        nt    = omp_get_num_threads();
        my_id = omp_get_thread_num();
        n_elems = n/nt;
        red     = n%nt;
        i_start = my_id*n_elems;
        i_start = i_start + (my_id<red)?my_id:red;
        i_end   = i_start + n_elems + (my_id<red);
        for (int i = i_start; i < i_end; ++i)
            y[i] = a*x[i] + y[i];
     }
   }
   ```

2. Write a new parallel version that obeys to an *input/output cyclic geometric data decomposition* strategy. Has the load balance improved with respect to the previous version?

   Solution:

   ```
   void saxpy(int n, float a, float * x, float * y) {
     #pragma omp parallel
     {
       int nt;
       int my_id;
       nt    = omp_get_num_threads();
       my_id = omp_get_thread_num();
       for (int i = my_id; i < n; i+=nt)
         y[i] = a*x[i] + y[i];
     }
   }
   ```

   Both implementantions block and cyclic geometric data decomposition have the same load unbalance: maximum 1 element.

3. Assuming the *input/output cyclic geometric data decomposition strategy* in the previous implementation, and that 1) the parallel system is composed of 2 NUMA nodes, each with a single processor and private cache; 2)

the parallel program is executed with 2 threads (i.e. *thread* i in NUMA node i); 3) the operating system makes use of "first touch" at page level to decide the allocation of memory addresses to NUMA nodes, with one line per memory page; 4) main memory, directories and private caches are empty when the function above starts its execution; and 5) the execution of the iterations assigned to the two threads are interleaved in time, as shown in the two leftmost columns in the table in the answer sheet for the first 4 iterations of the loop. Data coherence across NUMA nodes is provided by a write-invalidate MSU directory-based system. Complete this table with the information of the directory entries where elements of vector `x` and `y` are stored.

4. Write a final parallel version that obeys to an *output block-cyclic geometric data decomposition* strategy, so that the overhead associated with the coherence protocol in a NUMA multiprocessor architecture is minimised, i.e. exploiting the data locality of both input and output data. Is the load balance better/worse than the one achieved in the parallel versions in question 1 and 2?

   Solution:

   We set BS to the number of float elements that fits in a cache/memory line. In this way we avoid extra cache misses accessing both vector `x` and `y` and false sharing accessing `y`.

```
#define BS (64/4)
void saxpy(int n, float a, float * x, float * y) {
  #pragma omp parallel
  {
    int nt;
    int my_id;
    nt    = omp_get_num_threads();
    my_id = omp_get_thread_num();
    for (int ii = my_id*BS; ii < n; ii+=nt*BS)
      for (int i = ii; i < max(n,ii+BS); i++)
         y[i] = a*x[i] + y[i];
  }
}
```

   Block cyclic geometric data decomposition may have `BS` elements of load unbalance. Therefore, it is worse than before.

Student name: .....

Answer sheet for **Problem 4**.

Time	thread	Loop iteration i - vector access	Home NUMA node	Directory entry (sharers bit list)	Directory entry (status)
0	1	1 - read x[1]			
		1 - read y[1]			
		1 - write y[1]			
1	0	0 - read x[0]			
		0 - read y[0]			
		0 - write y[0]			
2	0	2 - read x[2]			
		2 - read y[2]			
		2 - write y[2]			
3	1	3 - read x[3]			
		3 - read y[3]			
		3 - write y[3]			

Solution for **Problem 4**.

Time	thread	Loop iteration i - vector access	Home NUMA node	Directory entry (sharers bit list)	Directory entry (status bits)
0	1	1 - read x[1]	1	10	S
		1 - read y[1]	1	10	S
		1 - write y[1]	1	10	M
1	0	0 - read x[0]	1	11	S
		0 - read y[0]	1	11	S
		0 - write y[0]	1	01	M
2	0	2 - read x[2]	1	11	S
		2 - read y[2]	1	01	M
		2 - write y[2]	1	01	M
3	1	3 - read x[3]	1	11	S
		3 - read y[3]	1	11	S
		3 - write y[3]	1	10	M

PAR – Final Exam – Course 2022/23-Q1

January 18^{th}, 2023

Problem 1 (2.5 points)

Given the following code with *tareador* task annotations:

```
#define p ...   // Number of processors
#define NR ...  // Number of rows
#define NC ...  // Number of columns

int M[NR][NC];
int BS = NR/p;  // Assume p divides NR exactly

for (int ii=0; ii<NR; ii+=BS) {                    // Matrix initialization
    tareador_start_task ("init");
    for (int i=ii; i<ii+BS; i++)
       for (int j=0; j<NC; j++)
         M[i][j] = init(i,j);                      // <-- Cost ti
    tareador_end_task ("init");
}

tareador_start_task ("comp1");                     // Begin computations
for (int i=0; i<BS; i++)
    for (int j=0; j<NC; j++)
        M[i][j] = comp1(M[i][j]);                  // <-- Cost tb
tareador_end_task ("comp1");

for (int ii=BS; ii<NR; ii+=BS) {                   // Final computations
    tareador_start_task ("comp2");
    for (int i=ii, int i0=0;
         i<ii+BS;
         i++, i0++)
       for (int j=0; j<NC; j++)
         M[i][j] = comp2(M[i0][j], M[i][j]);       // <-- Cost tf
    tareador_end_task ("comp2");
}
```

Let us assume the data sharing model explained in class based on a distributed memory architecture in which we consider that local memory accesses have no cost, but an access to data in different processors introduces a data–sharing overhead: the access time to remote data is determined by $t_{comm} = t_s + m \times t_w$, being t_s and t_w the "start-up" and sending time of an element, respectively, and being m the size of the message. Also, according to the data sharing model: at any time, each processor can simultaneously make one remote access to a different processor and serve one remote access from another processor.

Assume that the number of processors p divides the number of rows `NR` exactly; routines `init`, `comp1` and `comp2` do not modify any memory position; the execution time of one iteration of the body of the most internal loops is t_i, t_b and t_f respectively; tasks are scheduled to processes following the *owner-computes rule* so that a task will be executed by the processor who owns the memory that holds the output of that task; the matrix is already distributed in the memory of each processor when the computation starts; the resulting matrix remains distributed and there is no final communication to a single processor; the strategy used for decomposing matrix `M` follows a *block row distribution*: the matrix `M` is distributed so that each processor has $\frac{NR}{p}$ consecutive rows. **We ask** you to:

1. Draw a Task Dependence Graph (TDG) for the case where $p = 4$;
2. Draw a timeline for the execution with $p = 4$;
3. Provide a general expression that determines the parallel execution time on p processors (T_p): express T_p as a function of p, `NR`, `NC`, t_i, t_b, t_f, t_s and t_w.

Solution:

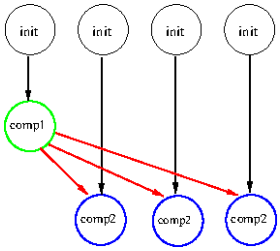


Figure 1: TDG

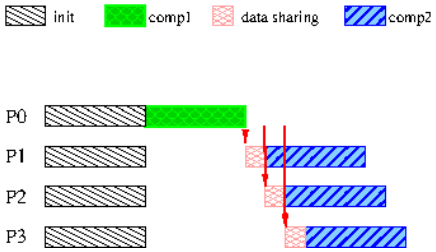


Figure 2: Timeline

$T_p = t_{data\_sharing} + t_{calc}$

$t_{data\_sharing} = (t_s + \frac{NR}{P} \times NC \times t_w) \times (P-1)$

$t_{calc} = \frac{NR}{P} \times NC \times t_i + \frac{NR}{P} \times NC \times t_b + \frac{NR}{P} \times NC \times t_f = \frac{NR}{P} \times NC \times (t_i + t_b + t_f)$

Problem 2 (2.5 points)

Assume a NUMA (non-uniform memory architecture) multiprocessor system composed of 4 NUMA-nodes ($node_{0:3}$), each node with 8 GBytes of main memory and 4 cores ($core_{0:3}$). Each core has only one level of cache memory of 2 MBytes (with cache lines of 32Bytes). The system includes all the necessary mechanisms (seen in class) to keep the memory coherence inside a NUMA-node and between NUMA-nodes.

1. (0.5 points) In order to support a write-invalidate MSI cache-coherence protocol within each NUMA-node, how many additional bits should be used in each cache line, and what are their role and possible values? How many bits in total per NUMA-node (only within each NUMA-node) are used for that purpose?

   Solution:

   We need 2 additional bits per cache line in order to store the state of the cache line. Those 2 bits can code up to four possible states:

   - I invalid
   - S shared (two or more nodes may have clean copies)
   - M modified (dirty)
   - Note that a fourth state can be coded with 2 bits. However, it is not necessary for the MSI protocol.

   The number of cache lines is calculated dividing the memory in a cache (2MB) by the cache line size (32B). Therefore: Number of cache lines $= \frac{2MB}{32B} = 2^{16} = 64K$ entries

   We need 2 bits per entry. And there are a total of 4 cache memories per NUMAnode.

   Thus, the total number of bits amounts to:

   $\frac{2MB}{32B}$ entries/cache $\times 2$ bits/entry $\times 4$ caches/NUMAnode $= 2^{19}$ bits $= 512K$ bits/NUMAnode

2. (0.5 points) In order to support a directory-based write-invalidate MSU cache-coherence protocol between NUMA-nodes, how many bits should be used in the directory for each line in main memory, and what are their role and possible values? How many bits in total per NUMA-node (directory) are used for that purpose?

   Solution:

   The home NUMA-node is in charge of the coherence of its physical memory lines by means of the directory entries. A directory entry stores the line state and the identities of other NUMA-nodes sharing this memory line.

Bits per directory entry:

- Presence bits (nodes currently having the line), 1 bit per NUMA-node: i.e. 4 bits
- State bits (to track the state of memory lines): 2 bits
  Those 2 bits can code up to four possible states:
  - U uncached, not valid in any cache
  - S shared (two or more nodes may have clean copies)
  - M modified (dirty)
  - Note that a fourth state can be coded with 2 bits. However, it is not necessary for the MSI protocol.

Total: 6 bits per directory entry.

The number of entries in the directory structure per NUMA-node is calculated dividing the memory in a NUMA-node (8GB) by the memoy line size (32B). Therefore:

Number of Directory Entries $= \frac{8GB}{32B} = 2^{28} = 256$ Mega entries

Thus, with 6 bits per directory entry, the total number of bits per NUMAnode will be:

$\frac{8GB}{32B}$ entries/NUMAnode $\times 6$ bits/entry $= 3 \times 2^{29} = 1536$ Mega bits/NUMAnode

3. (1.5 points) Consider the following parallel program executed on only two cores (processors) inside $node_0$ of the previous multiprocessor system:

```
...
double A[M][N];
...
```

Core 0: Update even-numbered rows

```
for ( j = 0 ; j < M ; j += 2 )
   for ( k = 0 ; k < N ; k++ )
        A[j][k] = f(j,k);
```

Core 1: Update odd-numbered rows

```
for ( j = 1 ; j < M ; j += 2 )
   for ( k = 0 ; k < N ; k++ )
       A[j][k] = g(j,k);
```

Assume that matrix `A` is stored in main memory of $node_0$ (without copies of any of its elements in the caches of the NUMAnodes), that each element of matrix `A` is 8 Bytes and that the first element of `A` is aligned on a cache line boundary, $N = 2$ and M is a value multiple of 2.

(a) What would be the maximum number of invalidations that would be sent through the bus expressed as a function of M? What is causing such a memory coherence problem?

Solution:

For $N = 2$, every two rows fall in the same cache line, causing up to 3 invalidations due to false sharing for every pair of lines. Thus, the number of invalidations is $3 \times M/2$.

(b) Modify the declaration of matrix A so that you do not have to change the code and avoid the previous coherence problem?

Solution:

We add enough padding to the second dimension of matrix A to make the size (in bytes) of a row of A equal or multiple of the number of bytes of a cache line.

```
...
#define PADDING ((CACHE_LINE_SIZE - (N*sizeof(double) % CACHE_LINE_SIZE))/sizeof(double))
double A[M][N+PADDING];
...
```

Problem 3 (2.5 points)
Given the following code:

```
#define SIZE_INDEX 256
#define N 1024*1024*1024

void histogram(unsigned int *S, int n, unsigned int *index) {
 unsigned int i, tmp;
 for (i=0; i<n; i++) {
   tmp = S[i]%SIZE_INDEX;
   index[tmp]++;
 } }

void main() {
  unsigned int S[N];
  unsigned int index[SIZE_INDEX];
  ...  // Here we have initialized S to random numbers and index to 0's
  histogram(S, N, index);
  ...
}
```

Write two different OpenMP parallel implementations of function `histogram` using the strategies presented below. You can modify the sequential code, add local variables and use any OpenMP pragma (except `omp for` worksharing-loop construct) and function you may need. Both implementations should minimize the use of synchronizations and load unbalance between threads during the processing of vector S.

1. (1 point) Cyclic Data Decomposition of the Output vector `index`.
   Solution:

   ```
   #define SIZE_INDEX 256
   #define N 1024*1024*1024
   ...

   void histogram(unsigned int *S, int n, unsigned int *index)
   {
    unsigned int i, tmp;

   #pragma omp parallel private(i, tmp)
   {
      int id = omp_get_thread_num();
      int num_threads = omp_get_num_threads();

      for (i=0;i<n;i++) {
        tmp = S[i]%SIZE_INDEX;
        if ((tmp%num_threads)==myid)
          index[tmp]++;
    }
   }

   }

   ...
   ```

2. (1.5 points) Block Data Decomposition of the Input vector `S`.
   Solution:

   ```
   #define SIZE_INDEX 256
   ```

```
#define N 1024*1024*1024
...

void histogram(unsigned int *S, int n, unsigned int *index)
{
 unsigned int i, tmp;

 #pragma omp parallel private(i, tmp)
 {
  int myid        = omp_get_thread_num();
  int num_threads = omp_get_num_threads();
  int i_start     = myid * (N/num_threads);
  int i_end       = (myid+1) * (N/num_threads);
  int res         = N%num_threads;
  if (res)
  {
     i_start = i_start + (myid<res)?myid:res);
     i_end   = i_end   +   (myid<res)?myid+1:res);
  }

  unsigned int local_index[SIZE_INDEX];
  for (i=0;i<SIZE_INDEX;i++)
   local_index[i]=0;

  for (i=i_start;i<i_end;i++) {
     tmp = S[i]%SIZE_INDEX;
     local_index[tmp]++;
  }

  for (i=0;i<SIZE_INDEX;i++)
  {
   #pragma omp atomic
   index[i] += local_index[i];
  }

 }
}
```

Problem 4 (2.5 points)

We ask you to write two additional parallel OpenMP implementations of the code that computes the `histogram`, this time using *task decomposition* strategies:

1. (1 point) Write an efficient OpenMP parallel version of the histogram program in Problem 3 using *explicit tasks*.

   Solution: A possible implementation:

```
...
void histogram(unsigned int *S, int n, unsigned int *index)
{
 unsigned int i, tmp;

 #pragma omp parallel
 #pragma omp single
 #pragma omp taskloop private(tmp)
 for (i=0; i<n; i++) {
   tmp = S[i]%SIZE_INDEX;
   #pragma omp atomic
   index[tmp]++;
 }
}
```

2. (1.5 points) Write a *recursive tree task decompostion* with *cutoff* based on the depth of the recursivity for the following code:

```
#define SIZE_INDEX 256
#define N 1024*1024*1024
#define BASE_SIZE 512
#define CUTOFF 3

void histogram(unsigned int *S, int n, unsigned int *index) {
 unsigned int i, tmp;
 for (i=0; i<n; i++) {
   tmp = S[i]%SIZE_INDEX;
   index[tmp]++;
} }

void histogram_rec(unsigned int *S, int n, unsigned int *index) {
  unsigned int n2=n/2;

  if (n > BASE_SIZE) {
    histogram_rec(S, n2, index);
    histogram_rec(&S[n2], n-n2, index);
  } else {
      histogram(S, n, index);
 }
}

void main() {
  unsigned int S[N];
  unsigned int index[SIZE_INDEX];
  ...
  // Here we have initialized S to random numbers and index to 0's
  histogram_rec(S, N, index);
  ...
}
```

Solution: A possible implementation:

```
...
void histogram(unsigned int *S, int n, unsigned int *index) {
 unsigned int i, tmp;
 for (i=0; i<n; i++) {
   tmp = S[i]%SIZE_INDEX;
   #pragma omp atomic
   index[tmp]++;
 }
}

void histogram_rec(unsigned int *S, int n, unsigned int *index, int depth) {
  unsigned int i, tmp, n2;

  if (n > BASE_SIZE) {
     n2 = n/2;
     if ( !omp_in_final() ) {
       #pragma omp task final(depth>=CUTOFF)
       histogram_rec(S, n2, index, depth+1);
       #pragma omp task final(depth>=CUTOFF)
       histogram_rec(&S[n2], n-n2, index, depth+1);
     }
     else {
       histogram_rec(S, n2, index, depth+1);
       histogram_rec(&S[n2], n-n2, index, depth+1);
     }
  else {
      histogram(S, n, index);
 }
}

void main() {
  unsigned int S[N];
  unsigned int index[SIZE_INDEX];
  ...
  // Here we have initialized S to random numbers and index to 0's
  #pragma omp parallel
  #pragma omp single
  histogram_rec(S, N, index, 0);
  ...
}
```

PAR – Final Exam Theory/Problems– Course 2022/23-Q2

June 21^{th}, 2023

Problem 1 (2.5 points)

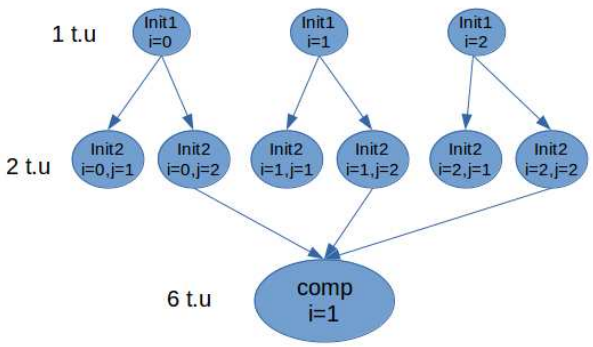
Given the following code including *Tareador* task annotations:

```
#define N 3
int I[N][N];
int R[N][N];
...
for (int i=0; i<N; i++) {
    tareador_start_task ("init1");
    I[i][0] = foo(i);   // cost = 1 t.u.
    tareador_end_task ("init1");

    for (int j=1; j<N; j++) {
       tareador_start_task ("init2");
       I[i][j] = j*I[i][0]; // cost = 2 t.u.
       tareador_end_task ("init2");
} }

for (int i=1; i<N-1; i++) {
    tareador_start_task ("comp");
    for (int j=i+1; j<N; j++)
       R[i][j] = i*(I[i-1][j]+I[i][j]+I[i+1][j]); // cost = 6 t.u
    tareador_end_task ("comp");
}
```

Assume: a) the execution time for tasks `init1` and `init2` is 1 and 2 t.u., respectively; b) the execution time for each iteration of the loop body for task `comp` is 6 t.u.; c) the rest of code has negligible cost; d) all scalar variables (i.e. all variables except matrices `I` and `R`) are stored in registers; e) function `foo` does not access any memory location during its execution; and f) `N` with the value defined in the code above. **We ask you** to:

1. Draw the Task Dependence Graph (*TDG*), indicating the cost of each task. Label each task with the values of i, and when necessary, with the pair of values i and j.

   Solution:



2. Based on the *TDG*, compute the values for T_1 and T_∞.

   Solution:

   $T_1 = 1 \times 3 + 2 \times 6 + 6 \times 1 = 21$ time units.

   $T_\infty = 1 + 2 + 6 = 9$ time units , determined by the critical path: *init1, init2, comp.*

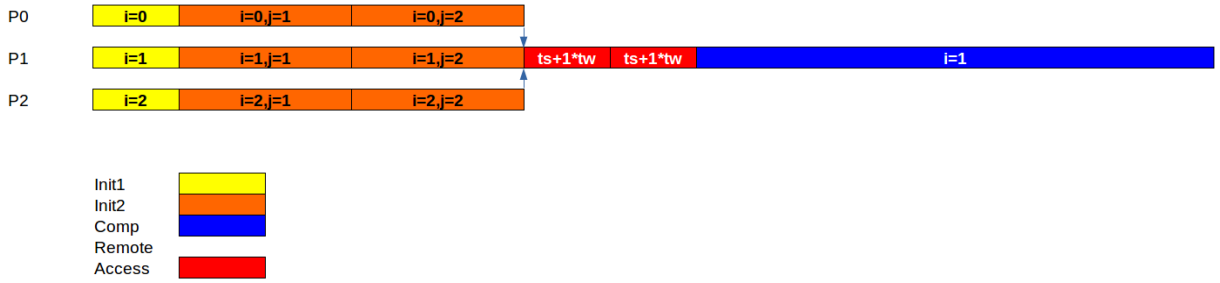
Next consider the data sharing model explained in class based on a distributed memory architecture in which we consider that local memory accesses do not introduce any overhead, but an access to data in different processors introduces a data–sharing overhead, determined by $t_{comm} = t_s + m \times t_w$; t_s is the "start-up" time, t_w is transfer time for one element and m the number of elements transferred during the remote access. Also, according to the data sharing model: at any time, each processor can simultaneously make one remote access to a different processor and serve one remote access from another processor.

Assuming a) the number of processors p is equal to N; b) matrices `I` and `R` are already distributed in the memory of each processor when the computation starts, following a *block row data decomposition*; c) the resulting matrices should remain distributed in the same way at the end of the execution; and d) tasks are assigned to processors following the *owner-computes rule* so that a task is executed by the processor that owns the memory that holds the output of that task. **We ask you:**

3. Draw the timeline for the execution with $p = 3$, showing both the computational and remote memory access costs.

   Solution:

   Block row data decomposition of a matrix with a number of rows N equal to P means that each processor i will have row i of the matrix. Therefore, tasks `init1` and `init2` accessing row i of matrix I are assigned to processor i because their own row i of that matrix. Task `comp` access row 1 of matrix R. Then, it is assigned to processor 1. Following owner-compute rules, the mapping and execution of the tasks follows:



4. Obtain the expression that determines the parallel execution time on $p = 3$ processors (T_p), as a function of t_s and t_w, assuming the task durations obtained before.

   Solution:

   $T_p = 1 + 2 \times 2 + 2 \times (t_s + 1 \times t_w) + 6 = 11 + 2 \times (t_s + 1 \times t_w)$ time units

Problem 2 (2.5 points)

Given the following recursive sequential code:

```
#define N_MAX (1<<29) // 512*1024*1024
struct t_person{
   t_data data; // personal information of bank client
   float balance; // current balance for client
   float interest; // interest for client
};
struct t_person best_client;

void find_best_client(struct t_person *people, int n) {
   int n2 = n/2;
   if (n==1) {
      if (best_client.balance < people[0].balance)
          best_client = people[0]; // copy all info of the person to best_client
   } else {
       find_best_client(people, n2);
       find_best_client(people+n2, n-n2);
} }

int main() {
   struct t_person bank_info[N_MAX];
   ...
   best_client.balance=0.0;
   find_best_client(bank_info, N_MAX);
   ...
}
```

Write an OpenMP version following a tree recursive task decomposition, taking into account the task creation overheads and minimizing both task synchronizations and synchronizations to update shared variables.

A Solution:

```
... // same declarations

#define CUTOFF (4)

void find_best_client(struct t_person *people, int n, int depth) {
   int n2 = n/2;
   if (n==1) {
      if (best_client.balance < people[0].balance) // FIRST TEST (reduce # criticals)
      #pragma omp critical
      {
        if (best_client.balance < people[0].balance) // SECOND TEST (necessary!)
          best_client = people[0]; // copy all info of the person to best_client
      }
   } else {
      if (omp_in_final())
      {
       find_best_client(people, n2, depth);
       find_best_client(people+n2, n-n2, depth);
      } else {
       #pragma omp task final(depth>=CUTOFF)
       find_best_client(people, n2, depth+1);
       #pragma omp task final(depth>=CUTOFF)
       find_best_client(people+n2, n-n2, depth+1);
       // NO TASKWAIT
      }
} }
```

```
int main() {
   struct t_person bank_info[N_MAX];
   ...
   best_client.balance=0.0;
   #pragma omp parallel
   #pragma omp single
   find_best_client(bank_info, N_MAX, 0);
   ...
}
```

Problem 3 (2.5 points)
Given the following sequential C code:

```
#define N 10000000
#define NUMBINS 100
int input[N];
int histogram[NUMBINS];

int findMax(int *v); // returns the maximum value encountered in vector v
int findMin(int *v); // returns the minimum value encountered in vector v
...
int min = findMin(input);
int max = findMax(input);
int binInterval = (max-min)/NUMBINS;

for (int i=0; i<N; i++) {
    int bin = (input[i]-min)/binInterval;
    histogram[bin]++;
}
```

We ask you to:

1. Implement a parallel *OpenMP* version of the code that follows an ***output block geometric data decomposition***, where synchronization overheads are minimized and parallelism is maximized.

   Solution:

```
#define N 10000000
#define NUMBINS 100
int input[N];
int histogram[NUMBINS];

int findMax(int *v); // returns the maximum value encountered in vector v
int findMin(int *v); // returns the minimum value encountered in vector v
...
int min = findMin(input);
int max = findMax(input);
int binInterval = (max-min)/NUMBINS;

#pragma omp parallel
{
 int id = omp_get_thread_num();
 int nths = omp_get_num_threads();
 int BS = NUMBINS / nths;
 int mod = NUMBINS % nths;
 int start = id*BS;
 int end = start + BS;
 if (mod > 0) {
   if (id < mod) {
      start += id;
```

```
      end = start + BS + 1;
    }
    else {
      start += mod;
      end += mod;
    }
  }
  for (int i=0; i<N; i++) {
     int bin = (input[i]-min)/binInterval;
     if (bin >= start && bin < end)
         histogram[bin]++;
  }
}
```

2. Assume that the processor's cache line size is 64 bytes, also that the size of the `int` data type is 4 bytes and the initial addresses of `input` and `histogram` vectors are both aligned with the start of a cache line. Write a new parallel *OpenMP* version of the code (without changing the definition of the used data structures) to avoid *false sharing* overheads.

   Solution:

   The cache line size is 64 bytes, so the total number of elements from `histogram` that fit in one cache line is 16. In order to avoid false sharing we propose a block-cyclic data decomposition in such a way that each block has a number of elements that fit in one cache line, so each thread should be allocated a block of size 16.

```
#define N 10000000
#define NUMBINS 100
#define CACHE_LINE_SIZE 64
int input[N];
int histogram[NUMBINS];

int findMax(int *v); // returns the maximum value encountered in vector v
int findMin(int *v); // returns the minimum value encountered in vector v
...
int min = findMin(input);
int max = findMax(input);
int binInterval = (max-min)/NUMBINS;

#pragma omp parallel
{
 int id = omp_get_thread_num();
 int nths = omp_get_num_threads();
 int BS = CACHE_LINE_SIZE / sizeof(int);

 for (int i=0; i<N; i++) {
    int bin = (input[i]-min)/binInterval;
    if ((bin/BS)%nths == id)
        histogram[bin]++;
 }
}
```

Problem 4 (2.5 points) Assume a multiprocessor system composed of two NUMA nodes, each with 16 GB of main memory. Each NUMA node has two cores (NUMAnode0: core0-1, NUMAnode1: core2-3), each core with a private cache of 4 MB. Cache and memory lines are 32 bytes wide and data coherence in the system is maintained using Write-Invalidate protocols, with a Snoopy attached to each cache memory to provide MSI coherency within each NUMA node and a MSU directory-based coherence among the two NUMA nodes.

Given the following parallel *OpenMP* code excerpt to be executed on the described system:

```
1   #define IMAGESIZE 128*128
2   #define MAXITERS 100
3   typedef float tpixel[3]; // r,g,b values
4   tpixel image[IMAGESIZE], result[IMAGESIZE];
    ...
5   initialization (result); // on core 0
6   #pragma omp parallel num_threads(3)
7   {
8     int id = omp_get_thread_num();
9     for (int iter=0; iter<MAXITERS; iter++)
10        for (int i=0; i<IMAGESIZE; i++)
11           result[i][id] = apply_func (image[i][id],   // apply_func does not affect
12                                       result[i][id]); // any other variables
13  }
```

and assuming that: 1) vectors `image` and `result` are the only variables that will be stored in memory (the rest of variables will be all in registers of the processors); 2) the initial addresses of `image` and `result` vectors are aligned with the start of a cache line; 3) the size of a `float` data type is 4 bytes; and 4) $thread_i$ always executes on $core_i$, where i = [0,1,2], **we ask you to:**

1. Indicate how many entries in the directory structure are needed to store the coherence information of the `result` vector. In addition, for each entry indicate the number of bits needed and their function.

   Solution:

   Each element of the `result` vector has 12 bytes of total size ($3 \times 4bytes = 12bytes$), the total size of the vector would be: $12 \times (128 \times 128) = 12 \times 2^{14}$

   The number of memory lines needed to allocate the `result` vector is : $(12 \times 2^{14})/2^6 = 12 \times 2^8$

   For each memory line it is necessary to keep the status information (*M,S,U*), which can be stored in 2 bits and the presence bits (one bit per NUMA node, total = 2 bits).

   The total number of bits needed is : $12 \times 2^8 \times (2 + 2)$

2. Compute the total number of bits that are required to maintain the coherence information at cache level within each NUMA node. Indicate also the number of bits per cache line and their function.

   Solution:

   Each cache has a total size of 4 MB and the cache line size is 64 bytes, so the total number of lines per cache = $2^{22}/2^6 = 2^{16}$ cache lines per cache.

   For each cache line it is necessary to keep the status information (*M,S,I*), which can be stored in 2 bits.

   So finally, the total number of bits necessary to keep the information of the Snoopy MSI coherence protocol is: $2^{16}2 = 2^{17}$

3. Complete the table provided with information for each enumerated memory operation (after it is completed), from the previous code, executed by the threads in the parallel region. You should specify the relative number of the memory line affected (with respect to the start address of the *result* vector, i.e. *result[0][0]* affects line 0), hit or miss, bus transactions of the MSI Snoopy protocol, state of the cache line in the MSI Snoopy protocol, whether there are or not commands from the Directory protocol between NUMA nodes (yes or no), state of the memory line in the Directory and, Presence bits.
4. The speedup achieved when executing the previous parallel code, with three threads, is far below 3x. Explain briefly the reason and suggest any modification to the data structures in order to improve its performance. Re-write the necessary code.

Action	Memory line	Cache Miss/Hit	Bus command	State MC_0	State MC_1	State MC_2	NUMA comands (y/n)	Directory State	Presence bits
Initial state result[0][0:2]				M	-	-		M	01
$thread_1$ reads result[0][1]									
$thread_2$ reads result[0][2]									
$thread_1$ writes result[0][1]									
$thread_0$ reads result[0][0]									

Solution									
Action	Memory line	Cache Miss/Hit	Bus command	State MC_0	State MC_1	State MC_2	NUMA comands (y/n)	Directory State	Presence bits
Initial state result[0][0:2]				M	-	-		M	01
$thread_1$ reads result[0][1]	0	Miss	$BusRd_1$/ $Flush_0$	S	S	.	n	S	01
$thread_2$ reads result[0][2]	0	Miss	$BusRd_2$	S	S	S	y	S	11
$thread_1$ writes result[0][1]	0	Hit	$BusUpgr_1$	I	M	I	y	M	01
$thread_0$ reads result[0][0]	0	Miss	$BusRd_0$/ $Flush_1$	S	S	I	n	S	01

Solution:

Each element of the `result` vector has 3 fields of `float` data type, computing a total size of 12 bytes per element. The size of a cache line is 64 bytes. Given that each element of the result vector is being updated by the three threads, a false sharing situation occurs with high probability (every time more than one thread is updating the same entry at the result vector).

In order to avoid such inefficiency we can apply "padding", that is add some extra bytes to prevent different threads from accessing the same cache line for updating it concurrently. One possible solution could be to add extra bytes following each field of the `tpixel` structure in order to complete a cache line (number of extra bytes = 64 - 4 = 60 bytes, which makes 60/4 = 15 extra elements) In this case we need to substitute line 3 of the code by:

```
1   #define IMAGESIZE 128*128
2   #define MAXITERS 100
3   typedef float tpixel[3][1+15]; // r ,g ,b values with padding
4   tpixel image[IMAGESIZE], result[IMAGESIZE];
    ...
5   initialization (result); // on core 0
6   #pragma omp parallel num_threads(3)
7   {
8     int id = omp_get_thread_num();
9     for (int iter=0; iter<MAXITERS; iter++)
10        for (int i=0; i<IMAGESIZE; i++)
11           result[i][id][0] = apply_func (image[i][id][0],   // apply_func does not affect
```

```
12                                      result[i][id][0]); // any other variables
13   }
```

PAR – Final Exam Laboratory– Course 2022/23-Q2

June 21^{th}, 2023

Problem 1: Lab 1 (2.5 points)

Given the following outputs obtained after the interactive execution of `pi_omp` with different number of threads:

```
par@boada-7> OMP_NUM_THREADS=1 /usr/bin/time ./pi_omp 1000000000
Number pi after 1000000000 iterations with 1 threads = 3.141592653589828
Execution time (secs.): 2.368062
2.36user 0.01system 0:02.37elapsed 99%CPU (0avgtext+0avgdata 4320maxresident)k
0inputs+0outputs (1major+275minor)pagefaults 0swaps

par@boada-7> OMP_NUM_THREADS=2 /usr/bin/time ./pi_omp 1000000000
Number pi after 1000000000 iterations with 2 threads = 3.141592653589845
Execution time (secs.): 1.186354
2.36user 0.00system 0:01.19elapsed 198%CPU (0avgtext+0avgdata 4668maxresident)k
0inputs+0outputs (1major+381minor)pagefaults 0swaps

par@boada-7> OMP_NUM_THREADS=4 /usr/bin/time ./pi_omp 1000000000
Number pi after 1000000000 iterations with 4 threads = 3.141592653589845
Execution time (secs.): 1.188191
2.36user 0.01system 0:01.19elapsed 198%CPU (0avgtext+0avgdata 4672maxresident)k
0inputs+8outputs (1major+306minor)pagefaults 0swaps
```

1. Explain the meaning the four first values reported by the `/usr/bin/time` command when executed with a single thread: *2.36user 0.01system 0:02.37elapsed 99%CPU*.

   Solution:

   They are the user CPU time, system CPU time, elapsed time and % of elapsed time dedicated to CPU time (user+system CPU time*100/elapsed time), respectively.

2. When executed with 2 and 4 threads, explain why *2.36user* does not change with respect to the value reported for a single thread, and why the *198%CPU* is the same for 2 and 4 threads.

   Solution:

   The value *2.36user* corresponds to the addition of the CPU time of each of the 2 or 4 threads during the execution. Each thread lasts of $2.36/nt$ approx, being nt the number of threads.

   The %CPU is close to 200% for 2 threads because the CPU time remains the same while the elapsed time is half the elapsed time of a single thread execution. In the case of 4 threads is still the same because we only have two cores in interactive mode, and the elapsed time remains the same than the case with 2 threads.

Problem 2: Lab 3 (2.5 points)

Assuming the following task decomposition strategies to parallelize the *Mandelbrot set*, which one do you think would be more efficient? Please reason your answer taking into account the number of tasks generated/executed and their granularity for each strategy.

```
#pragma omp parallel    // Code 1
#pragma omp single
 for (int row = 0; row < height; ++row)
   for (int col = 0; col < width; ++col) {
     #pragma omp task firstprivate(row,col)
     {
             . . .
   } }
```

```
#pragma omp parallel    // Code 2
#pragma omp single
 #pragma omp taskloop
 for (int row = 0; row < height; ++row) {
   for (int col = 0; col < width; ++col) {
                . . .
 } }
```

Solution:

Code 2 is much more efficient than Code 1. Code 1 corresponds with the first point strategy implementation that you evaluated in the laboratory session. One of the threads in Code 1 has to create a huge number of very fine-grain tasks sequentially. This incurs a significant overhead of task creation. Code 2 corresponds with the row strategy implementation with taskloop that you also evaluated in the laboratory session. This reduces the number of tasks created, which usually is proportional to the number of threads in the parallel region and much less than the number of tasks created in Code 1. The task creation overhead is significantly reduced with larger task granularities.

Problem 3: Lab 4 (2.5 points)

Consider the following sequential *multisort* code.

```
1   void merge(long n, T left[n], T right[n], T result[n*2], long start, long length) {
2    if (length < MIN_MERGE_SIZE*2L) { // Base case
3       basicmerge(n, left, right, result, start, length);
4    } else { // Recursive decomposition
5       merge(n, left, right, result, start, length/2);
6       merge(n, left, right, result, start + length/2, length/2);
7   }}
8
9  void multisort(long n, T data[n], T tmp[n]) {
10    if (n >= MIN_SORT_SIZE*4L) {     // Recursive decomposition
11       multisort(n/4L, &data[0], &tmp[0]);
12       multisort(n/4L, &data[n/4L], &tmp[n/4L]);
13       multisort(n/4L, &data[n/2L], &tmp[n/2L]);
14       multisort(n/4L, &data[3L*n/4L], &tmp[3L*n/4L]);
15
16       merge(n/4L, &data[0], &data[n/4L], &tmp[0], 0, n/2L);
17       merge(n/4L, &data[n/2L], &data[3L*n/4L], &tmp[n/2L], 0, n/2L);
18
19       merge(n/2L, &tmp[0], &tmp[n/2L], &data[0], 0, n);
20    } else     // Base case
21       basicsort(n, data);
22 }
23 void main() {
24   ...
25   multisort(n,elements,elements_tmp);
36   ...
37 }
```

Let's suppose that you parallelize it following a tree strategy decomposition with a given cut-off level. Assuming the laboratory problem size, 1024K elements to sort, and MIN_SORT_SIZE and MIN_MERGE_SIZE equal to 256. Answer the following questions:

1. Assume you want to use the `final(depth>=cut_off)` clause in the task pragmas, being "depth" the recursion level. Indicate where (i.e, in the sequential code line number above) you will add the task creation control to continue creating tasks or not, and write the code line to do it.

   Solution:

   Substitute the code at lines 5-6 by the following code structure:

   ```
   11  if (!omp_in_final())
       {
         Code with tasks
       } else {
         Code without tasks
       }
   ```

Also, substitute the code at lines 11-19 with the same code structure.

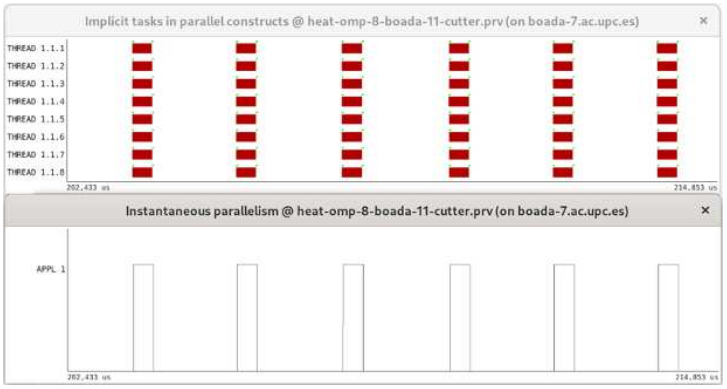
2. Assume now you are using task with depend clauses (with no cut-off control). Indicate the synchronisation (`depend`s clauses, `taskwait`s or/and `taskgroup`s) that you need to add in your tree-recursive task decomposition (i.e the sequential code line number, before and/or after you will insert them) when using tasks with depend clauses to guarantee the functionality of the *multisort* program.

   Solution:

   No solution provided. Look at your laboratory deliverable and feedback.

Problem 4: Lab 5 (2.5 points)

Assume that you have parallelized *Jacobi* solver function with the data decomposition seen at the laboratory. You run *modelfactors* and use the instantaneous parallelism and implicit tasks in parallel constructs hints with *Paraver* to figure out why *Jacobi* application is not scaling properly. The "zoom in capture" of part of the execution trace for 8 threads of the first implementation version of heat application with *Jacobi* solver follows (top: implicit tasks in parallel constructions, bottom: instantaneous parallelism):



We ask you to briefly explain your answer to the following questions:

1. What do you observe with instantaneous parallelism hint at the view of *Paraver*? What is the instantaneous parallelism achieved during the *Jacobi* solver function and during the execution of the other parts of the code?

   Solution: The amount of useful work done (state running) by all the threads at a certain instant of time. That is, if two threads are running, instantaneous parallelism is 2. The instantaneous parallelism achieved by *Jacobi* is about *nt*, being *nt* the number of threads. However, it is 1 for other parts of the code because they are not parallelized.

2. How did you solve the lack of parallelism and performance problem of the initial *Jacobi* implementation to achieve linear scalability?

   Solution:

   No solution provided. Look at your laboratory deliverable and feedback.

PAR – Final Exam Theory/Problems– Course 2023/24-Q1

January 17^{th}, 2024

Problem 1 (6.0 points)

Given the following code fragment written in C:

```
int m[N][N];
...
for (int i=0; i<N; i++) {
   for (int k=0; k<N; k++) {
       m[i][k] = calculation (m[N-1-i][k], m[i][k]); // 100 time units
   }
 }
```

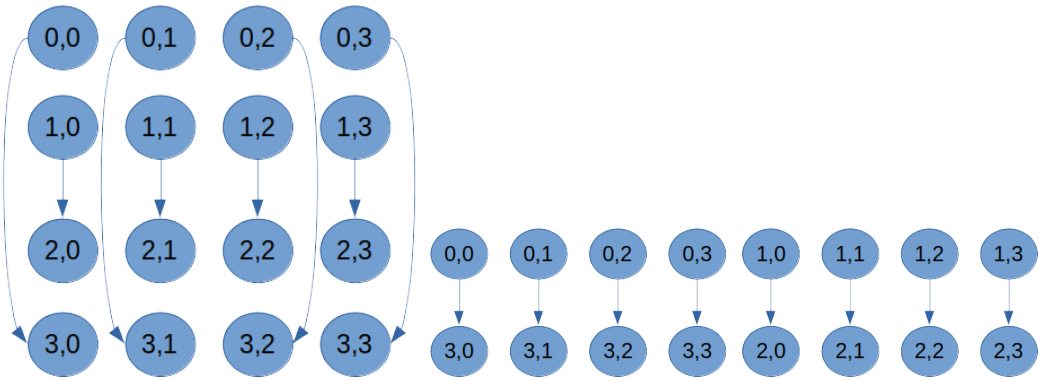
We ask you to:

1. (1.0 Points) Assume 1) a very fine-grain strategy in which a task is defined for each single k loop iteration, with cost equal to 100 t.u. (time units); 2) function `calculation` only reads the two values received as arguments and does not modify any other memory locations; and 3) the rest of variables in the program are stored in registers of the processors. **Also assume $N = 4$ for the following two subquestions:**

   (a) Draw the *Task Dependency Graph (TDG)*, identifying each task in the *TDG* with a label (i, k), being i and k the values of the `i` and `k` loop control variables, respectively.

   Solution:

   Two possible views of the TDG.



   (b) Compute the values for the metrics T_1, T_∞ and P_{min}.

   Solution:

   $T_1 = 16 \times 100 = 1600$ time units.

   $T_\infty = 2 \times 100 = 200$ time units.

   $P_{min} = 8$.

2. (2.5 Points) Write an *OpenMP* parallel version of the code following the most appropriate *geometric data decomposition strategy* for matrix `m` considering that your parallel code should: a) minimize the synchronization overhead among implicit tasks based on the previous *TDG* analysis; and b) guarantee that the load unbalance is limited to N elements (i.e. the number of elements in a row or column of the matrix). Assume that matrix `m` is allocated in memory by rows and that N is a multiple of 2 and not necessarily multiple of the number of processors to be used to execute the program in parallel.

   Solution:

```
int m[N][N];
...
```

```
#pragma omp parallel
{
 int id = omp_get_thread_num();
 int nths = omp_get_num_threads();
 int BS = N/nths;
 int mod = N%nths;
 int start = id*BS;
 int end = start + BS;
 if (mod > 0) {
    if (id < mod) {
       start += id;
       end = start + BS + 1;
    }
    else {
       start += mod;
       end += mod;
    }
 }
 for (int i=0; i<N; i++) {
   for (int k=start; k<end; k++) {
       m[i][k] = calculation (m[N-1-i][k], m[i][k]);
   }
 }
```

We apply a *block geometric data decomposition by columns*. There are RAW (true) dependencies among elements from the same column, consequently blocks are defined by columns, ensuring no need for synchronization among implicit tasks. The size of the blocks are adjusted as N is not guarantee to be a multiple of the number of threads. In this way, the unbalance is limited to N elements (an entire column).

3. (2.5 Points) Modify the previous *OpenMP* parallel version or write a new one and, if necessary, change the definition of matrix `m` to follow the most appropriate *geometric data decomposition strategy* considering that: a) the program is executed on a parallel machine where memory lines are 128 bytes long; b) matrix `m` is allocated in memory so that its first element is aligned to the start of a memory line and `sizeof(int)` = 4 bytes; and c) you can not consider that N is multiple of the size of one element. Your proposed solution should: a) maximize parallelism among implicit tasks; b) maximize data locality and reduce coherence traffic; and c) minimize load unbalance.

   Solution:

```
#define MEMLINESIZE 128
#define X MEMLINESIZE/sizeof(int)
int m[N][N + (X - N % X)];
...
#pragma omp parallel
{
 int id = omp_get_thread_num();
 int nths = omp_get_num_threads();
 int BS = (N/2)/nths;
 int mod = (N/2)%nths;
 int start = id*BS;
 int end = start + BS;
 if (mod > 0) {
    if (id < mod) {
       start += id;
       end = start + BS + 1;
    }
    else {
```

```
        start += mod;
        end += mod;
     }
  }

  for (int i=start; i<end; i++) {
    for (int k=0; k<N; k++) {
        m[i][k] = calculation (m[N-1-i][k], m[i][k]);
        m[N-1-i][k] = calculation (m[i][k], m[N-1-i][k]);
    }
  }
}
```

In the proposed solution we apply a *block geometric data decomposition strategy by rows* on the first half of the matrix `m`. The rows on the second half are processed by the thread that processes the mirror row in the first half of the matrix, so the code is modified accordingly. In this way dependencies are preserved and there is no need for synchronization between implicit tasks. In order to minimize coherence traffic, we avoid false sharing by adding padding as extra columns at the end of each row of the matrix, so that we complete an entire memory line in case N is not a multiple of the memory line size. In this way we avoid false sharing. Data locality is preserved as the matrix processing is performed by rows. Finally, unbalance is minimized to two rows (2N elements) in the worst case.

Another solution could be to modify the proposed solution of the previous part, and apply a *block cyclic gemotreci data decomposition by columns*:

```
#define MEMLINESIZE 128
#define BS MEMLINESIZE / sizeof(int)
int m[N][N + (BS - (N % BS)];
...
#pragma omp parallel
{
 int id = omp_get_thread_num();
 int nths = omp_get_num_threads();

 for (int i=0; i<N; i++) {
   for (int kk=id*BS; kk<N; kk+=nths*BS) {
      for (int k=kk; k<min(kk+BS,N); k++) {
       m[i][k] = calculation (m[N-1-i][k], m[i][k]);
   }
 }
}
```

Padding is also added to avoid false sharing and consequently minimize coherence traffic. The size of the block ensures data locality. This solution has the disadvantage with respect to the previous proposed one, that the unbalance in the worst case could be up to (BS-1) columns, that is (BS-1)*N elements.

Problem 2 (4.0 points) Assume a NUMA system with two NUMAnodes, each NUMAnode with 16GB of main memory and 2 processors (cores). Each processor has its own local cache memory of 4MB (cache line size of 128 bytes). Data coherence is maintained using *Write-Invalidate MSI* protocols, with a Snoopy attached to each per–core local cache memory to provide coherency within each NUMAnode and directory-based coherence among the two NUMAnodes.

We ask you to:

1. (2.0 Points) Compute the total number of bits that are used by each snoopy to maintain the coherence between caches inside a NUMAnode and, the total number of bits in each node directory to maintain coherence among NUMAnodes.

   Solution:

   Intra-node coherence:

   The number of cache lines is calculated dividing the memory in a cache (4MB) by the cache line size (128B). Therefore: Number of cache lines $= \frac{4MB}{128B} = \frac{4\times 2^{20}}{2^7} = 2^{15}$ entries

   We need 2 bits per entry, so the total number of bits used by each snoopy is: 2^{16}

   Inter-node coherence:

   The number of entries in the directory structure per NUMA-node is calculated dividing the memory in a NUMA-node (16GB) by the memoy line size (128B). Therefore:

   Number of Directory Entries $= \frac{16GB}{128B} = 2^{27}$ entries

   Each entry has 4 bits: 2 Presence bits and 2 bits for the Status

   Thus, with 4 bits per directory entry, the total number of bits per NUMAnode is: $4 \times 2^{27} = 2^{29}$

2. (2.0 Points) Let's consider the following definition of matrix `m` with a given value of `N`:

   ```
   #define N 32
   int m[N][N];
   ```

   Assume that a) matrix `m` is allocated in memory aligned to the start of a memory line and b) `sizeof(int)` = 4 bytes.

   The code initializing matrix `m` is executed entirely by $core_0$ (on $NUMAnode_0$) so at the end of the initialization the entire matrix is loaded into the cache of $core_0$. Consequently, all memory lines and cache lines that hold matrix `m` are in *Modified state* with *presence bits = 01* in the directory of the *home* NUMAnode. **We ask you to:** fill in the attached table the sequence of processor commands (Core), cache Miss or Hit, bus transactions within NUMA nodes indicating clearly which attached Snoopy generated the command (Snoopy), state for each cache line affected (for each cache memory of the system, MC_0 to MC_3), state for each memory line (Directory state) and the presence bits, to keep cache coherence, AFTER the execution of each of the following of commands:

   - $core_0$ from $NUMAnode_0$ reads `m[0][0]`
   - $core_0$ from $NUMAnode_0$ reads `m[1][24]`
   - $core_3$ from $NUMAnode_1$ writes `m[0][24]`
   - $core_3$ from $NUMAnode_1$ reads `m[1][24]`
   - $core_0$ from $NUMAnode_0$ writes `m[0][0]`
   - $core_3$ from $NUMAnode_1$ writes `m[1][24]`

Solution:

Action	Core	Cache Miss/Hit	Snoopy	State MC_0	State MC_1	State MC_2	State MC_3	Directory State	Presence bits
$core_0$ reads m[0][0]									
$core_0$ reads m[1][24]									
$core_3$ writes m[0][24]									
$core_3$ reads m[1][24]									
$core_0$ writes m[0][0]									
$core_3$ writes m[1][24]									

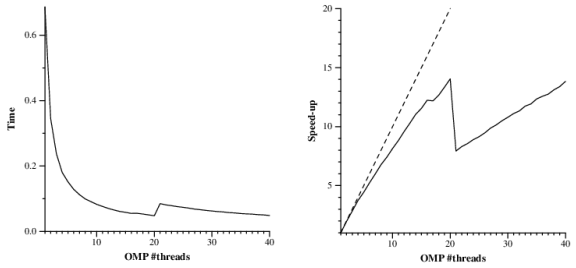
Action	Core	Cache Miss/Hit	Snoopy	State MC_0	State MC_1	State MC_2	State MC_3	Directory State	Presence bits
$core_0$ reads m[0][0]	$PrRd_0$	Hit	-	M	-	-	-	M	01
$core_0$ reads m[1][24]	$PrRd_0$	Hit	-	M	-	-	-	M	01
$core_3$ writes m[0][24]	$PrWr_3$	Miss	$BusRdX_3$ $BusRdX$ in $NumaNode_0$ issued by Hub $Flush_0$	I	-	-	M	M	10
$core_3$ reads m[1][24]	$PrRd_3$	Miss	$BusRd_3$ $BusRd$ in $NumaNode_0$ issued by Hub $Flush_0$	S	-	-	S	S	11
$core_0$ writes m[0][0]	$PrWr_0$	Miss	$BusRdX_0$ $BusRdX$ in $NumaNode_1$ issued by Hub $Flush_3$	M	-	-	I	M	01
$core_3$ writes m[1][24]	$PrWr_3$	Hit	$BusUpgr_3$ $BusUpgr$ in $NumaNode_0$ issued by Hub	I	-	-	M	M	10

PAR – Final Exam Laboratory– Course 2023/24-Q1

January 17^{th}, 2023

Problem 1: Lab 1 (2.5 points)

Let's assume the `pi_omp` code in *Lab 1*. Below we include the execution time and speedup scalability plots obtained with the `submit-strong-omp.sh` script when setting `np_MAX=40`. Recall that this script executes the parallel code using from 1 to `np_MAX` threads. **We ask you to:** Briefly explain the reason why the speed-up goes down abruptly when going from 20 to 21 *OpenMP* threads, and why the performance for 20 and 40 *OpenMP* threads is very similar.



Solution: No solution provided. Look at the Atenea survey of Questions laboratory 1 - Session 1 - third question.

Problem 2: Lab 3 (2.5 points)

1. Assume the following task decomposition strategies for the `dot_product` code:

```
// Strategy 1
float result = 0.0;
#pragma omp parallel
#pragma omp single
#pragma omp taskloop reduction(+:result)
for (int i = 0; i < n; ++i) {
   result += x[i] + y[i];
}
```

```
// Strategy 2
float result = 0.0;
#pragma omp parallel
#pragma omp single
#pragma omp taskgroup task_reduction(+: result)
for (int i = 0; i < n; ++i) {
   #pragma omp task in_reduction (+:result)
   result += x[i] + y[i];
}
```

   We ask you to: Indicate which of the two strategies would obtain better scalability. Justify briefly your answer.

   Solution:

   Strategy 2 applies a task decomposition with task granularity much finer than Strategy 1. Observe that in Strategy 2, tasks are created for every loop iteration, while in Strategy 1, tasks are created with a bunch of consecutive iterations (taskloop behaviour). The work performed at each loop iteration (accumulation on a variable with a sum up) do not justify the task creation at this level. We can affirm because in Laboratory 3, where for Mandelbrot set calculation (which involved a greater number of operations), the Point strategy suffered from a great task creation overhead, and Row strategy showed better performance.

2. Take a look at the two versions of *Modelfactors* tables generated after executing the *Mandelbrot* application parallelized using the *Point* strategy and `taskloop` pragma to create the explicit tasks.

   We ask you to: Analyze the main differences between the two executions and indicate which is the optimization applied to *Parallelization 2*. Reason briefly your answer.

   Solution: No solution provided. Look at your laboratory deliverable and feedback

Statistics about explicit tasks in parallel fraction					
Number of processors	1	4	8	12	16
Number of explicit tasks executed (total)	3200.0	12800.0	25600.0	38400.0	51200.0
LB (number of explicit tasks executed)	1.0	0.93	0.84	0.53	0.48
LB (time executing explicit tasks)	1.0	0.95	0.95	0.95	0.96
Time per explicit task (average us)	129.09	33.54	18.3	12.66	9.68
Overhead per explicit task (synch %)	0.04	26.9	102.7	229.43	456.08
Overhead per explicit task (sched %)	0.33	3.84	12.88	20.52	30.87
Number of taskwait/taskgroup (total)	320.0	320.0	320.0	320.0	320.0

Figure 1: Modelfactors table for Parallelization 1

Statistics about explicit tasks in parallel fraction					
Number of processors	1	4	8	12	16
Number of explicit tasks executed (total)	3200.0	12800.0	25600.0	38400.0	51200.0
LB (number of explicit tasks executed)	1.0	0.51	0.62	0.75	0.93
LB (time executing explicit tasks)	1.0	0.98	0.98	0.97	0.97
Time per explicit task (average us)	129.1	33.49	18.22	12.69	9.71
Overhead per explicit task (synch %)	0.0	3.6	64.34	180.14	396.75
Overhead per explicit task (sched %)	0.31	2.29	10.68	18.35	28.53
Number of taskwait/taskgroup (total)	0.0	0.0	0.0	0.0	0.0

Figure 2: Modelfactors table for Parallelization 2

Problem 3: Lab 4 (2.5 points)

Consider the following sequential recursive version for the `dot_product` problem presented in the assignment for *Lab 4*:

```
#define N 512
#define MIN_SIZE 64

int iter_dot_product(int *A, int *B, int n);

int rec_dot_product(int *A, int *B, int n) {
   int res1, res2=0;
   if (n>MIN_SIZE) {
       int n2 = n / 2;
       res1 = rec_dot_product(A, B, n2);
       res2 = rec_dot_product(A+n2, B+n2, n-n2);
   }
   else res1 = iter_dot_product(A, B, n);
   return res1+res2;
}

void main() {
   int result;
   result = rec_dot_product(a, b, N);
}
```

We ask you to:

1. Assuming we only add the necessary OpenMP directives to parallelize the previous code following a Recursive Task Decomposition using a *Leaf* strategy, indicate which would be the **Maximum** *Instantaneous Parallelism* we can achieve. Reason your answer. **Note:** Including the parallel code in your answer is not mandatory.

   Solution: The maximum instantaneous parallelism that we can achieve is 1. Observe that the `rec_dot_product` function cannot return until `res1` and `res2` are calculated. So in the case of a leaf strategy, as tasks are created sequentially, and have to finish execution before returning the result, there won't be more than one task executing at any given moment.

2. Assuming we only add the necessary OpenMP directives to parallelize the previous code following a Recursive Task Decomposition using a *Tree* strategy, indicate which would be the **Maximum** *Instantaneous Parallelism* we can achieve. Reason your answer. **Note:** Including the parallel code in your answer is not mandatory.

   Solution: The maximum instantaneous parallelism that we can achieve is 8, after three levels of recursive calls and one task created per recursive call (two tasks per level). Note that after three recursive levels the base case of the recursivity is achieved, when 8 tasks can be potentially executed in parallel.

Problem 4: Lab 5 (2.5 points)

Assume a parallelization strategy for the Gauss-Seidel solver in *Lab5* that uses a *geometric block by rows data decomposition*. Remember that the code in the laboratory assignment included an argument, *userparam*, to be used to determine the number of blocks each thread has to process.

1. Indicate why *userparam* impacts directly on the amount of parallelism obtained. Reason your answer.

   Solution: No solution provided. Look at your laboratory deliverable and feedback.

2. Explain the reason why in *Lab5* assignment you needed to implement your own synchronization between threads.

   Solution:

   When working with implicit tasks, there is no way to specify dependencies using OpenMP clauses. Consequently, all the implicit tasks will be executed simultaneously without taking care of dependencies between blocks.

PAR – Final Exam Theory/Problems– Course 2023/24-Q2

June 11^{th}, 2024

Problem 1 (2.5 points) Given the following C program instrumented with `Tareador`:

```
#define BS  4
#define N   16
int ii, jj, i, j;
char stringMessage[128];
int M[N][N];

for (ii=0; ii<N; ii+=BS)
  for (jj=0; jj<N; jj+=BS) {
    sprintf(stringMessage,"CoP(%d,%d)",ii,jj);
    tareador_start_task(stringMessage);

    for (i=ii; i<min(ii+BS,N) ; i++)
      for (j=max(1,jj); j<min(jj+BS,N-1) ; j++)
        M[i][j]= M[i][j] + M[i][j-1] + M[i][j+1] + color_pixel(i,j); // 10 t.u.

    tareador_end_task(stringMessage);
   }
```

Assume each iteration of the innermost loop body takes 10 time units, `BS` and `N` are 4 and 16, respectively, only matrix `M` is stored in memory (rest of variables are in registers), and function `color_pixel` only computes a value function of the induction variables `i` and `j`. **We ask you to:**

1. Draw the TDG of the parallel strategy above. Indicate which is the cost of each task.

   Solution:

   ```
   CoP(0,0)  -> CoP(0,4)  -> CoP(0,8)  -> CoP(0,12)
    (120)        (160)        (160)        (120)

   CoP(4,0)  -> CoP(4,4)  -> CoP(4,8)  -> CoP(4,12)
    (120)        (160)        (160)        (120)

   CoP(8,0)  -> CoP(8,4)  -> CoP(8,8)  -> CoP(8,12)
    (120)        (160)        (160)        (120)

   CoP(12,0) -> CoP(12,4) -> CoP(12,8) -> CoP(12,12)
    (120)        (160)        (160)        (120)
   ```

2. Compute T_1, T_∞ and P_{min}.

   Solution:

   $T_1 = 4(120 + 160 + 160 + 120)t.u. = 2240t.u.$
   $T_\infty = 120 + 160 + 160 + 120t.u. = 560t.u.$
   $P_{min} = 4$

3. Write the expression that determines the execution time T_4, clearly indicating the contribution of the computation time T_4^{comp} and data sharing overhead T_4^{mov}, for the two following assignments of tasks to processors:

#proc	Assignment 1 (task id)	Assignment 2 (task id)
0	CoP(0,0), CoP(0,4), CoP(0,8), CoP(0,12)	CoP(0,0), CoP(4,0), CoP(8,0), CoP(12,0)
1	CoP(4,0), CoP(4,4), CoP(4,8), CoP(4,12)	CoP(0,4), CoP(4,4), CoP(8,4), CoP(12,4)
2	CoP(8,0), CoP(8,4), CoP(8,8), CoP(8,12)	CoP(0,8), CoP(4,8), CoP(8,8), CoP(12,8)
3	CoP(12,0),CoP(12,4),CoP(12,8),CoP(12,12)	CoP(0,12),CoP(4,12),CoP(8,12),CoP(12,12)

You can assume: 1) a distributed-memory architecture with 4 processors; 2) matrix `M`, is initially distributed by rows in Assignment 1 (N/BS consecutive rows per processor) and initially distributed by columns in Assignment 2 (N/BS consecutive columns per processor); 3) once the loop is finished, you don't need the return matrix to their original distribution; 4) data sharing model with communication time per message $t_{comm} = t_s + m \times t_w$, being t_s, m, t_w the start–up time, the number of elements, and transfer time of one element, respectively; 5) BS perfectly divides N; and 6) the execution time for a single iteration of the innermost loop body takes 10 t.u..

Assignment 1 Solution:

$T_4 = T_4^{comp} + T_4^{mov}$
$T_4^{mov} = 0$; All data is local. No communication is needed.
$T_4^{comp} = (2 \times ((BS-1) \times BS) + (\frac{N}{BS} - 2) \times (BS \times BS)) \times 10t.u.$;

Assignment 2 Solution:

$P = 4$;
$T_4 = T_4^{comp} + T_4^{mov}$

$T_4^{mov} = t_s + N \times t_w + (P-2) \times (t_s + BS \times t_w) + \frac{N}{BS} \times (t_s + BS \times t_w)$;

$T_4^{comp} = ((BS-1) \times BS + (P-2) \times (BS \times BS) + (\frac{N}{BS} - 1) \times (BS \times BS) + (BS-1) \times BS) \times 10t.u.$;

Note : Regarding to the T_4^{mov}, there is a initial communication of the right boundary (first column of next processor), that all processor but last one have to do. This communication can be done by all processors in parallel. Then, each processor (but first one) hast to read BS elements of the left boundary, produced by a task in previous processor (dependence).

Regarding to T_4^{comp}, the last row of tasks executed by processor $P-1$ has $BS \times BS - 1$ elements to process each, but this processor has to wait for $\frac{N}{BS} - 1$ tasks of previous processor that last each for $BS \times BS \times 10t.u.$, therefore, this time should be taken into account for the first three tasks of last processor. Then, it has to process $BS \times BS - 1$ elements of the very last task.

Problem 2 (1.5 points) Consider the following sequential program that performs a matrix multiplication ($C = A \times B$):

```
int A[L][M], B[M][N], C[L][N];
int main() {
   int l, n, m;
   ...
   // Matrix Mutiplication
   for (l=0; l<L; l++)
      for (n=0; n<N; n++)
         for (m=0; m<M; m++)
             C[l][n] += A[l][m]*B[m][n];
   ...
   // Output results
   for (l=0; l<L; l++)
      for (n=0; n<N; n++)
          printf("C[%d][%d]=%d\n",l,n,C[l][n]);
}
```

We ask you to:

1. Write an OpenMP version to parallelize `Matrix multiplication` computation code using an iterative task decomposition strategy where: 1) the granularity of the tasks should be 2 iterations of the middle loop (loop n); and 2) the synchronization overheads are minimized.

   A Solution:

   ```
   ...
   int A[L][M], B[M][N], C[L][N];

   int main() {
      int l, n, m;
      ...

      #pragma omp parallel private(l,m)
      #pragma omp single
        for (l=0; l<L; l++)
           #pragma omp taskloop grainsize(2) nogroup
           for (n=0; n<N; n++)
              for (m=0; m<M; m++)
                 C[l][n] += A[l][m]*B[m][n];
      ...

      for (l=0; l<L; l++)
         for (n=0; n<N; n++)
            printf("C[%d][%d]=%d\n",l,n,C[l][n]);

   }
   ```

   Note: Tasks are independent among them. Thus, it is not necessary to add any synchronization between tasks neither for solving data race conditions. On the other hand, it is important to add nogroup to avoid taskgroup sinchronization, that is not necessary and we want to minimize synchronization overheads.

2. Do you think parallelizing the `Output results` code can improve the performance of the program while obtaining the expected output? Justify briefly your answer.

A Solution: The output results code have to executed sequentially as the order of executing between the different `printf`'s have to be preserved. For this reason, there is no point on parallelizing it as it will not improve performance.

Problem 3 (1.5 points) Consider the following recursive program that copies an array into another one:

```
double X[N],Y[N];

int copy(double * __restrict__ input, double * __restrict__ output, int n) {
   if (n<=32)
      for (int i=0; i<n; i++) output[i] = input[i];
   else {
      copy(input, output, n/2);
      copy(input+n/2, output+n/2, n-n/2);
} }

int main() {
   ...
   copy(X,Y,N);
   ...
}
```

We ask you to: write a parallel OpenMP program that performs a parallel and efficient recursive task decomposition, reducing the task creation overheads and minimizing data sharing synchronizations. Note that `__restrict__` indicates that input and output are not overlapped in memory.

A Solution:

```
SOLUTION:
...
#define CUTOFF 5

double X[N],Y[N];

int copy(double *__restrict__ input, double * __restrict__ output, int n, int depth) {
   if (n<32)
       for (int i=0; i<n; i++) output[i] = input[i];
   else {
    if (!omp_in_final()) {
      #pragma omp task final(depth>=CUTOFF)
      copy(input, output, n/2, depth+1);
      #pragma omp task final(depth>=CUTOFF)
      copy(input+n/2, output+n/2, n-n/2, depth+1);
    } else {
      copy(input, output, n/2, depth+1);
      copy(input+n/2, output+n/2, n-n/2, depth+1);
    }
  }
}

int main() {
        #pragma omp parallel
        #pragma omp single
        copy(X,Y,N,0);
}
```

Problem 4 (4.5 points) Given the following C code excerpt:

```
#define MAXHISTO P // P is the number of cores
typedef struct {
    double total;
    unsigned long long num;
} telem;
telem histo[MAXHISTO];
double v[MAXELEM]; // MAXELEM is a very large value
int i, pos;
 ...
// histo initialization phase
for (i=0; i<MAXHISTO; i++) {
   histo[i].total = 0;
   histo[i].num = 0;
}
// histo computation phase
for (i=0; i<MAXELEM; i++) {
   pos = getpos (v[i], MAXHISTO);   // returns a value between 0 and MAXHISTO-1
   // complex update of field "total" from vector "histo" at position "pos"
   complex_update (&histo[pos].total, v[i]);
   histo[pos].num ++;
}
```

We ask you to:

1. (1.5 points) Add the necessary `OpenMP` pragmas and clauses to parallelize the code in *histo computation* phase, on `P` cores, making use of *implicit tasks* and applying an *INPUT geometric block data decomposition*. The value `MAXELEM` is not necessarily a multiple of `P`. Your solution should maximize parallelism among implicit tasks.

   Solution:

```
omp_lock_t locks[P];
int i, pos;
 ...
// histo initialization phase
for (i=0; i<MAXHISTO; i++) {
   histo[i].total = 0;
   histo[i].num = 0;
   omp_init_lock (&locks[i]);
}
// histo computation phase
#pragma omp parallel private (i, pos) num_threads(P)
{
 int id = omp_get_thread_num();
 int BS = MAXELEM / P;
 int mod = MAXELEM % P;
 int start = id * BS;
 int end = start + BS;
 if (mod > 0) {
    if (id < mod) {
       start += id;
       end = start + BS + 1;
    } else {
       start += mod;
       end += mod;
    }
 }
 for (i=start; i<end; i++) {
```

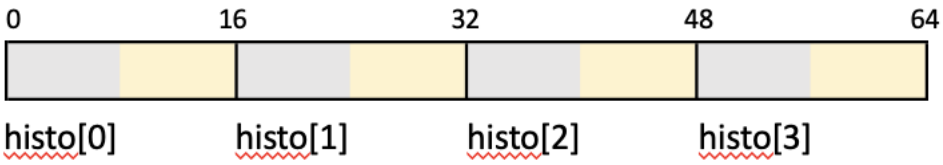
```
    pos = getpos (v[i], MAXHISTO);

    omp_set_lock (&locks[pos]);
    complex_update (&histo[pos].total, v[i]); // complex update of field "total" from vector
    omp_unset_lock (&locks[pos]);

    #pragma omp atomic
    histo [pos].num ++;
  }
 }
 ...
```

2. (1.5 points) Let's consider a shared-memory multiprocessor system composed by two NUMA nodes, each with two processors (cores) and 16 GBytes of shared main memory (MM). Each core has a private cache of 8MBytes. Cache and memory lines are 32 bytes wide. Cores 0 and 1 are allocated in NUMA node 0, and cores 2 and 3 are allocated in NUMA node 1. Data coherence is maintained using Write-Invalidate MSI protocols, with a Snoopy attached to each cache memory to provide coherency within each NUMA node and Write-Invalidate MSU directory-based coherence among the two NUMA nodes.

   (a) Consider `P=4` and draw a picture to show how many memory lines are necessary to allocate vector `histo` in the MM of the previously described system. You need to know that `sizeof(double) = 8 Bytes` and `sizeof(unsigned long long) = 8 Bytes`, the allocation of vectors `v` and `histo` are aligned to the start of memory line.

   Solution:

   To store vector `histo` it is necessary two memory lines (64 bytes):



   (b) **Compute the amount of bits taken by each snoopy** to maintain the coherence between caches inside a NUMA node and, **compute the amount of bits in each node directory** to maintain the coherence among NUMA nodes **ONLY for vector `histo`**.

   Solution:

   To store vector `histo` we need 2 memory lines. In order to keep coherency within a NUMA node using MSI Snoopy protocol, we need to keep 2 bits for each cache line. So the total number of bits needed by the MSI Snoopy protocol is **4 bits**.

   We need two entries in the directory structure to store the needed information to keep coherency. Each entry contain 2 bits for the state (MSU) and 2 bits for the sharers list. Consequently we need $2 * (2 + 2) =$ **8 bits** to keep coherency among NUMA nodes for vector `histo`.

   (c) Assuming that vector `histo` is entirely allocated in the MM of Numa Node 0, and all cache memories of the multiprocessor system are empty, **fill in the attached table** with the sequence of processor commands (Core), bus transactions within NUMA nodes (Snoopy), transactions yes/no between NUMA nodes (Directory), the presence bits, state for each cache and memory line, to keep cache coherence, AFTER the execution of each of the following sequence of commands:

   i. $core_2$ reads the contents of histo[1].total
   ii. $core_2$ writes the contents of histo[2].num
   iii. $core_1$ reads the contents of histo[0].total

3. (1.5 points) Let's assume that we want to execute the program on the shared-memory multiprocessor system described previously. Decide the most appropriate *geometric data decomposition* and write the parallel code in order to minimize synchronizations and reduce coherence traffic. You can also re-write the data structures.

Solution:

We define an OUTPUT geometric block data decomposition on vector `histo`, where the block size is equal to one element. In order to reduce coherence traffic generated by the false sharing when writing to different elements of `histo` but are allocated to the same cache line, we add padding (extra bytes) the the `telem` data structure. No synchronization is needed among implicit tasks as they update different elements of vector `histo`.

```
#define CACHE_LINE_SIZE 32
typedef struct {
    double total;
    unsigned long long num;
    char padding[CACHE_LINE_SIZE - sizeof(double) - sizeof(unsigned long long)];
} telem;
telem histo[MAXHISTO];
 ...
#pragma omp parallel
{
 int id = omp_get_thread_num();
 histo[id]=0;
 for (int i=0; i<MAXELEM; i++)
   int pos = getpos (v[i], MAXHISTO); // returns a value between 0 and MAXHISTO-1
   if (id == pos) {
      histo [pos].total += v[i];
      histo [pos].num ++;
   }
 }
 ...
```

Surname, name .....

Answers to **Question 4, Section 2.c**

Command	Coherence actions			Presence bits	State in MM	State in cache			
	Core	Snoopy	Directory (yes/no)			cache0	cache1	cache2	cache3
$core_2$ reads histo[1].total									
$core_2$ writes histo[2].num									
$core_1$ reads histo[0].total									

Solution:

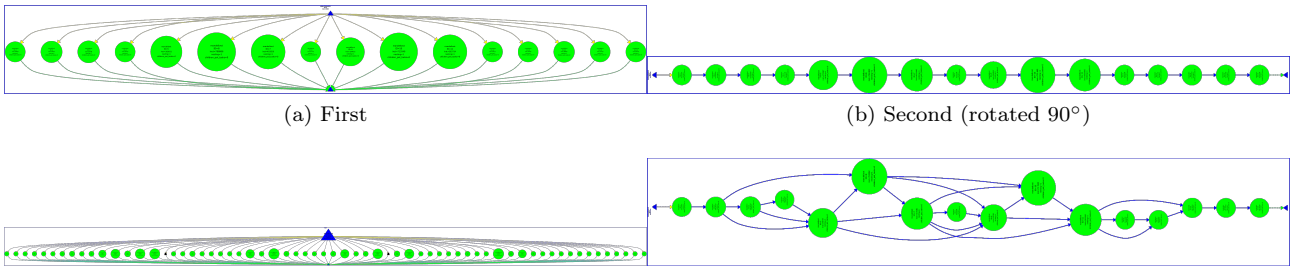
Command	Coherence actions			Presence bits	State in MM	State in cache			
	Core	Snoopy	Directory (y/n)			cache0	cache1	cache2	cache3
$core_2$ reads histo[1].total	$PrRd_2$	$BusRd_2$	y	10	S	-	-	S	-
$core_2$ writes histo[2].num	$PrWr_2$	$BusRdX_2$	y	10	M	-	-	M	-
$core_1$ reads histo[0].total	$PrRd_1$	$BusRd_1$, $Flush_2$	n	11	S	-	S	S	-

PAR – Final Exam Laboratory – Course 2023/2024-Q2

Student name: ..... Subgroup: .....

June 11th, 2024

Problem 1: Lab 3 (3.0 points) Given the following `Tareador` views:



(a) First

(b) Second (rotated 90°)

(c) Third

(d) Fourth (rotated 90°)

We ask you to:

1. (1 point) Indicate which parallel strategy corresponds to each figure. Briefly reason your answer.

```
- No comments are given because it is part of the lab sessions.
first: Coarse grain Iterative (original)
second: Coarse grain Iterative (original, executed using -d)
third: Leaf recursive
fourth: Coarse grain Iterative (original, executed using -h)
```

2. (1 point) Identify the data structures that provoke the data dependencies shown in the second and fourth figures, and how did you run the program in both cases with `run-tareador.sh`.

```
- No comments about the dependences shown are given because it is
  part of the lab sessions.
- Run: ./run-tareador.sh mandelbrot -h (fourth) o -d (second)
```

3. (1 point) Assuming the data dependencies of the previous question:

   (a) Can we make them disappear from `Tareador` views? If so, explain how.

```
% Yes, we can. Using tareador_disable_object(&variable)
```

   (b) How did you deal with those dependencies in Lab 4? Specify which OpenMP directives you used in order to preserve the correctness of your code.

```
- No comments are given because it is part of the lab sessions.
```

Problem 2: Lab 4 (4.5 points) Consider the following table with the speedups obtained from the execution of `submit-omp-strong.sh` script for each of the task decomposition implementations:

Number of threads	Speedup S1	Speedup S2	Speedup S3	Speedup S4
1	0.9	0.9	0.9	0.9
4	1.3	3.8	3.7	3.3
8	1.2	7.1	6.6	4.3
12	1.2	10.3	9.3	5.0
16	1.2	13.7	11.8	5.1
20	1.2	16.8	14.1	5.1

We ask you to answer the following questions:

1. (1 point) Identify the task decomposition that corresponds to each strategy ($S1 - S4$ in the table) based on the approximate speedups shown.

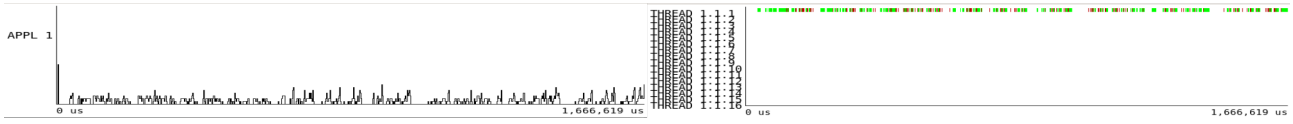
```
S1: Leaf Recursive
S2: Fine Grain Iterative
S3: Tree Recursive
S4: Original Coarse grain Iterative
```

2. (0.5 points) Briefly explain why the speedup of all task decompositions is less than 1 when using one thread.

   This is due to the parallel region overheads, synchronizations overheads, added to the fact we are using only one thread, which is sequential.

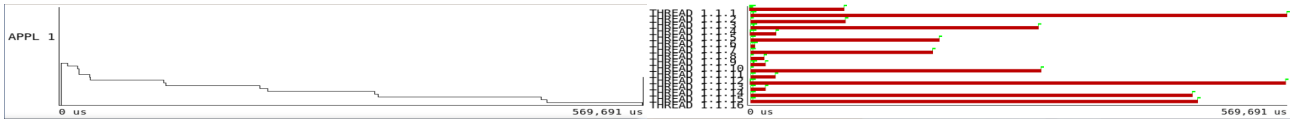
3. (1.5 points) Identify the task decomposition that corresponds to each pair of `Paraver` views for:

   (a) (0.5 points) an execution trace with 16 threads using the "instantaneous parallelism" and the "explicit tasks function created" hints. Briefly reason your answer.



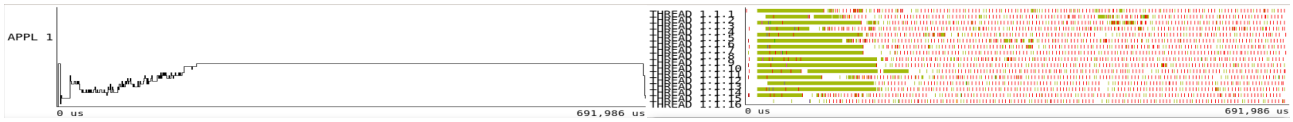
   Leaf recursive task decomposition.
   No comments are given because it is part of the lab sessions.

   (b) (0.5 points) an execution trace with 16 threads using the "instantaneous parallelism" and the "explicit tasks function executed" hints. Briefly reason your answer.



   Coarse grain (original) iterative task decomposition.
   No comments are given because it is part of the lab sessions.

   (c) (0.5 points) an execution trace with 16 threads using the "instantaneous parallelism" and the "explicit tasks function created" hints. Briefly reason your answer.



   Tree recursive task decomposition
   No comments are given because it is part of the lab sessions.

4. (1.5 points) Propose an OpenMP version of this portion of code in order to exploit the parallelism between the two for loops while maintaining the correctness of the subsequent code:

```
...
for (int px = x; px < x + TILE; px++) {
  M[y][px] = pixel_dwell(COLS, ROWS, CminR, CminI, CmaxR, CmaxI, px, y,...);
  M[y+TILE-1][px] = pixel_dwell(COLS, ROWS, CminR, CminI, CmaxR, CmaxI, px, y+TILE-1,...);
  equal = equal && (M[y][x] == M[y][px]);
  equal = equal && (M[y][x] == M[y + TILE - 1][px]);
}

for (int py = y; py < y + TILE; py++) {
  M[py][x] = pixel_dwell(COLS, ROWS, CminR, CminI, CmaxR, CmaxI, x, py,...);
  M[py][x+TILE-1] = pixel_dwell(COLS, ROWS, CminR, CminI, CmaxR, CmaxI, x+TILE-1,py,...);
  equal = equal && (M[y][x] == M[py][x]);
  equal = equal && (M[y][x] == M[py][x + TILE - 1]);
}

if (equal && M[y][x]==maxiter)
 { ...  }
else { ... }
```

```
No code is given because it is part of the lab sessions.
```

Problem 3: Lab 5 (2.5 points) Complete the following table with the information regarding the data decompositions done in the laboratory. You should fill the table with the data decompositions (second column) that have the characteristic of the first column, and briefly reason why in the last column. Note: S_p stands for the speedup using p threads.

Characteristic	Data Decomposition(s)	Reason Why
Load balancing for implicit tasks less than 50%		
Load balancing for implicit tasks is 100%, S_{20} is very high		
Load balancing for implicit tasks is 100%, but $S_{20} \leq S_{16}$		
It has the lowest number of L2 misses for 20 threads		
It has the highest number of L2 misses for 20 threads		
IPC is very low due to false sharing coherence protocol		
If `maxiter` is set to 1 it may have the same speedup as the best strategy		
It is difficult to affect its/their load balancing although we increase `maxiter`		

No comments are given in some parts because it is reported in the lab sessions.

Possible Solution:

Characteristic	Data Decomposition(s)	Reason Why
Load balancing for implicit tasks less than 50%	block by columns	
Load balancing for implicit tasks is 100%, S_{20} is very high	cyclic by rows	
Load balancing for implicit tasks is 100%, but $S_{20} \leq S_{16}$	cyclic by columns	
It has the lowest number of L2 misses for 20 threads	cyclic by rows	
It has the highest number of L2 misses for 20 threads	cyclic by columns	
IPC is very low due to false sharing coherence protocol	cyclic by columns	
If `maxiter` is set to 1 it may have the same speedup as the best strategy	block by columns	if the mandelbrot to be computed has not unbalance, it would not have unbalance
It is difficult to affect its/their load balancing although we increase `maxiter`	cyclic by rows or columns	they are very well balanced (1 row/1 column)

PAR – Final Exam Theory/Problems– Course 2024/25-Q1

January 16^{th}, 2025

Grade Publication January 22^{nd}, 2025 - Exam Review 23^{rd} - 16:00h–17:00h - C6E106

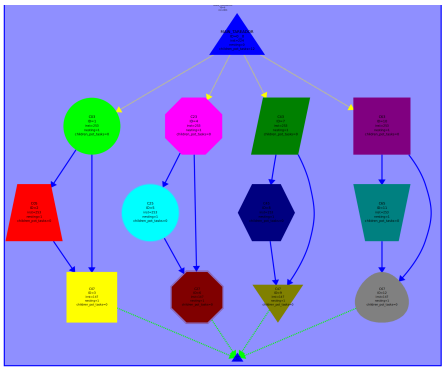
Problem 1 (2.5 points) Given the following C program instrumented with Tareador:

```
#define N 8
double M[N][N];

void main() {
char str[10];

  for (int ii=0; ii<N; ii+=2)
    for (int jj=3; jj<N; jj+=2) {
      sprintf(str,"C%d%d",ii,jj)
      tareador_start_task(str);
      for (int i=ii; i<ii+2; i++)
        for (int j=jj; j<min(jj+2,N); j++)
          M[i][j] = foo(M[i][j-1], M[i][j-2], M[i][j-3]);
      tareador_end_task(str);
    }
}
```

We ask you to:

1. Draw the TDG based on the above task definitions. Each task should be clearly labeled with the values for variables *ii* and *jj* at the time of task instantiation.

   Solution:



2. Consider that the cost of executing function `foo` is constant and equal to 10 t.u. and the cost of the rest of the code is negligible. Calculate T_1, P_{min} and T_∞.

   Solution:

   T1 = 400 t.u. (40*4+40*4+20*4) Pmin = 4 Tinf = T4 = 100 t.u. (40+40+20)

3. Consider now that the program is executed on a distributed memory architecture with 4 processors and that matrix M is initially stored in the memory assigned to P_0. Consider the data sharing model explained in class in which the latency of a remote memory access is $t_s + t_w \times m$, where t_s is 5 t.u. and t_w is 1 t.u. Function `foo` takes 10 t.u. as stated in the last question. We ask you to calculate T_4 assuming the best task scheduling policy. There is no need to move data back to P_0 once memory is modified.

   Solution:

   T_4 = 100 + 3x16 t.u. = 148 t.u.

   Tasks $C_{ii,jj}$ are assigned to processor $P_{ii/2}$. All tasks executing in P_0 have the data they need. Tasks $C_{2,3}$, $C_{4,3}$ and $C_{6,3}$ need data from P_0, the first 3 elements in each row to be processed. 3 elements has to be read from P_0 from two different rows, and therefore, two messages of 3 elements are required: $2 \times (t_s + 3 \times t_w)$, 16 t.u. Data has to be read sequentially from processor 0, and the critical path will be stablished by the time to receive the data at the last processor reading from processor 0 plus the time to process the three tasks assigned to that processor.

Problem 2 (2.5 points) A *convolution* is an important operation in machine learning and signal processing. A *filter* is a small matrix storing values useful for detecting some pattern in the input. Through the convolution the filter slides across the input data (a larger matrix) in a step-wise manner. At each step, an *element-wise multiplication and addition* is performed: at each position, the filter's values are multiplied by the corresponding values in the input data under it. These products are then summed to produce a single output value. The process is repeated across the entire input, generating an output matrix that highlights the presence of the filter's specific feature. Consider the following sequential program that performs a *convolution* using a 3×3 filter. The input matrix has a halo (initial and final row and column initialized to zeros). Thus, the input matrix has two more rows and columns than the output matrix:

```
#define PADDED_N(N) ((N) + 2) // Dimension of padded input matrix size including halo
void applyConvolutionIterative(int *input, int *output, int *filter, int N);

int main() {
    int N = 1000;               // Output matrix dimension
    int paddedN = PADDED_N(N);  //  Input matrix dimension

    // Allocate and initialize input and output matrices
    int *output = (int *) calloc(N * N, sizeof(int)); // initializes to 0's
    int *input  = (int *) malloc(paddedN * paddedN * sizeof(int));
    init_input_matrix(input);

    // Example of a 3x3 filter
    int filter[3 * 3] = { 0, 1, 0,     1, -4, 1,     0, 1, 0 };

    applyConvolutionIterative(input, output, filter, N);
    ...
    print_matrix(output);
    free(input); free(output);
    return 0;
}

void applyConvolutionIterative(int *input, int *output, int *filter, int N) {
    int row, col, sum;
```

```
    for (row = 1; row <= N; row++) {
        for (col = 1; col <= N; col++) {
            sum = 0;

            // Apply the 3x3 filter to a neighborhood of 3x3 elements in the input matrix:
            for (int fi = -1; fi <= 1; fi++) {
                for (int fj = -1; fj <= 1; fj++) {
                    int ni = row + fi; // Neighbor row index
                    int nj = col + fj; // Neighbor column index

                    int neighbor_idx = ni * PADDED_N(N) + nj; // Row-wise storage
                    int filter_idx = (fi + 1) * 3 + (fj + 1);
                    // Perform the element-wise multiplication and addition:
                    sum += input[neighbor_idx] * filter[filter_idx];
                }
            }
            output[(row - 1) * N + (col - 1)] = sum; // Row-wise storage
        }
    }
}
```

Note: When writing your solutions you do NOT need to write the whole code. Instead, write the minimum code excerpts that allows the reader to understand clearly how the parallelization has been done.

We ask you to: write a parallel version of routine `applyConvolutionIterative` using OpenMP's explicit tasks. The solution must try to exploit the maximum parallelism among threads while minimizing the data sharing synchronization and task creation overheads.

Solution: There are no loop-carried dependencies in this code. In order to be as efficient as possible:
1) work must be split by rows since row-wise storage is used for both the input and the output matrix: and 2) tasks must have an appropriate granularity, namely a chunk of approximately $\frac{N}{NT}$ consecutive rows being NT the number of threads being used, departing from a very fine grain granularity because no load unbalancing is expected since all iterations should take the same amount of associated work.

One possible solution using `#pragma omp taskloop`:

```
void applyConvolutionIterative(int *input, int *output, int *filter, int N) {
    int row, col, sum;

    #pragma omp parallel
    #pragma omp single
    #pragma omp taskloop private(col, sum) grainsize(N / omp_get_num_threads())
    for (row = 1; row <= N; row++) {
        for (col = 1; col <= N; col++) {
            sum = 0;

            // Apply the 3x3 filter
            ... // Not changed

            output[(row - 1) * N + (col - 1)] = sum;
        }
    }
}
```

Another possible solution using `#pragma omp task`:

```
void applyConvolutionIterative(int *input, int *output, int *filter, int N) {
    int row, col, sum;
```

```
    #pragma omp parallel
    #pragma omp single
      {
        int BS = N / omp_get_num_threads();
        for (int row_start = 1; row_start <= N; row_start += BS) {
            int row_end = (row_start + BS - 1 <= N) ? row_start + BS - 1 : N;
            #pragma omp task  private(row, col, sum)  // default firstprivate(row_start, row_end)
            for (row = row_start; row <= row_end; row++) {
                for (col = 1; col <= N; col++) {
                    sum = 0;

                    // Apply the 3x3 filter
                    ... // Not changed

                    output[(row - 1) * N + (col - 1)] = sum;
                }
            }
        }
      }
}
```

We have a recursive implementation which we also want to parallelize.

```
#define PADDED_N(N) ((N) + 2) // Dimension of padded input matrix size including halo
#define THRESHOLD 50          // Base case threshold

void applyConvolutionBaseCase(int *input, int *output, int *filter, int N,
                              int row_start, int row_end);

void applyConvolutionRecursive(int *input, int *output, int *filter, int N,
                               int row_start, int row_end);
int main() {
    // Same main as in the previous question except for the call to the recursive version

    ...
    // Apply convolution recursively
    applyConvolutionRecursive(input, output, filter, N, 1, N);
    ...
}

// Iterative base case for convolution
void applyConvolutionBaseCase(int *input, int *output, int *filter, int N,
                              int row_start, int row_end) {
    for (int row = row_start; row <= row_end; row++) {
        for (int col = 1; col <= N; col++) {
            int sum = 0;

            // Apply the 3x3 filter
            ... // Same as in the iterative code in the previous question.

            output[(row - 1) * N + (col - 1)] = sum; // Row-wise storage
        }
} }

// Recursive function to apply convolution
void applyConvolutionRecursive(int *input, int *output, int *filter, int N,
                               int row_start, int row_end) {
    if (row_end - row_start + 1 <= THRESHOLD) {
```

```
        applyConvolutionBaseCase(input, output, filter, N, row_start, row_end);
        return;
    }

    int mid = (row_start + row_end) / 2;

    applyConvolutionRecursive(input, output, filter, N, row_start, mid);
    applyConvolutionRecursive(input, output, filter, N, mid + 1, row_end);
}
```

We ask you to: write a parallel version of routine `applyConvolutionRecursive` using OpenMP's explicit tasks. The solution must be efficient, exploiting parallelism as soon as possible but not on the leaves of the recursivity. In addition the code must be prepared to control the creation of tasks based on the depth of the recursivity and a value stored in a constant named MAX_DEPTH.

Solution: In order to exploit parallelism as soon as possible we have to implement a *Tree* decomposition. The solution implements *cutoff* based on the depth of the recursivity.

```
#define MAX_DEPTH 4           // Maximum depth for task creation

// Iterative base case for convolution is not changed at all
...

// Recursive function to apply convolution
void applyConvolutionRecursive(int *input, int *output, int *filter, int N,
                               int row_start, int row_end, int depth) {
    if (row_end - row_start + 1 <= THRESHOLD) {
        applyConvolutionBaseCase(input, output, filter, N, row_start, row_end);
        return;
    }

    int mid = (row_start + row_end) / 2;

    if ( !omp_in_final() ) {
        #pragma omp task final(depth >= MAX_DEPTH)
        applyConvolutionRecursive(input, output, filter, N, row_start, mid, depth + 1);

        #pragma omp task final(depth >= MAX_DEPTH)
        applyConvolutionRecursive(input, output, filter, N, mid + 1, row_end, depth + 1);

    } else {
        applyConvolutionRecursive(input, output, filter, N, row_start, mid, depth + 1);
        applyConvolutionRecursive(input, output, filter, N, mid + 1, row_end, depth + 1);
    }
}


int main() {
    // Same main as in the previous question except for the call to the recursive version

    ...
    // Apply convolution recursively in parallel
    #pragma omp parallel
    #pragma omp single
    applyConvolutionRecursive(input, output, filter, N, 1, N, 0);
    ...
}
```

Problem 3 (2.5 points) Given the following code fragment computing matrix `A[N][N]`, an `N` much larger than the number of processors `P` (particularly `N` » `4P`).

```
#define N .. // a value much larger than P
#define k N/4

float A[N][N], B[N][N];
...
for (int i = k; i < N-k; i++)
    for (int j = i; j < N-k; j++)
         A[i][j] = foo (A[i][j-k], A[i][j], B[i-k][j]); // computation function
```

We ask you to: Decide the most appropriate *Input/Output Geometric Data Decomposition* for matrices `A` and `B` and write a parallel version of the code by adding the *OpenMP* necessary directives and clauses, making use of only implicit tasks. Your solution proposal should:

- Minimize the synchronization overhead among implicit tasks
- Minimize the load unbalance
- Maximize data locality

Solution:

The best option is a geometric cyclic data decomposition of matrix A and matrix B by rows along the P processors, starting allocation on processor 0, of row k for matrix A, and of row 0 (i-k) for matrix B. This will help to exploit data locality and avoid data dependence synchronization.

```
#define N .. // a value much larger than P
#define k N/4

float A[N][N], B[N][N];
...
#pragma omp parallel num_threads(P)
{
 int myid = omp_get_thread_num();

 for (int i = k+myid; i < N-k; i += P)
    for (int j = i; j < N-k; j++)
         A[i][j] = foo (A[i][j-k], A[i][j], B[i-k][j]);
}
```

Surname, name .....

Problem 4 (2.5 points)

Assume the definition and allocation of matrices of the previous exercise. In addition, consider a shared-memory multiprocessor system composed by two NUMA nodes, each with two processors (cores) and 64 GBytes of shared main memory (MM). Each core has a private cache of 32 MBytes. Memory lines are 64 bytes long (one memory line per memory page), the allocation in memory for matrix `A` and matrix `B` are aligned to the start of a memory line and `sizeof(float)` is 4 bytes.

Cores 0 and 1 belong to NUMA node 0 and cores 2 and 3 are allocated in NUMA node 1. Data coherence is maintained using Write-Invalidate MSI protocols, with a Snoopy attached to each cache memory to provide coherency within each NUMA node and Write-Invalidate MSU directory-based coherence among the two NUMA nodes.

Assume now that matrix `A` is entirely allocated in the MM of Numa Node 0, and all cache memories of the multiprocessor system are empty, fill in the attached table with the sequence of processor commands (Core), bus transactions within NUMA nodes (Snoopy), transactions yes/no between NUMA nodes (Directory), the presence bits, state for each cache and memory line, to keep cache coherence, AFTER the execution of each of the following sequence of commands:

Memory access	Core	Coherence commands		Presence bits	State in MM	State in cache			
		Snoopy	Directory (yes/no)			cache0	cache1	cache2	cache3
$core_0$ reads `A[0][0]`									
$core_3$ reads `A[0][5]`									
$core_1$ writes `A[0][15]`									
$core_2$ writes `A[1][15]`									

Solution:

Memory access	Core	Coherence commands		Presence bits	State in MM	State in cache			
		Snoopy	Directory (yes/no)			cache0	cache1	cache2	cache3
$core_0$ reads A[0][0]	$PrRd_0$	$BusRd_0$	*no*	01	S	S	-	-	-
$core_3$ reads A[0][5]	$PrRd_3$	$BusRd_3$	*yes*	11	S	S	-	-	S
$core_1$ writes A[0][15]	$PrWr_1$	$BusRdX_1$	*yes*	01	M	I	M	-	I
$core_2$ writes A[1][15]	$PrWr_2$	$BusRdX_2$	*yes*	10	M	-	-	M	-

PAR – Final Exam Laboratory – Course 2024/2025-Q1

Student name: ..... Subgroup: .....

January 16th, 2025

Grade Publication January 22^{nd}, 2025 - Exam Review 23^{rd} - 16:00h–17:00h - C6E106

Disclamer: Only few solutions are given since all the answers are part of the practices done during the lab sessions.

Problem 1: Lab 3 (3.0 points) Given the following excerpt of the instrumentation with `Tareador` of the iterative `mandelbrot` code:

```
for (int py = y; py < y + TILE; py++) {
   tareador_start_task("mandelbrot_py_fill");
   for (int px = x; px < x + TILE; px++) {
      M[py][px] = M[y][x];
      if (output2display) {
         if (setup_return == EXIT_SUCCESS) {
             XSetForeground(display, gc, color);
             XDrawPoint(display, win, gc, px, py);
   } } }
   tareador_end_task("mandelbrot_py_fill");
}
```

We ask you to: Draw the Task Dependency Graph (TDG) corresponding to this part of the code, assuming $TILE = 4$ and the mandelbrot program is executed with `Tareador` and: case 1) no arguments and case 2) `-d` argument. For each TDG, and in case you find dependencies, indicate if they are due to data sharing and how you would deactivate them using `Tareador` API. Finally, how will you protect or avoid them, if necessary, in your OpenMP implementation?.

Solution:

No comments are given because it is part of the lab sessions.

Problem 2: Lab 4 (4.0 points) After doing the analysis with `Modelfactors` and `Paraver`, the speedup and efficiency (speedup, efficiency in %) of three of the four `Mandebrot` iterative and recursive parallel implementations in lab4, when running with 16 threads, are: 1) (1.1x,7%), 2) (4.4x,27%) and 3) (13.0x,80%) approximately. As a reminder, the names of the four implementations are: Iterative `Tile`, Iterative `Fine Grain`, Recursive `Leaf` and Recursive `Tree`. **We ask you to answer the following questions:**

1. (1.0 points) Which implementation is executed to obtain each result? Briefly reason your answer.

   Solution:

   No comments are given because it is part of the lab sessions. 1) Leaf Recursive, 2) Tile (original), 3) Fine grain Iterative.

2. (1.5 points) For the fine grain task decomposition, the recursive tree task decomposition and the recursive leaf task decompositon, assume you visualize a `Paraver` execution trace for each of them using `OpenMP tasking/explicit tasks function created and executed` hint. How many threads do you expect to find creating tasks for each implementation? Briefly reason your answer.

   Solution:

   No comments are given because it is part of the lab sessions.

3. (1.5 points) For the tile (original) task decomposition, the recursive tree task decomposition and the recursive leaf task decompositon, draft a picture of the expected Instantaneous Parallelism view using 16 threads. Briefly explain your pictures.

   Solution:

   No comments are given because it is part of the lab sessions.

Problem 3: Lab 5 (3.0 points) Assuming the implementation of `Mandelbrot` using a 1D cyclic data decomposition by rows, **we ask you to answer the following questions:**

1. (0.25 points) What is the value of *phi* you expect to obtain with `Modelfactors`?

   Solution:

   No comments are given because it is part of the lab sessions.

2. (0.5 points) Assume the loop body of the inner-most loop of the mandel function to be: `M[py][px]=...` (you do not have to write the code to display the image and to update histogram, just write "..."). Write the excerpt of the parallel code of the 1D cyclic data decomposition by rows of `mandel` function.

   Solution:

   No comments are given because it is part of the lab sessions.

3. (0.75 points) What is the approximate expected order of magnitude in the number of $l2$ misses of your code using 20 threads? Do you think a 1D cyclic data decomposition **by columns** will be more efficient thant yours in number of $l2$ misses? Briefly reason your answer.

   Solution:

   No comments are given because it is part of the lab sessions. However, it cannot be better than the version by columns because the version by columns will have much more number of l2 misses; it will not exploit the spatial locality at all.

4. (0.25 points) What is the approximate expected % load balancing (`Modelfactors` metric) of your code using 20 threads? Briefly reason your answer.

   Solution:

   No comments are given because it is part of the lab sessions.

5. (0.5 points) What is the approximate expected speedup with 20 threads? And, if we decide to use 21 threads, instead of the 20 threads, which is the expected speedup increment? Briefly reason your answer.

   Solution:

   No comments are given because it is part of the lab sessions. Remember lab1: with 21 threads we will have to share one physical core and we saw that this is not good for performance.

6. (0.75 points) Are 1D block-cyclic data decomposition by columns or 1D block data decomposition by columns strategies more globally efficient (`Modelfactors` metric) than 1D cyclic data decomposition by rows? Why? Briefly reason your answer.

   Solution:

   No comments are given because it is part of the lab sessions.

PAR – Final Exam Theory/Problems – Course 2024/25-Q2

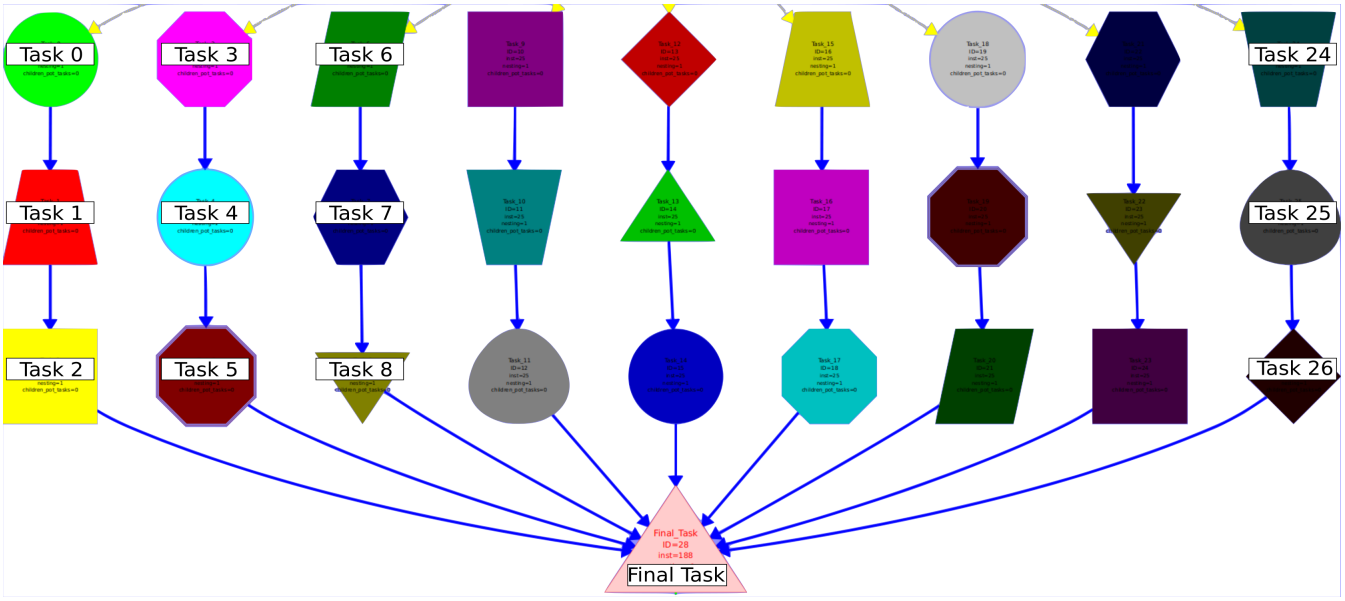
Grade Publication June 20^{th}, 2025 - Exam Review 25^{th} - 15:00h–16:00h - C6E101
June 10^{th}, 2025

Grade Publication June 20^{th}, 2025

Reviews
Theory Exam Review 25^{th} - 15:00h–16:00h - C6E101
Lab Exam Review (all groups but 12, 21, 23, 43) 25^{th} - 15:00h–16:00h - C6E101
Lab Exam Review (group 43) 25^{th} - 16:00h–17:00h - C6E101
Lab Exam Review (groups 12, 21, 23) 26^{th} - 11:00h–12:00h - C6E101

Problem 1 (2.0 points)

We have instrumented a code with `Tareador` and we have obtained the following TDG (with some labels manually enlarged):



However, due to a backup error, part of the code has been lost. We ask you to:

1. (0.5 points) Complete the following code to achieve the same TDG. We ask you to fix "??" of the code and map M1 and M2 to one matrix: A or C.

```
#define N 3
int C[N][N], A[N][N], B[N][N];
/* Init ... */
int CountTask = 0;
tareador_disable_object(&CountTask);

for (int i=0; i<N; i+=??)
  for (int j=0; j<N; j+=??)
    for (int k=0; k<N; k+=??)
    {
       sprintf(taskname,"Task_%d",CountTask++);
       tareador_start_task(taskname);
       M1[??][??]+= M2[i][k]*B[j][k];
```

```
        tareador_end_task(taskname);
    }
tareador_start_task("Final_task");
for (int i=0; i<N; i++)
  for (int j=0; j<N; j++)
     printf("C[i][j]:%d\n",C[i][j]);
tareador_end_task("Final_task");
/*...*/
```

2. (0.75 points) Assuming that each task `Task_number` has a cost of t_c and the `Final_task` $N \times N \times 10 \times t_c$, compute T_1, T_∞, and P_{min} as a function of N.

3. (0.75 points) Write the time diagram and the expression that determines the execution time T_3, clearly indicating the contribution of the computation time T_3^{comp} and data sharing overhead T_3^{mov}. You can assume: 1) a distributed-memory architecture with 3 processors; 2) matrices A and C are initially distributed by rows ($N/3$ consecutive rows per processor, being N multiple of 3) and matrix B is replicated; 3) the scheduling of `Task_number` follows the owner-computes rule, i.e. it executes on the processor where the data it processes is allocated; 4) once the nested loops are finished, `Final_task` is executed on processor 0; 5) data sharing model with $t_{comm} = t_s + m \times t_w$, being t_s and t_w the start-up time and transfer time of one element, respectively; and 6) the task costs of the previous exercise.

Solution:

1. (0.5 points) Complete the following code to achieve the same TDG. We ask you to fix "??" of the code and map M1 and M2 to one matrix: A or C.

   M1 maps to C. M2 maps to A.

```
#define N 3
int C[N][N], A[N][N], B[N][N];
/* Init ... */
int CountTask = 0;
tareador_disable_object(&CountTask);

for (int i=0; i<N; i+=1)
  for (int j=0; j<N; j+=1)
    for (int k=0; k<N; k+=1)
    {
       sprintf(taskname,"Task_%d",CountTask++);
       tareador_start_task(taskname);
       C[i][j]+= A[i][k]*B[j][k];
       tareador_end_task(taskname);
    }
tareador_start_task("Final_task");
for (int i=0; i<N; i++)
  for (int j=0; j<N; j++)
     printf("C[i][j]:%d\n",C[i][j]);
tareador_end_task("Final_task");
/*...*/
```

2. (0.75 points) Assuming that each task `Task_number` has a cost of t_c and the `Final_task` $N \times N \times 10 \times t_c$, compute T_1, T_∞, and P_{min} as a function of N.

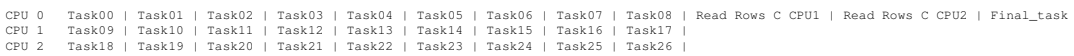
   $T_1 = N^3 \times t_c + N \times N \times 10 \times t_c$

   $T_\infty = N \times t_c + N \times N \times 10 \times t_c$

   $P_{min} = N^2$

3. (0.75 points) Write the time diagram and the expression that determines the execution time T_3, clearly indicating the contribution of the computation time T_3^{comp} and data sharing overhead T_3^{mov}. You can assume: 1) a distributed-memory architecture with 3 processors; 2) matrices A and C are initially distributed by rows ($N/3$ consecutive rows per processor, being N multiple of 3) and matrix B is replicated; 3) the scheduling of `Task_number` follows the owner-computes rule, i.e. it executes on the processor where the data it processes is allocated; 4) once the nested loops are finished, `Final_task` is executed on processor 0; 5) data sharing model with $t_{comm} = t_s + m \times t_w$, being t_s and t_w the start-up time and transfer time of one element, respectively; and 6) the task costs of the previous exercise.

   Time diagram for N=3.



   $T_3 = T_3^{comp} + T_3^{mov}$

   $T_3^{mov} = 2 \times (t_s + N^2/3 \times t_w)$

   $T_3^{comp} = (N^3/3) \times t_c + N \times N \times 10 \times t_c$

Problem 2 (3.0 points) Given the following C code:

```
#define N 1048576
#define MIN(a,b) (a<b?a:b)
#define NELEMS X /* a value between min_size and N */

int min_size = 16;
int nbranches = 5; /* any value, not necessarily a submultiple of N */

/* Performs heavy computations. */
/* Does not modify any variable other than the returned result. */
int compute_base (int *v, int n);

int compute_rec (int *v, int n) {
    int res = 0;
    if (n < min_size) {
        res = compute_base (v, n);
    }
    else {
        int partition_size = n/nbranches;
        for (int i = 0; i < n; i += partition_size) {
            int n_local = MIN(partition_size, n-i);
            res += compute_rec (&v[i], n_local);
        }
    }
    return res;
}
int main() {
    int v[N];
    ...
    int x = compute_rec (v, N);
    ...
}
```

We ask you to: Add the necessary OpenMP directives and clauses in order to parallelize the code following a *Recursive Tree Task Decomposition* strategy. Your implementation should avoid creating more tasks when processing fewer than `NELEMS` elements.

Solution:

```
#define N 1048576
#define MIN(a,b) (a<b?a:b)
#define NELEMS X /* some value between min_size and N */

int min_size = 16;
int nbranches = 5;

/* Performs heavy computations. */
/* Does not modify any variable other than the returned result. */
int compute_base (int *v, int n);

int compute_rec (int *v, int n) {
    int res = 0;
    if (n < min_size) {
        res = compute_base (v, n);
    }
    else {
        int partition_size = n/nbranches;
        #pragma omp taskgroup task_reduction (+:res)
        {
         for (int i = 0; i < n; i + =partition_size) {
            int n_local = MIN(partition_size, n-i);
            if (!omp_in_final())
                #pragma omp task in_reduction (+:res) final (n_local <= NELEMS)
                res += compute_rec (&v[i], n_local);
            else
                res += compute_rec (&v[i], n_local);
         }
        }
    }
    return res;
}
int main() {
    int v[N];
    ...
    #pragma omp parallel
    #pragma omp single
    int result = compute_rec (v, N);
    ...
}
```

Problem 3 (2.0 points)

Assume a multiprocessor system composed of three NUMA nodes (labeled *NUMAnode0*, *NUMAnode1* and *NUMAnode2*), each with 8 GB of main memory shared by two sockets (labeled *socket0* and *socket1* for *NUMAnode0*, *socket2* and *socket3* for *NUMAnode1*, and *socket4* and *socket5* for *NUMAnode2*). Each socket labeled *socketX* has one core (labeled *coreX*) with a cache memory of 8 MB. **The cache line size is 32 bytes**. Data coherence in the system is maintained using **Write-Invalidate MSI protocols**, with a **Snoopy attached to each cache memory** to provide coherency within each NUMA node and **directory-based coherence among the three NUMA nodes**.

We ask you to

1. (1.0 points) Compute the amount of bits taken by each snoopy to maintain the coherence between caches inside a NUMA node and, the amount of bits in each node directory to maintain the coherence among NUMA nodes.
2. (1.0 points) Given the code below and assuming that: 1) the Operating System decides the data allocation using a "first touch" policy at the memory page level and that each memory page contains a single memory line; 2) vector x is the only variable that will be stored in memory (the rest of variables will be all in registers of

the processors); 3) the initial address of vector x is aligned with the start of a cache line; 4) the size of an int data type is 4 bytes; and 5) $thread_i$ always executes on $core_i$. How many directory entries and NUMAnode homes will be used to store vector x once you reach the omp barrier? Is there any extra cache coherence overhead during the update of the data of vector x if we cyclically distribute the work to be done with vector x between 6 threads after the barrier?

```
#define N (36)
int x[N];
...
#pragma omp parallel num_threads(6)
{
    int myid = omp_get_thread_num();
    /* Init Code */
    if (myid==0)
      for (int i=0; i<N; i++) x[i]=init();
    /* End Init Code */
    #pragma omp barrier

    /* All 6 threads cyclically update vector x */
    ...
}
```

Solution:

1. (1.0 points) Compute the amount of bits taken by each snoopy to maintain the coherence between caches inside a NUMA node and, the amount of bits in each node directory to maintain the coherence among NUMA nodes.

   Snoopy: Each cache has $8MB$ bytes, that is $2^3 \times 2^{20}$ bytes. Each cache line is 32 bytes, that is 2^5 bytes. Each cache line needs 2 bits to keep M, S, I coherence status. Then, the number of bits per snoopy, required to keep the coherence, is the number of cache lines multiplied by 2: $2^{23}/2^5 \times 2 = 2^{19}$ bits per cache.

   Directory: Each NumaNode has a memory of $8GB$ bytes, that is $2^3 \times 2^{30}$ bytes. Each memory line is 32 bytes, that is 2^5 bytes. Each directory entry needs 2 bits to keep M, S, U, and 3 bits for the shared list (one per Numa Node). Then, the number of bits per directory, required to keep the coherence, is the number of memory lines multiplied by $2+3$: $2^{33}/2^5 \times (2+3) = 5 \times 2^{28}$ bits per directory.

2. (1.0 points) Given the code below and assuming that: 1) the Operating System decides the data allocation using a "first touch" policy at the memory page level and that each memory page contains a single memory line; 2) vector x is the only variable that will be stored in memory (the rest of variables will be all in registers of the processors); 3) the initial address of vector x is aligned with the start of a cache line; 4) the size of an int data type is 4 bytes; and 5) $thread_i$ always executes on $core_i$. How many directory entries and NUMAnode homes will be used to store vector x once you reach the omp barrier? Is there any extra cache coherence overhead during the update of the data of vector x if we cyclically distribute the work to be done with vector x between 6 threads after the barrier?

   ```
   #define N (36)
   int x[N];
   ...
   #pragma omp parallel num_threads(6)
   {
       int myid = omp_get_thread_num();
       /* Init Code */
       if (myid==0)
         for (int i=0; i<N; i++) x[i]=init();
       /* End Init Code */
       #pragma omp barrier

       /* All 6 threads cyclically update vector x */
       ...
   }
   ```

   How many directory entries and NUMAnode homes will be used to store vector x once you reach the omp barrier?

The number of directory entries, required for vector x, is the number of memory lines it needs to be stored. The number of bytes of vector x is 4 bytes per element multiplied by 36 elements; 144 bytes total. The cache line is 32 bytes. Then, vector x needs $144/32 = 4.5 \rightarrow 5$ lines of memory, that is 5 entries of the directory. All these memory lines will be in Numa Node 0 since thread 0, in core 0, computes the init code before the barrier. Then, following first-touch policy the data goes to Numa Node 0.

Is there any extra cache coherence overhead during the update of the data of vector x if we cyclically distribute the work to be done with vector x between 6 threads after the barrier?

There is a False Sharing problem since each thread will write in consecutive memory positions of the same cache line, provoking invalidations.

Problem 4 (3.0 points) We have a code that reads an input vector, computes the square of each element and writes the result in the corresponding position of an output vector:

```
#define N 100000003  // Arbitrary size (not divisible by thread count)

int  vin[N];
int vout[N];

  ...
  for (int i = 0; i < N; i++) {
      vout[i] = vin[i] * vin[i];  // Square each element
  }
  ...
```

Assuming the beginning of both vectors are aligned to a cache line boundary, and knowing that we can use a routine that returns the number of bytes in a cache line:

```
int get_cache_line_size() {
    int size = sysconf(_SC_LEVEL1_DCACHE_LINESIZE);
    return (size > 0) ? size : 64;  // Default to 64 bytes if query fails
}
```

we ask you to write:

1. an *OUTPUT block data decomposition* which ensures load balancing.
   Solution:

   ```
   #pragma omp parallel
   {
       int tid = omp_get_thread_num();
       int  nt = omp_get_num_threads();

       int BS = N / nt;
       int extra_iterations = N % nt;

       // Distribute excess workload if it exists
       int start = tid * BS + (tid < extra_iterations ? tid : extra_iterations);
       int end = start + BS + (tid < extra_iterations ? 1 : 0);

       for (int i = start; i < end; i++) {
           vout[i] = vin[i] * vin[i];
       }
   }
   ```

2. an *OUTPUT block-cyclic data decomposition* which maximizes exploitation of spatial locality.
   Solution:

   ```
   #define min(a, b) ( (a < b) ? a : b )
   int cache_line_size = get_cache_line_size();
   ```

```
int BS = cache_line_size / sizeof(int); // Block size oriented towards cache lines

#pragma omp parallel
{
    int tid = omp_get_thread_num();
    int  nt = omp_get_num_threads();

    for (int ii = tid * BS; ii < N; ii += nt * BS) {
        for (int i = ii; i < min(ii+BS, N); i++) {
            vout[i] = vin[i] * vin[i];
        }
    }
}
```

3. We want to have a variant of the codes above that, additionally, returns the sum of all the squares computed in the loop. We have parallelized the code assuming that 1) we are interested in keeping the partial results computed by each thread beyond the span of the parallel region; and 2) the number of threads being used is kept in variable NUM_THREADS. The following code excerpt focuses on the code added for the reduction:

```
int sums[NUM_THREADS]; /* Data structure used to store partial results */

...
for (int i = 0; i < NUM_THREADS; i++) {
    sums[i] = 0;  // Initialize partial result for each thread
}
#pragma omp parallel
{
    int tid = ... /* Retrieves the identifier of the thread */

    ...
       for ( /* Inner loop */) {
         vout[i] = vin[i] * vin[i];  // Square each element
         sums[tid] += vout[i];
       }
    ...
}
...
/* Here data structure sums holds the partial results computed by each thread. */
```

We have observed that after adding the update of array `sums` in the inner loop the resulting parallel execution is slower than expected. For this reason we ask you to propose an alternative data structure for `sums` that allows us to keep the partial results computed by each thread while maximizing parallel performance. You only need to show the definition of the data structure, but have to define it precisely. However, you do not have to write the whole code using it.

Solution:

The code above suffers *False Sharing* when threads update `sums` in parallel. In order to avoid it we need to ensure that each thread will have exclusive access to a cache line when updating its partial result. Thus, we apply *padding* by defining a 2-dimensional array in which each row of the new array `sums` has the number of elements that fits in a cache line:

```
#define BYTES_PER_CACHE_LINE 64
#define ELEMS_PER_CACHE_LINE BYTES_PER_CACHE_LINE / sizeof(int)

int sums[NUM_THREADS][ELEMS_PER_CACHE_LINE]

... sums[tid][0] ...
```

PAR – Final Exam Laboratory – Course 2024/25-Q2

Student name: ..... Subgroup: .....

June 10th, 2025

Grade Publication June 20^{th}, 2025

Reviews

Theory Exam Review 25^{th} - 15:00h–16:00h - C6E101
Lab Exam Review (all groups but 12, 21, 23, 43) 25^{th} - 15:00h–16:00h - C6E101
Lab Exam Review (group 43) 25^{th} - 16:00h–17:00h - C6E101
Lab Exam Review (groups 12, 21, 23) 26^{th} - 11:00h–12:00h - C6E101

Problem 1: Lab 1-3 (3 points) Answer the following questions:

1. (0.25 points) Is the instrumented code with `Tareador` executed in parallel by `Tareador` to detect the task dependencies? Briefly reason your answer.

   Solution: Tareador does not run the program in parallel. It sequentially runs the program taking care of loads and stores in the programmer tasks.

2. (1.5 points) How did you detect the dependences due to data sharing in the Original Parallel Strategy using the `Tareador` GUI? Please, enumerate the steps you did to figure them out.

   Partial Solution: 1) we run the program instrumented with Tareador with Tareador. 2) We select the task we want to identify the input dependences. 3) Rigth-click in the mouse and select edge-in input. 4) Then, we select the source task of the edge-in and look at the intersection information. 5) Finally, we can look at the information of where (line of code) this variable/memory position was declared or allocated. With this, we can look at the code and figure out the data dependency. No more comments are given because it is part of the lab sessions.

3. (1.25 points) What should you do in your `Tareador` instrumented program to make vertical and horizontal check `Tareador` tasks appear without dependences between them, while keeping the necessary dependences with the rest of the program? Please, write a pseudocode with the main modifications you did. Do not worry about the exact code of the vertical and horizontal tasks, but you should specify the variable/s that provoke the depedences.

   Solution:

   No comments are given because it is part of the lab sessions.

Problem 2: Lab 5 (3 points) **Fill in the following table** for each data decomposition assuming an execution with 16 threads:

Data Decomposition Characteristics	Cyclic by rows	Block-cyclic by columns	Block by columns
Computational Load Balance (Possible answers: Bad, Medium, Good)			
Data Distribution Load Balance (Possible answers: #elements unbalance)			
Order in Explotation of Data Locality (Possible answers: 1, 2, 3) (Possible type answers: Spatial, Temporal)			
False Sharing (Possible answer: There is, There is not)			

In addition, we ask you to:

Briefly explain the answers to `Data Decomposition Load Balance` and `False sharing`. Remember that `ROWS` and `COLS` were defined as 5120, and assume that the `Mandebrot` matrix has its first element aligned to cache line .

Solution No comments are given because it is part of the lab sessions.

Problem 3: Lab 4 (4 points) Consider the following table with the elapsed time results obtained from the execution of `submit-omp-strong.sh` script for each of the task decomposition implementations:

Number of threads	Original iter	Fine Grain Iter	Leaf Recursive	Tree Recursive
1	3.05	3.05	1.58	1.58
4	0.90	0.77	1.23	0.41
8	0.69	0.42	1.36	0.24
12	0.70	0.29	1.36	0.18
16	0.70	0.22	1.36	0.14
20	0.70	0.17	1.36	0.12

We ask you to:

1. For the strategies *Original Iter* and *Recursive Leaf*, complete the table below by identifying the main performance issues you observed (excluding scalability). Additionally, specify which experimental result provided you evidence for each of the identified issues.

Strategy	**Performance problem**	**Experiment that evidences it**	**How did you identify the issue in the experiment?**
Original Iter			
Leaf Recursive			

Table 1: Problem 2.1: Complete

Solution:

No comments are given because it is part of the lab sessions.

2. For the strategies Fine Grain Iter and Recursive Tree, complete the table below by explaining how each one improves upon its counterpart (Original Iter and Recursive Leaf, respectively). Additionally, indicate which experimental result supports each of the identified improvements.

Solution:

No comments are given because it is part of the lab sessions.

Strategy	**Cause of performance improvement**	**Experiment that evidences it**
Fine Grain Iter vs Original Iter		
Tree Recursive vs Leaf Recursive		

Table 2: Problem 2.2: Complete